

★★★ 반드시 내 것으로 ★★★

#MUSTHAVE

27년 개발, 12년 강의 노하우를 담은 자바 입문서가 나타났다

이재환의 자바 Java 프로그래밍 입문

설치 없이 익히는 《선수 수업》부터 딱 필요한 만큼 핵심 문법과 미니 프로젝트까지

이재환 지음

취업에
필요한
핵심만 딱

별책 부록
《선수 수업》
제공

3가지
미니 프로젝트

설치 없이 손으로 익히기

선수 수업

#MUSTHAVE

☐ 학습 목표	자바 선수 수업은 무작정 자바 코드를 보고 실행해보면서 프로그램이 어떻게 돌아가는지 눈과 손으로 확인하고 흥미를 유발하는 데 목적을 둡니다. 다른 언어를 익힌 현업 프로그래머라면 선수 수업을 건너뛰고 본책의 0장 자바 개발 환경 구축으로 이동해도 좋습니다. 프로그래밍 입문자라면 하나하나 타이핑해가며 실습하기 바랍니다.
☐ 학습 순서	1 왜 선수 수업을 할까요? 2 온라인에서 설치 없이 자바 사용하기 3 내용 출력하기 4 변수 사용하기 5 산술 연산자 사용하기 6 조건문 사용하기 7 while문 사용하기 8 for문 사용하기 9 반복문의 중첩 10 반복문 고급 사용 11 사용자 입력받기 12 사용자 입력받아 사칙연산 결과 출력하기



알려드려요

선수 수업은 문턱을 낮춰 코딩과 친숙해질 수 있게 전문 용어와 기술적인 내용을 최대한 피했습니다(전문 용어와 깊이 있는 설명은 본책에서 다룹니다).

1 왜 선수 수업을 할까요?

책과 인강 등으로 자바를 배울 때 대부분의 초보자들은 다음과 같은 과정을 거칩니다.

- 1 먼저 JDK를 내려받아 설치
- 2 윈도우의 환경 변수에 자바 관련 정보 설정
- 3 이클립스를 내려받아 설치
- 4 자바 프로젝트를 만들고 “Hello World!” 출력

프로그래머가 아니더라도 컴퓨터를 조금이라도 능숙하게 사용하던 분이라면 별일 아니라고 생각하지만 이 과정에서도 적지 않은 입문자가 정말 많이 힘들어 합니다. 프로그램을 배우고 싶은 사람 모두가 컴퓨터에 익숙한 것은 아니기 때문에 이해가 되는 부분이기도 합니다.

실제로 직업훈련기관에서 수업을 할 때 컴퓨터에 익숙하지 않은 학생 몇몇은 앞에서 제가 하는 것을 보면서 따라 하는 데도 힘들어 합니다. 그러니 혼자 책을 보면서 공부를 한다거나 동영상 보면서 공부를 하는 경우에는 더 힘들어 할 수도 있겠다 싶습니다.

그래서 이 선수 수업을 준비했습니다. 컴퓨터를 잘 활용하지 못하더라도 프로그래밍을 배우고 싶다면, 웹 브라우저만 켜고 사용할 수 있다면 쉽게 배울 수 있도록 구성해보았습니다.

설치 등의 과정은 다 빼고 먼저 사용부터 하겠습니다. 설치부터 힘들어 하기 때문에 설치 없이 사용에 집중하려는 겁니다. 그리고 프로그래밍에 필요한 이론도 최소한만 설명해드리겠습니다. 우리가 태어나서 국어 문법 다 배우고 문법에 맞게 한국어를 사용하는 것이 아닌 것처럼, 자바도 최소한의 문법만으로도 사용할 수 있습니다.

반복해서 사용하면서 익숙해지고, 흥미가 생기는 게 먼저입니다. 그 이후에 직접 설치해 개발 환경도 구성해보고 이론도 자세히 공부하겠습니다. 일단 사용해보면서 코딩에 친숙해지고 몇 가지라도 용어를 익힌다면 목표 달성입니다.

2 온라인에서 설치 없이 자바 사용하기

선수 수업에서는 PC에 자바 개발 환경을 설치하는 대신, 온라인 자바 컴파일 서비스를 제공하는 리플잇repl.it을 활용합니다. 리플잇에서 자바 코드를 입력하고 컴파일하고 실행해보는 방법을 알아 보겠습니다.

학습 순서

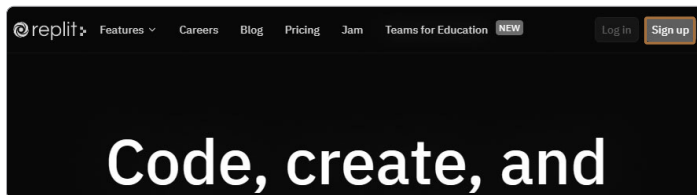
- 1 리플잇 가입
- 2 리플잇 화면 구성 소개
- 3 리플잇에서 자바 코드 컴파일 및 실행

2.1 리플잇 가입

리플잇에 가입하고 계정을 생성하겠습니다. 계정 생성 없이 바로 사용할 수도 있지만, 자신이 만든 코드를 저장하고 보관하려면 계정을 생성하여 로그인을 하고 사용하는 것이 좋습니다.

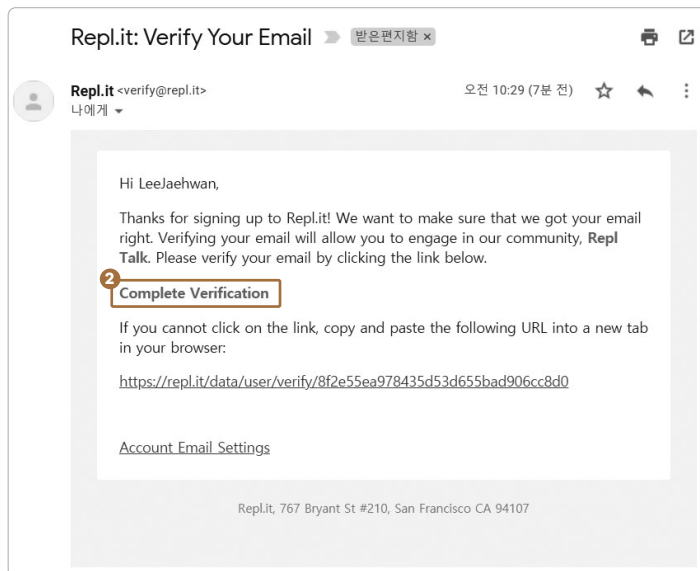
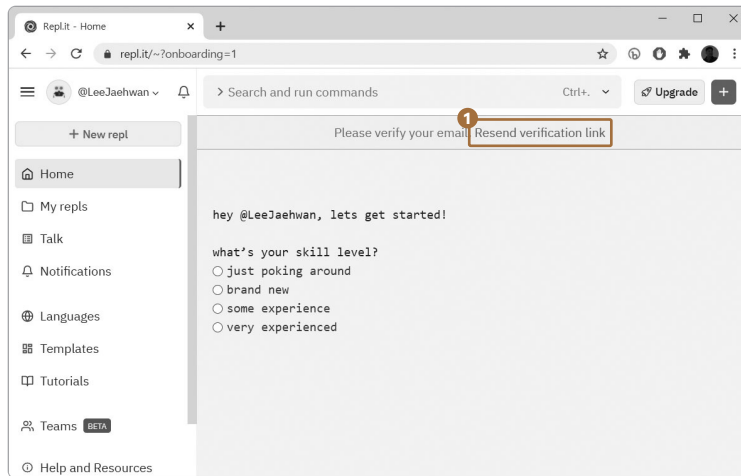
To Do 01 리플잇(repl.it)에 접속합니다.

02 [Sign up] 버튼을 클릭합니다. 현재 화면에서는 우측 상단에 있습니다.

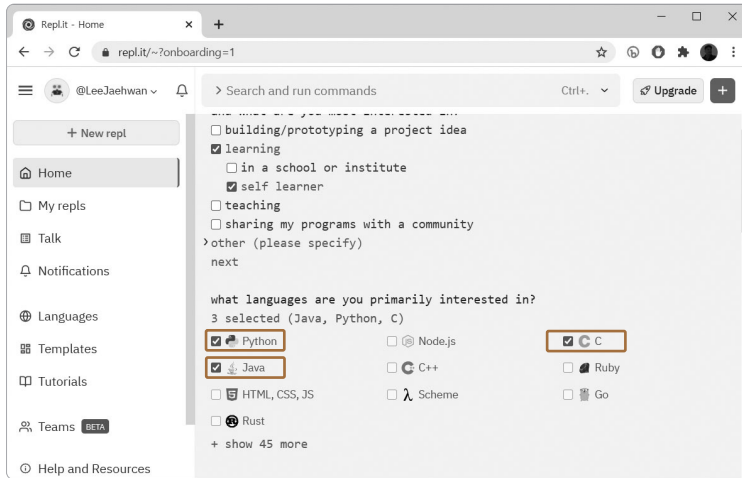


03 ① username, ② email, ③ password를 기입하고 [Sign up] 버튼을 클릭합니다.

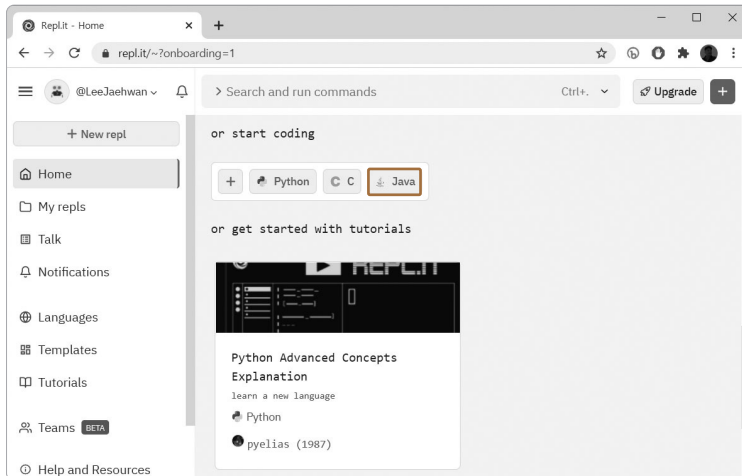
- 04 ① 메시지에서 지시하는 것처럼 앞에서 입력한 이메일 계정으로 발송된 ② 인증 메일에서 인증을 해주고 다시 이 화면으로 옵니다.



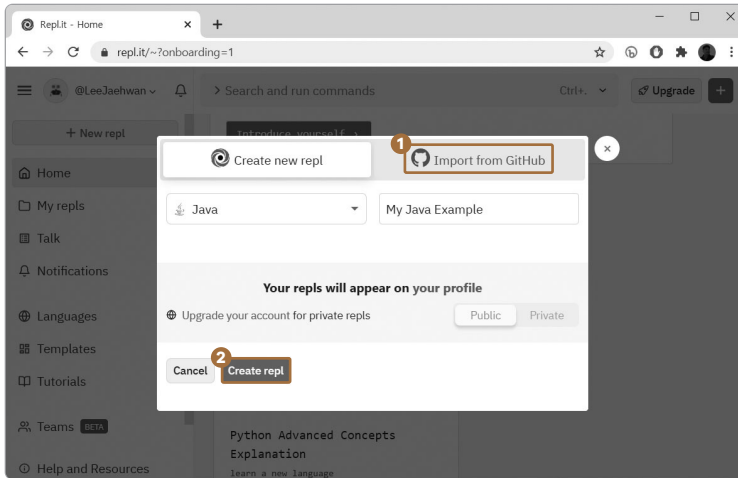
- 05 이제 간단한 설문에 답변을 달아줍니다. 중간에 선호하는 언어를 체크해줍니다. 전 Java, Python, C 언어를 체크했습니다.



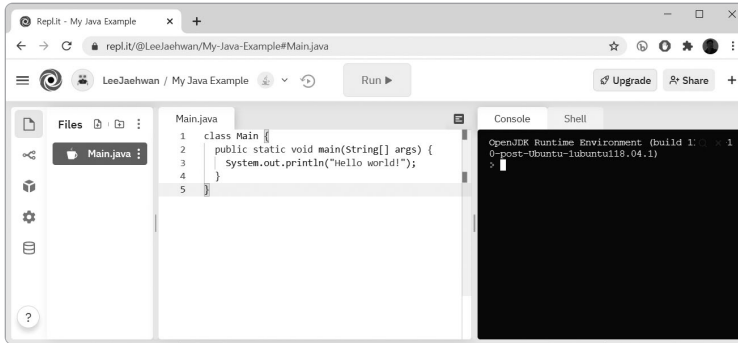
- 06 모두 입력을 하고 맨 아래로 오면 다음과 같은 화면입니다. 이제 [Java]를 클릭합니다(앞에서 3개 이상 클릭하면 Java 항목이 안 보일 수도 있습니다).



- 07** ❶ 우측 입력 상자에 “My Java Example”이라고 입력하고 ❷ [Create repl] 버튼을 클릭합니다.

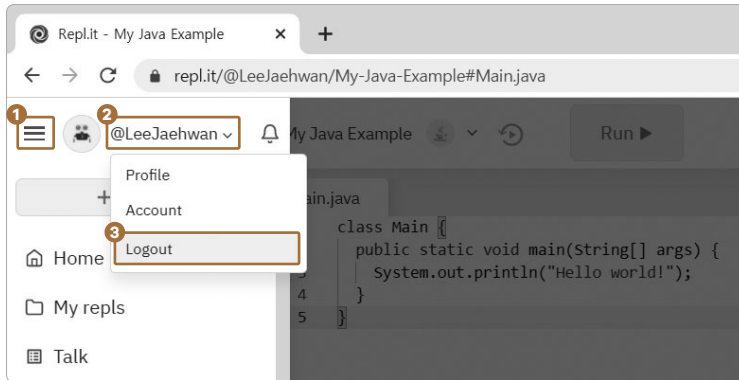


- 08** 다음과 같은 화면이 나오면 성공입니다.

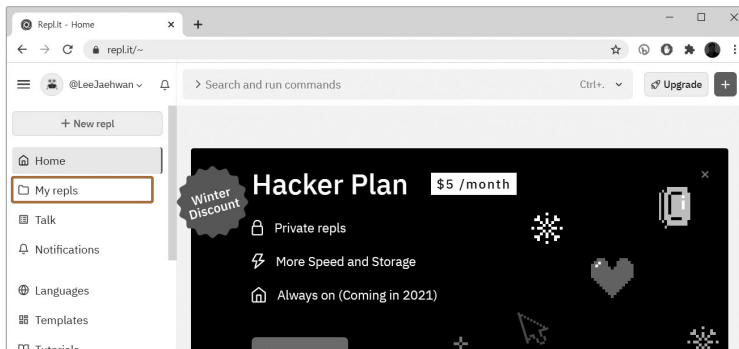


2.2 로그아웃하고 다시 로그인하기

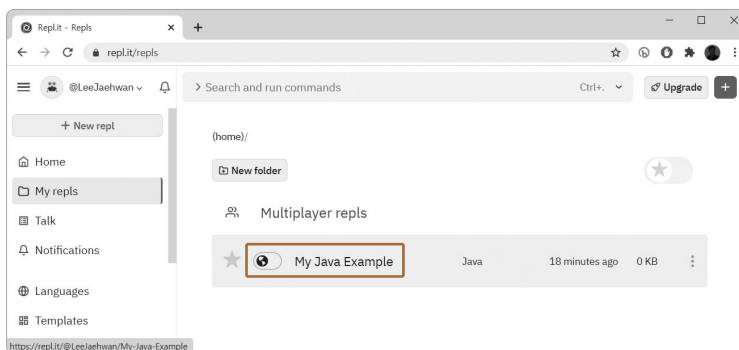
- To Do 01** 다음 순서대로 클릭하여 로그아웃을 합니다. ❶ 부분을 클릭하면 좌측 메뉴가 펼쳐집니다. ❷ 본인 ID를 누르고 ❸ [Logout]을 클릭합니다.



02 다시 로그인을 하면 다음과 같이 좌측에 메뉴가 보입니다. 여기서 [My relpls] 메뉴를 선택합니다.



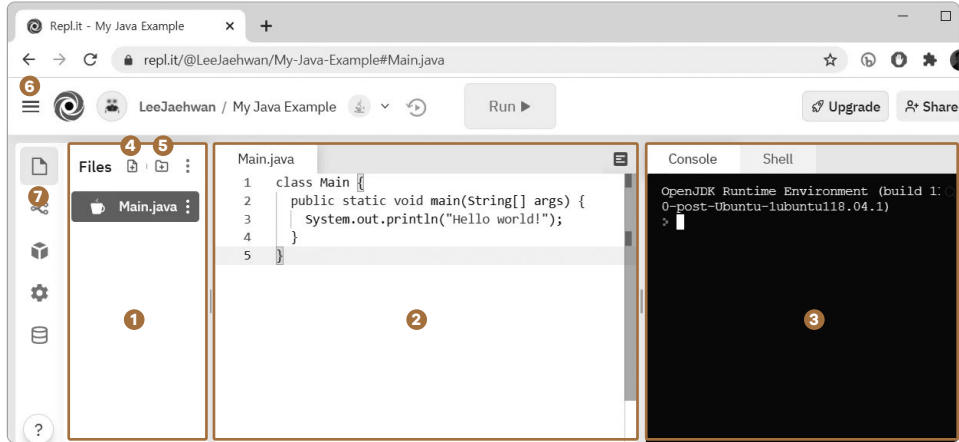
03 다음 화면에서 우리가 만들었던 폴더를 선택합니다. 그러면 앞에서 봤던 화면과 같은 화면을 볼 수 있습니다.



2.3 리플릿 화면 구성 익히기

이제 리플릿의 기본 화면 구성을 살펴보겠습니다.

▼ repl.it 기본 화면



이 책의 실습으로 사용하게 될 기능 위주로 설명합니다.

- 1 폴더 및 파일 구조를 확인할 수 있는 탐색기창입니다.
- 2 코드 편집기창입니다. 1번 메뉴에서 선택한 소스 코드 파일을 열어 코드를 수정하고 추가 및 삭제할 수 있습니다.
- 3 자바 코드를 컴파일하고 실행시킬 수 있는 콘솔창입니다.
- 4 파일을 추가합니다.
- 5 폴더를 추가합니다.
- 6 전체 메뉴를 표시합니다.
- 7 탐색기창을 열고 닫을 수 있습니다. 코드 편집기창이 그만큼 늘어나거나 줄어들게 되므로 노트북 등 화면이 작을 때 유용하게 사용할 수 있습니다.

2.4 자바 사용해보기

이제 자바를 사용해보겠습니다. 자바 코드를 만들고 실행하는 순서는 다음과 같습니다.

- 1 파일을 만들고

- ❷ 파일에 자바 코드를 입력하고
- ❸ JDK로 컴파일하고
- ❹ JRE로 실행시킵니다.

❶에 Main.java라는 파일이 이미 만들어져 있습니다. 그리고 ❷ 편집기창에는 자바 코드도 이미 작성되어 있습니다. 그러면 이제 ❸번 컴파일, ❹번 실행을 순서대로 진행하면 됩니다.

To Do 01 우측 콘솔창을 마우스로 클릭하고 다음과 같이 ❶ 영어 소문자로 ls(소문자 L과 소문자 S)를 입력합니다. 이 명령어는 리눅스 운영체제에서 현재 폴더의 파일 목록을 보여줍니다.

```

OpenJDK Runtime Environment (build 1:0-1
0-post-Ubuntu-1ubuntu118.04.1)
❶ > ls
Main.java
>
  
```

02 자바 개발 도구인 JDK로 컴파일합니다. 컴파일이란 우리가 만든 텍스트 코드를 이용하여 실행할 때 사용되는 자바 바이트 코드를 만들어주는 단계로 이해하시면 됩니다. ❶ javac 명령어를 다음과 같이 입력합니다.

```
javac Main.java
```

```

OpenJDK Runtime Environment (build 1:0-1
0-post-Ubuntu-1ubuntu118.04.1)
> ls
Main.java
❶ > javac Main.java
>
  
```

Warning 파일명은 대소문자를 정확히 입력해주어야 합니다. 컴파일할 때는 확장자까지 입력합니다.

03 1~2초 커서가 깜빡이다가 프롬프트가 다음 줄로 바뀝니다. 아무런 메시지 없이 다음 프롬프트 화면이 뜬다면 정상적으로 컴파일된 겁니다.

❶ 다시 ls 명령어를 입력합니다. 컴파일 결과로 만들어진 Main.class가 보일 겁니다. Main.class 파일은 우리가 만든 코드를 실행시킬 수 있는 바이트 코드로 이루어진 파일입니다.

```
ls
```

```
Console Shell
OpenJDK Runtime Environment (build 1:0.0.0-1
0-post-Ubuntu-1ubuntu118.04.1)
> ls
Main.java
> javac Main.java
1 > ls
Main.class Main.java
```

04 ❶ java 명령어를 이용해서 실행시켜봅니다. 파일명은 대소문자를 정확히 입력해주어야 합니다. 실행할 때는 확장자를 입력하지 않습니다.

```
java Main
```

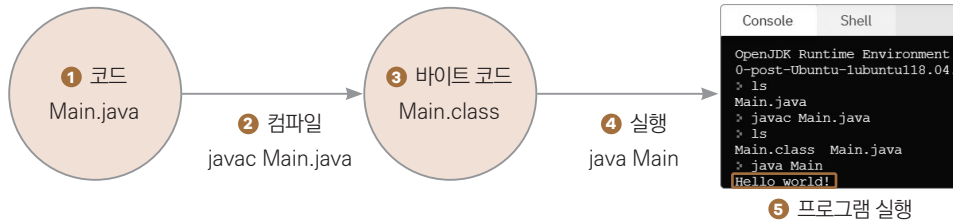
```
Console Shell
OpenJDK Runtime Environment (build 1:0.0.0-1
0-post-Ubuntu-1ubuntu118.04.1)
> ls
Main.java
> javac Main.java
> ls
Main.class Main.java
1 > java Main
Hello world!
>
```

실행 결과로 화면에 “Hello world!”가 출력되었습니다. 첫 번째 자바 프로그램을 컴파일하고 실행시켰습니다. 앞으로도 코드를 만들면 → javac를 이용하여 class 파일을 만들고 → java를 이용하여 class 파일을 실행시키면 됩니다.

Tip 윈도우에서도 여기에서 사용한 javac, java 등의 명령어를 똑같이 사용할 수 있습니다.

자 정리하면 이번 시간에 배운 것은 다음과 같습니다.

- ❶ 코드를 작성하고 (소스 파일을 생성하고)
- ❷ javac라는 컴파일러를 이용해서 ~.java 소스 파일을 컴파일을 하면
- ❸ ~.class라는 바이트 코드가 생성이 됩니다.
- ❹ 이후 java를 이용해서 Main.class 바이트 코드를 실행할 수 있습니다.
- ❺ 자바 프로그램이 실행됩니다.



‘컴파일할 때는 .java를 붙이고 실행할 때는 .class를 붙이지 않는다’는 규칙입니다.

초보자 대부분이 프로그래밍을 배울 때 하는 오해가 있습니다. 프로그래밍을 잘하려면 뭔가 어려운 것을 많이 배우고 이해해야 한다고 생각하는데, 아닙니다. 컴퓨터 프로그래밍을 배우는 것은 미리 정해진 단순한 규칙을 외우는 겁니다. 규칙을 외우고 잘 사용하는 것이 프로그래밍의 시작입니다.

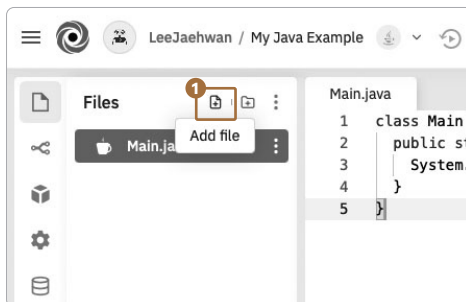
3 내용 출력하기

자바 코드를 직접 만들고 내용을 출력하는 몇 가지 방법을 연습해봅니다.

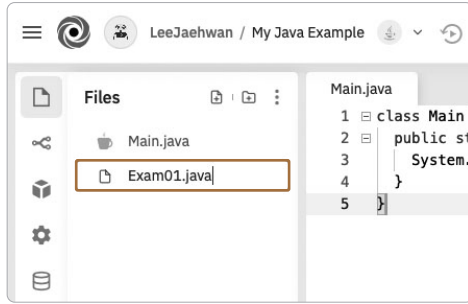
3.1 클래스 기본형 만들기

모든 코드를 다 외워서 작성하면 좋겠지만 사실 그럴 필요는 없습니다. 자바 코드의 기본적인 부분은 저도 직접 입력하지 않습니다. 그러므로 다음처럼 간단히 코드를 만들어보겠습니다.

To Do 01 1 파일을 클릭합니다.

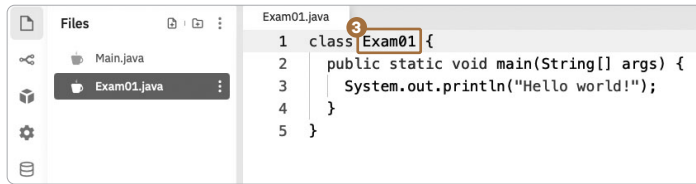


02 파일 이름은 Exam01.java로 만듭니다.



03 파일이 처음 만들어지면 오른쪽 편집기창의 내용은 비어 있습니다.

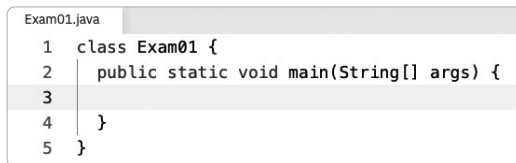
- ① Main.java 파일을 선택하고 편집기창에서 전체 내용 복사
- ② Exam01.java 파일을 선택하고 편집기창에 내용을 붙여넣기
- ③ 코드에서 Main을 Exam01로 수정



③ Exam01 부분을 클래스명이라고 합니다. 즉, 클래스명을 변경한 겁니다. main을 가진 클래스명은 파일명과 동일해야 하고 대문자로 시작해야 합니다. 이 또한 규칙입니다.

Warning 초보자가 제일 많이 실수하는 규칙입니다. 파일명과 main() 메서드를 가진 클래스의 이름이 다르면 자바 프로그램이 실행되지 않습니다.

04 3번 라인의 내용을 지우면 자바 클래스의 기본형이 됩니다. 이제 앞으로 작성할 모든 코드는 이 기본형을 만들고 내용을 작성하면 됩니다.



클래스의 기본형에서 class, public, static, void, main 등 나머지 다른 부분은 나중에 배울 때까지는 그냥 그대로 사용합니다. 뒤에서 다 배우게 될 내용이지만 일단 이 상태를 통으

로 기억하고 사용하면 됩니다.

제가 수업시간에 엄청 자주 하는 말이 있습니다.

“자바를 공부할 때 모든 것을 하나 하나 다 배우고 이해하고 실행하려고 하면
책 전체를 다 공부해야 첫 프로그램을 작성할 수 있습니다”

우리가 자동차를 운전할 때도 자동차가 어떻게 만들어지는지 알지 못해도 상관없지 않습니까? 우리는 그저 시동을 켜고 핸들을 붙잡고 운전을 하면 되는 겁니다. 일단은 거기서부터 시작하는 겁니다. 운전엔 익숙해지면 본넷도 열어서 엔진도 구경해보고, 오디오도 바꾸면서 튜닝도 하게 되는 겁니다.

일단은 써 보면서 하나 하나 용어를 익히고 익숙해지는 것이 중요합니다.

3.2 코드 작성

이제 코드를 다음과 작성하도록 합니다. 다음 코드를 보고 편집기창에 입력하면 됩니다. 들여쓰기 위치에 주의해서 작성해주시면 됩니다.

```
01 class Exam01 {
02     public static void main(String[] args) {
03         // 화면에 내용 출력하는 예제 <-- 주석(설명문)
04
05         // 내용 출력하고 줄 바꿈
06         System.out.println("홍길동");
07
08         // 내용만 출력하기 줄 안 바꿈
09         System.out.print("전우치");
10         System.out.print("손오공");
11
12         // 내용 없이 줄바꿈만 출력
13         System.out.println();
14     }
15 }
```

System이 뭔지 out이 뭔지 다 개별적인 의미가 있지만 여기서는

```
System.out.println(" ... ");
```

이렇게 한꺼번에 사용하면 큰따옴표 “” 안의 내용을 화면에 출력해준다고 알면 됩니다. 이 부분을 조금 더 알아보겠습니다.

06 : println처럼 ln까지 써주면 내용을 출력하고 줄바꿈을 해줍니다.

09 : ln 없이 print만 입력하면 줄바꿈이 생기지 않습니다.

13 : 내용을 입력하지 않고 println만 사용할 수도 있습니다. 이 경우 줄바꿈 처리만 합니다.

기본형 안에서 문장을 입력할 때 문장의 끝을 표시하기 위해 세미콜론 ;을 문장 마지막에 입력합니다. (03라인에서처럼) 슬래시 두 개를 연속해서 입력하면 코드에 설명을 달 수 있게 되는데 이를 주석이라고 부릅니다. 이 주석은 컴파일 대상에서 제외됩니다.

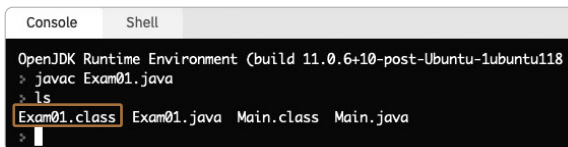
이번 예제에서는 이 정도만 알면 충분합니다. 이제 코드를 컴파일하고 실행해봅시다.

To Do 01 우측 콘솔창을 마우스로 클릭하고 다음과 같이 입력해 자바 소스 파일을 바이트 코드로 컴파일합니다.

```
javac Exam01.java
```

Warning 파일명은 대소문자를 정확히 입력해주어야 합니다. 컴파일을 할 때는 확장자까지 입력합니다.

02 1~2초 커서가 깜빡이다가 프롬프트가 다음 줄로 바뀝니다. 아무런 메시지 없이 다음 프롬프트 화면이 뜨면 정상적으로 컴파일된 겁니다. ls 명령어를 입력합니다. 컴파일 결과로 만들어진 Exam01.class를 확인합니다.



```
Console  Shell
OpenJDK Runtime Environment (build 11.0.6+10-post-Ubuntu-1ubuntu118)
> javac Exam01.java
> ls
Exam01.class Exam01.java Main.class Main.java
>
```

03 java 명령어를 이용해서 실행시켜봅니다.

Warning 파일명은 대소문자를 정확히 입력해주어야 합니다. 실행을 할 때는 확장자를 입력하지 않습니다.

```
java Exam01
```



```
Console  Shell
> java Exam01
홍길동
전우치손오공
>
```

앞의 코드 설명을 다시 한번 보겠습니다.

06 : 홍길동을 출력하고 줄이 바뀌고

09 : 전우치를 출력하고 줄이 바뀌지 않았습니다.

10 : 손오공 출력 후에도 줄이 바뀌진 않지만

13 : 내용 출력 없이 줄만 바꿉니다.

모든 주석은 출력되지 않고 있습니다. 우리가 작성한 코드대로 실행되고 있는 것을 확인할 수 있습니다.

Tip 컴파일을 하거나 실행을 할 때 에러가 난다면 다음을 찾아봅니다.

여러분이 자바를 배우고 사용하는 초기 6개월 동안 가장 많이 내는 에러는 거의 다음과 같습니다.

- 클래스명과 파일명이 다른 경우
- 코드 내용의 대소문자가 다른 경우
- 코드 내용의 단순 오타

이 정도가 거의 99.9%입니다. 여러분이 뭔가 대단한 에러를 내지는 않습니다.

4 변수 사용하기

프로그래밍에서 사용하는 용어에 변수^{variable}와 상수^{constant}가 있습니다. 어떤 의미이고, 어떻게 사용되는지 알아보겠습니다.

4.1 변수, 상수의 개념

여러분이 이미 알고 있는 개념입니다. 다음 수식을 통해 다시 한번 정리해보겠습니다.

```
y = x + 1;
```

x의 값이 2면 y의 값은 3이 됩니다.

x의 값이 3면 y의 값은 4가 됩니다.

이처럼 x, y는 대입되는 값에 따라 값이 변한다고 해서 변수라고 합니다. 그리고 1, 2, 3, 4 등은 이미 정해져 있는 값이라고 해서 상수라고 합니다.

그러나 수학과는 달리 프로그래밍에서는 변수의 값에 숫자만 사용하는 것이 아니고 다음과 같이 여러 형태의 값이 사용됩니다(숫자, 글자, 참/거짓).

```
x = 5;  
y = "홍길동";  
z = true;
```

그리고 수학에서는 변수명을 보통 x, y, z라고만 사용하지만 프로그래밍에서는 더 많은 변수를 사용하기 때문에 알아보기 쉬운 단어를 만들어서 변수명을 정합니다. 변수명만 보고도 그 안에 담길 값의 종류를 생각할 수 있다면 좋은 변수명입니다.

```
myMathScore = 100;  
myEnglishScore = 100;
```

변수명 표기법

변수명을 mymathscore처럼 전부 소문자로 이어서 적으면 단어를 파악하고 의미를 유추하기가 힘듭니다. 그래서 변수명을 보고 알아보기 쉬운 표기법이 생기게 되었습니다.

myMathScore처럼 단어 단위로 첫 글자를 대문자로 써줍니다. 낙타 등처럼 올라갔다 내려갔다 하는 모양이라서 카멜 표기법이라고 합니다. 그러나 대소문자를 사용하지 못 할 경우도 가끔 있습니다. 그럴 경우에는 my_math_score처럼 스네이크 표기법을 사용하기도 합니다.

변수명은 카멜 표기법을 사용하여 만들지만 첫 글자는 소문자로 적는데, 앞에서 클래스의 이름은 첫 글자를 대문자로 적는다는 규칙과 겹치지 않게 하기 위함입니다.

- 대문자로 시작하면 클래스명
- 소문자로 시작하면 변수명

이처럼 변수명만 보고 그 안에 담길 값의 종류를 생각할 수 있도록 만들었는데 실수로 다른 형태의

값을 다시 받을 수도 있으므로, 그런 경우를 방지하기 위해 자바에서는 변수의 특징을 표시해서 지정해줍니다. 이렇게 정해진 형태와 다른 형태의 값은 변수에 받을 수가 없습니다.

```
int myMathScore = 100;           // 정수만 변수에 담겠습니다.
int myEnglishScore = 99;        // 정수만 변수에 담겠습니다.
double myAverageScore = 99.5;   // 실수만 변수에 담겠습니다.
String myName = "홍길동";       // 문자열만 변수에 담겠습니다.
```

이런 식으로 변수 성격을 정해주게 됩니다. 성격이 정해진 변수에 다른 성격의 값을 넣어주면 컴파일 시 에러가 발생하게 됩니다.

Tip 본문에서는 이클립스라는 통합 개발 도구를 사용하게 됩니다. 이클립스를 사용하면 컴파일하지 않더라도 편집기창에서 다른 형태의 값이 변수에 대입되었다고 에러 표시를 보여줍니다.

4.2 들여쓰기

앞의 예제에서 클래스 코드의 기본형에 우리가 추가하여 작성한 코드가 안쪽으로 들여써진 것을 볼 수 있는데, 이를 들여쓰기라고 합니다. 들여쓰기는 사람들이 코드를 볼 때 가독성을 높이는 방법입니다.

▼ 들여쓰기 예

```
Exam02.java
1  class Exam02
2  {
3      public static void main(String[] args)
4      {
5          // 내용 작성
6      }
7  }
```

컴파일러는 세미콜론 ;으로 실행할 문장의 끝을 판단하기 때문에 모든 코드를 한 줄에 작성해도 정상적으로 처리할 수 있습니다. 하지만 그렇게 한 줄로 작성된 코드라면 우리 사람은 알아보기가 힘들기 때문에 코드 의미를 파악하기도 힘들고 에러가 발생했을 때 에러를 찾기도 힘듭니다.

초보자는 코드에 들여쓰기를 어떤 규칙으로 적용해야 하는지 어려워합니다. 첫 번째 좋은 방법은 예제와 똑같은 모양으로 코드를 만들면서 자연스럽게 몸에 익히는 겁니다. 두 번째는 들여쓰기 지

시선을 이용하는 방법입니다. 우리가 만든 코드의 내용이 들여쓰기 지시선에 겹쳐져서 작성되지 않도록 합니다. 초보자들이 들여쓰기를 제대로 못하는 경우가 많기 때문에 요새 편집기는 이와 같이 들여쓰기 지시선을 표시해주기도 합니다.

▼ 들여쓰기 지시선

```
Exam02.java
1 class Exam02
2 {
3     public static void main(String[] args)
4     {
5         // 우리가 작성한 내용이 들여쓰기 지시선에
6         // 겹쳐 써지지 않도록 합니다.
7     }
8 }
```

Tip 좋은 코드는 엄청난 알고리즘이 들어 있는 코드가 아니고 누구나 알아보기 쉽게 작성된 보기 좋은 코드입니다.

좋은 코드

- 이해하기 쉬운 변수명 사용
- 들여쓰기가 잘 지켜진 코드

4.3 코드 작성

앞에서 한 방식대로 Exam02.java 파일을 만들고 기본형을 만듭니다. 그리고 다음 코드를 보고 편집기창에 입력하면 됩니다.

```
Exam02.java
01 class Exam02
02 {
03     public static void main(String[] args)
04     {
05         // 계산 결과 출력하는 예제
06         System.out.println(2 + 3);
07
08         // 변수, 상수, 자료(데이터)형, 대입
09         // 남자 주변 = "홍길동";
10         // 변수들은 메모리에 기록이 됩니다.
11         int num1 = 2;
12         int num2 = 3;
13         // 계산 결과를 변수에 대입
14         int result = num1 + num2;
15
16         // 변수에 저장된 값을 출력
17         System.out.println(num1);
18         System.out.println(num2);
19         System.out.println(result);
20     }
```

06 : 계산을 먼저 하고 그 결과를 출력합니다.

11 : 변수명은 num1이고 변수의 특성은 정수만 저장할 수 있도록 int로 지정되었습니다. 이 변수에는 정수가 아닌 실수(소숫점이 있는 수)나, 글자를 대입할 수 없습니다. 상수인 정수 2를 변수에 대입합니다.

14 : 변수를 만들고 계산의 결과를 변수에 저장합니다.

17 : 정수가 저장된 변수를 이용하여 변수의 내용을 출력합니다.

19 : 계산의 결과가 저장된 변수를 이용하여 변수의 내용을 출력합니다.

이처럼 println은 소괄호 안의 내용이 숫자이거나, 글자이거나 상관없이 출력합니다. 그리고 계산 식일 경우 계산을 먼저 하고 그 결과를 출력합니다.

이제 코드를 컴파일하고 실행해봅시다.

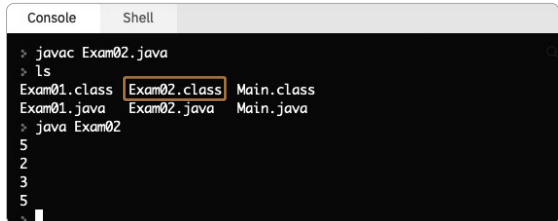
To Do 01 우측 콘솔창을 마우스로 클릭하고 다음과 같이 입력해 자바 소스 파일을 바이트 코드로 컴파일합니다.

```
javac Exam02.java
```

02 1~2초 커서가 깜빡이다가 프롬프트가 다음 줄로 바뀝니다. 아무런 메시지 없이 다음 프롬프트 화면이 뜬다면 정상적으로 컴파일된 겁니다. ls 명령어를 입력합니다. 컴파일 결과로 만들어진 Exam02.class를 확인합니다.

03 java 명령어를 이용해서 실행시켜봅니다. 실행 결과가 출력됩니다.

```
java Exam02
```



```
> javac Exam02.java
> ls
Exam01.class Exam02.class Main.class
Exam01.java Exam02.java Main.java
> java Exam02
5
2
3
5
>
```

우리 예제는 간단해서 변수에 값을 저장하고 한 번만 사용했지만, 길고 복잡한 코드에서는 변수에

값을 저장하고 필요할 때마다 여러 번 사용하게 됩니다.

5 산술 연산자 사용하기

프로그래밍을 할 때 컴퓨터로 하여금 변수나 값을 계산하는 데 사용되는 부호를 연산자라고 합니다.

5.1 계산을 위한 연산자

자바는 다양한 연산자를 제공하는데, 여기서는 흔히 사칙연산이라고 하는 기본적인 계산에 사용하는 산술 연산자를 살펴봅니다.

더하기 +, 빼기 -, 곱하기 *는 기본 사칙연산과 같은데, 나누기는 조금 다릅니다. 나누기는 몫을 구하는 /와 나머지를 구하는 %으로 나눕니다. 이렇게 나누기에서 두 가지 연산자를 제공하는 이유는 다음과 같습니다.

컴퓨터에서 처리하는 숫자는 1, 2, 3, 4, 5 ... 등의 정수와 1.0, 2.0, 3.0, 4.0, 5.0 ... 등의 실수로 나누게 됩니다. 정수형 변수는 int로 변수명 앞에 변수의 특징을 설정하고, 실수형 변수는 double로 변수명 앞에 변수의 특징을 설정합니다. 컴퓨터 중앙처리장치^{CPU} 안에는 연산 장치^{ALU}가 들어 있습니다. 간단하게 설명하면 이 연산 장치가 정숫값이 입력되면 정수 계산을 하고 결과를 정수로 반환하고, 실수값이 입력되면 실수 계산을 하고 실수 결과를 반환합니다.

수학에서는 $5 / 2 = 2.5$ 처럼 정수 나누기 정수는 실수가 가능하지만 컴퓨터에서는 '정수 나누기 정수는 정수'의 결과만을 반환하게 됩니다. 애초에 이렇게 만들어졌으니 이것도 외워야 할 규칙이라면 규칙입니다. 그러므로 정수형 타입의 변수를 사용해서 $5 / 2$ 결과를 정확히 표현하려면 다음의 두 가지 연산을 해야 합니다.

```
int nResult1 = 5 / 2; // 몫 구하기
int nResult2 = 5 % 2; // 나머지 구하기
```

정수형 변수 nResult1에는 나누기의 몫 2가 결과로 대입됩니다. 그리고 정수형 변수 nResult2에는 나머지 1이 결과로 대입됩니다.

5.2 코드 작성

앞에서 한 방식대로 Exam03.java 파일을 만들고 기본형을 만듭니다. 그리고 다음 코드를 보고 편집기창에 입력하면 됩니다.

```
01 class Exam03
02 {
03     public static void main(String[] args)
04     {
05         // 사칙연산, 나머지 : 연산자(연산 기호)
06         int num1 = 5;
07         int num2 = 2;
08         // 더하기
09         int result1 = num1 + num2;
10         System.out.println(result1);
11         // 빼기
12         int result2 = num1 - num2;
13         System.out.println(result2);
14         // 곱하기
15         int result3 = num1 * num2;
16         System.out.println(result3);
17         // 나누기
18         int result41 = num1 / num2;
19         System.out.println(result41);
20         // 나머지
21         int result42 = num1 % num2;
22         System.out.println(result42);
23     }
24 }
```

06 : 변수에 값을 저장합니다.

07 : 변수에 값을 저장합니다.

09 : 더하기 연산 결과를 변수에 저장합니다.

12 : 빼기 연산 결과를 변수에 저장합니다.

15 : 곱하기 연산 결과를 변수에 저장합니다.

18 : 나누기 연산에서 몫을 변수에 저장합니다.

21 : 나누기 연산에서 나머지를 변수에 저장합니다.

이제 코드를 컴파일하고 실행해봅시다.

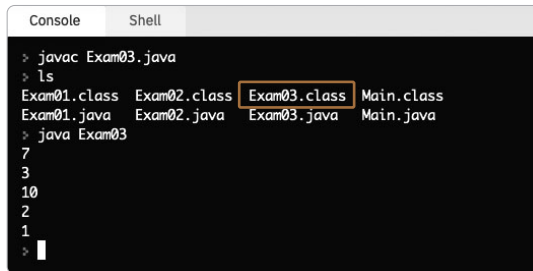
- To Do 01** 우측 콘솔창을 마우스로 클릭하고 다음과 같이 입력해 자바 소스 파일을 바이트 코드로 컴파일합니다.

```
javac Exam03.java
```

- 02** 1~2초 커서가 깜빡이다가 프롬프트가 다음 줄로 바뀝니다. 아무런 메시지 없이 다음 프롬프트 화면이 뜬다면 정상적으로 컴파일된 겁니다. ls 명령어를 입력합니다. 만들어진 Exam03.class를 확인합니다.

- 03** java 명령어를 이용해서 실행시켜봅니다. 실행 결과가 출력됩니다.

```
java Exam03
```



```
Console Shell
> javac Exam03.java
> ls
Exam01.class Exam02.class Exam03.class Main.class
Exam01.java Exam02.java Exam03.java Main.java
> java Exam03
7
3
10
2
1
>
```

자바 코드를 입력하여 에러 없이 컴파일하고, 그 결과물인 바이트 코드(.class)를 실행하여 결과를 보았다면, 이제 여러분은 간단한 계산쯤은 자바를 이용하여 프로그래밍할 수 있게 된 겁니다. 여기까지 공부한 것만으로도 여러분이 자바로 프로그램을 만들어서 계산 기능을 구현할 수 있다는 것이 중요한 겁니다.

6 조건문 사용하기

이번에는 자바에서 사용하는 제어문 중 하나인 조건문을 알아보겠습니다.

6.1 if 조건문

프로그램의 실행에서 어떤 연산이 조건으로 주어졌을 때 그 결과가 참이냐 거짓이냐에 따라 프로그

램의 서로 다른 부분을 실행하는 데 if 조건문을 사용합니다.

앞에서 프로그래밍은 규칙을 외우는 것이라고 했습니다. 자바에서 사용하는 조건문중 if문은 그 단어 뜻처럼 사용하면 되는데, if를 사용한 조건문의 규칙은 다음과 같이 세 가지입니다.

▼ 타입 #1: if문만 사용하기

```
if (연산식)
{
    // 연산식 결과가 참(true)이면 이 부분 실행
}
```

▼ 타입 #2: if ~ else ~ 사용하기

```
if (연산식)
{
    // 연산식 결과가 참(true)이면 이 부분 실행
}
else
{
    // 연산식 결과가 거짓(false)이면 이 부분 실행
}
```

▼ 타입 #3-1: if ~ else if ~ else ~ 사용하기

```
if (연산식 A)
{
    // 연산식 A의 결과가 참(true)이면 이 부분 실행
}
else if (연산식 B)
{
    // 연산식 A의 결과가 거짓(false)이고
    // 연산식 B의 결과가 참(true)이면 이 부분 실행
}
else
{
    // 연산식 B의 결과가 거짓(false)이면 이 부분 실행
}
```

▼ 타입 #3-2: if ~ else if ~ else ~ 사용하기

```
if (연산식 A)
```

```

{
    // 연산식 A의 결과가 참(true)이면 이 부분 실행
}
else if (연산식 B)
{
    // 연산식 A의 결과가 거짓(false)이고
    // 연산식 B의 결과가 참(true)이면 이 부분 실행
}
else if (연산식 C)
{
    // 연산식 B의 결과가 거짓(false)이고
    // 연산식 C의 결과가 참(true)이면 이 부분 실행
}
else
{
    // 연산식 C의 결과가 거짓(false)이면 이 부분 실행
}

```

6.2 코드 작성 1단계

먼저 조건의 결과를 참이나 거짓으로 판단하기 위해 결과를 출력해봅니다. 앞에서 한 방식대로 Exam04.java 파일을 만들고 기본형을 만듭니다. 그리고 다음 코드를 보고 편집기창에 입력하면 됩니다.

```

01 class Exam04
02 {
03     public static void main(String[] args)
04     {
05         System.out.println(2 < 3);
06         System.out.println(2 > 3);
07
08         boolean bMyCheck = (2 > 3);
09         System.out.println(bMyCheck);
10
11         // 이후 코드를 이 부분에 적습니다.
12     }
13 }

```

Exam04.java

- 05 : '2가 3보다 작다'라는 비교 연산 결과를 출력합니다.
- 06 : '2가 3보다 크다'라는 비교 연산 결과를 출력합니다.
- 08 : '2가 3보다 크다'라는 비교 연산 결과를 변수에 대입합니다.

이제 코드를 컴파일하고 실행해봅시다.

- To Do 01** 우측 콘솔창을 마우스로 클릭하고 다음과 같이 입력해 자바 소스 파일을 바이트 코드로 컴파일합니다.

```
javac Exam04.java
```

- 02** 1~2초 커서가 깜빡이다가 프롬프트가 다음 줄로 바뀝니다. 아무런 메시지 없이 다음 프롬프트 화면이 뜬다면 정상적으로 컴파일된 겁니다. ls 명령어를 입력합니다. 컴파일 결과로 만들어진 Exam04.class를 확인합니다.

- 03** java 명령어를 이용해서 실행시켜봅니다. 실행 결과가 출력됩니다.

```
java Exam04
```

```

> javac Exam04.java
> ls
Exam01.class Exam02.class Exam03.class Exam04.class Main.class
Exam01.java Exam02.java Exam03.java Exam04.java Main.java
> java Exam04
true
false
false
>

```

6.3 코드 작성 2단계

이번에는 if 조건문을 사용해보겠습니다. 기존 Exam04.java 파일에 다음 코드에 추가된 내용을 추가하면 됩니다. 11라인에서 15라인까지가 추가된 내용입니다.

```

01 class Exam04
02 {
03     public static void main(String[] args)
04     {
05         System.out.println(2 < 3);
06         System.out.println(2 > 3);

```

```

07
08     boolean bMyCheck = (2 > 3);
09     System.out.println(bMyCheck);
10
11     // 조건문
12     if (1 < 2)
13     {
14         System.out.println("Hello");
15     }
16
17     // 이후 코드를 이 부분에 적습니다.
18 }
19 }

```

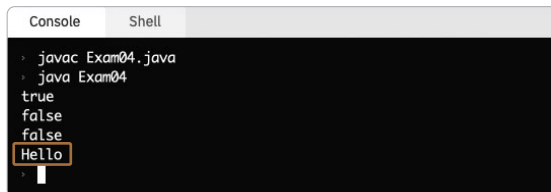
12라인의 if문 괄호 안에 있는 비교 연산식 결과가 true이면 중괄호 안의 코드인 14라인이 실행됩니다. 하지만 연산식 결과가 false라면 13라인부터 15라인까지는 아무것도 실행되지 않고 그냥 통과됩니다.

다시 코드를 컴파일하고 실행해봅시다.

To Do 01 우측 콘솔창을 마우스로 클릭하고 다음과 같이 입력해 자바 소스 파일을 바이트 코드로 컴파일합니다.

```
javac Exam04.java
```

02 컴파일이 끝나면 java 명령어를 이용해서 실행시켜봅니다. 실행 결과가 출력됩니다.



```

Console  Shell
> javac Exam04.java
> java Exam04
true
false
false
Hello
>

```

12라인에서 if문 괄호 안의 '1이 2보다 작다'라는 비교 연산 결과는 true입니다. 따라서 14라인이 실행되고 그 결과가 출력되었습니다.

6.4 코드 작성 3단계

계속해서 기존 Exam04.java 파일에 다음 코드를 추가합니다. 앞에서는 상수를 이용하여 비교 연산을 수행했지만 이번에는 변수에 값을 저장하고 그 변수를 이용하여 비교 연산을 해봅니다. 17라인에서 22라인까지가 추가된 내용입니다.

```
16
17      // 3은 2보다 작지 않으므로(false) 중괄호 덩어리 실행 안 됨
18      int num = 3;
19      if (num < 2)
20      {
21          System.out.println("Hi~");
22      }
23  }
24 }
```

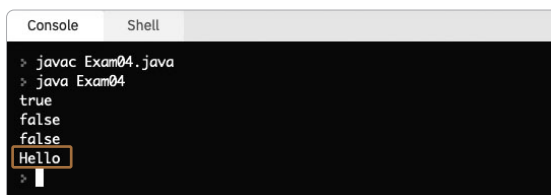
19라인의 if문 괄호 안에 있는 비교 연산식 결과가 true이면 중괄호 안의 코드인 21라인이 실행됩니다. 하지만 연산식 결과가 false라면 20라인부터 22라인까지는 아무것도 실행되지 않고 그냥 통과됩니다.

다시 코드를 컴파일하고 실행해봅시다.

To Do 01 우측 콘솔창을 마우스로 클릭하고 다음과 같이 입력해 자바 소스 파일을 바이트 코드로 컴파일합니다.

```
javac Exam04.java
```

02 컴파일이 끝나면 java 명령어를 이용해서 실행시켜봅니다. 실행 결과가 출력됩니다.



```
Console  Shell
> javac Exam04.java
> java Exam04
true
false
false
Hello
>
```

19라인 if문 괄호 안의 '변수 num에 저장된 값이 2보다 작다'라는 비교 연산 결과는 false입니다. num 변수에는 3이 저장되어 있기 때문입니다. 그에 따라 20라인부터 22라인까지는

실행되지 않고 그냥 통과되었기에 결과를 출력하는 콘솔창에는 “Hi~”가 출력되지 않았습니다.

6.5 코드 작성 4단계

계속해서 기존의 Exam04.java 파일에 다음 코드를 추가합니다. 연산 결과를 구하고 그 값을 다시 비교하는 연산을 합니다.

24라인에서 29라인까지가 추가된 내용입니다.

```
Exam04.java
23
24      // == 같다
25      int num2 = 4;
26      if ((num2 % 2) == 1)
27      {
28          System.out.println("나머지가 1이면 출력된다.");
29      }
30  }
31 }
```

26라인 if문 괄호 안에서 또 다른 소괄호 속 나머지를 구하는 연산이 먼저 수행이 되고 결과가 나옵니다. 그리고 그 결과가 1이랑 같은지를 비교하게 됩니다. 그 결과가 true이면 중괄호 안의 코드인 28라인이 실행됩니다.

Tip 등호 = 한 개는 이미 대입 연산에 사용되고 있기 때문에, 비교 연산은 등호 기호를 두 개 연달아 붙여서 사용합니다.

순서대로 표현하면 26라인의 코드는 내부적으로 다음과 같이 변하게 됩니다. 연산의 우선 순위는 우리가 예전에 수학 시간에 배웠던 것처럼 괄호를 사용하는 것이 제일 먼저 계산됩니다.

```
if ((num2 % 2) == 1)
    ↓
if ((4 % 2) == 1)
    ↓
if (0 == 1)
    ↓
if (false)
```

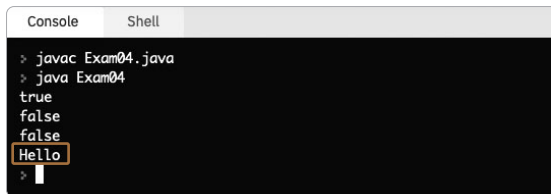
최종으로 수행되는 비교 연산 결과가 false이기에 27라인에서 29라인은 실행되지 않고 그냥 통과됩니다.

확인을 위해 다시 코드를 컴파일하고 실행해봅시다.

To Do 01 우측 콘솔창을 마우스로 클릭하고 다음과 같이 입력해 자바 소스 파일을 바이트 코드로 컴파일합니다.

```
javac Exam04.java
```

02 컴파일이 끝나면 java 명령어를 이용해서 실행시켜봅니다. 실행 결과가 출력됩니다.



```
Console Shell
> javac Exam04.java
> java Exam04
true
false
false
Hello
>
```

역시 이번에도 콘솔창에는 28라인의 내용이 출력되지 않았습니다.

6.6 코드 작성 5단계

계속해서 기존의 Exam04.java 파일에 다음 코드를 추가합니다. 이번에는 if문과 함께 사용할 수 있는 else문을 사용해봅시다. 31라인에서 38라인까지가 추가된 내용입니다.

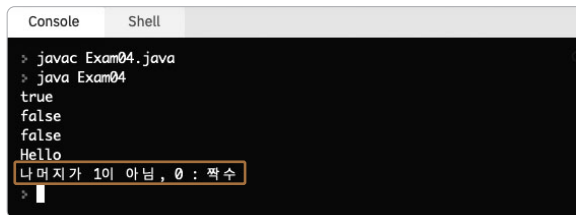
```
Exam04.java
30
31     if ((num2 % 2) == 1)
32     {
33         System.out.println("나머지가 1 : 홀수");
34     }
35     else
36     {
37         System.out.println("나머지가 1이 아님, 0 : 짝수");
38     }
39 }
40 }
```

31라인 if문의 비교 연산 결과가 true이면 32라인에서 34라인까지의 중괄호 영역이 실행됩니다. false라면 36라인에서 38라인까지의 중괄호 영역이 실행됩니다. 확인을 위해 다시 코드를 컴파일 하고 실행해봅시다.

To Do 01 우측 콘솔창을 마우스로 클릭하고 다음과 같이 입력해 자바 소스 파일을 바이트 코드로 컴파일합니다.

```
javac Exam04.java
```

02 컴파일이 끝나면 java 명령어를 이용해서 실행시켜봅니다. 실행 결과가 출력됩니다.



```
Console Shell
> javac Exam04.java
> java Exam04
true
false
false
Hello
나머지가 1이 아님, 0 : 짝수
>
```

31라인 if문에서 비교 연산 결과가 false이기에 else 영역인 36라인에서 38라인이 실행되어 37라인의 내용이 화면에 출력되었습니다.

이번 예제에서 사용된 전체 코드는 다음과 같습니다.

```
01 class Exam04
02 {
03     public static void main(String[] args)
04     {
05         System.out.println(2 < 3);
06         System.out.println(2 > 3);
07
08         boolean bMyCheck = (2 > 3);
09         System.out.println(bMyCheck);
10
11         // 조건문
12         if (1 < 2)
13         {
14             System.out.println("Hello");
15         }
16     }
```



```

17      // 3은 2보다 작지 않으므로(false) 중괄호 덩어리 실행 안 됨
18      int num = 3;
19      if (num < 2)
20      {
21          System.out.println("Hi~");
22      }
23
24      // == 같다
25      int num2 = 4;
26      if ((num2 % 2) == 1)
27      {
28          System.out.println("나머지가 1이면 출력된다.");
29      }
30
31      if ((num2 % 2) == 1)
32      {
33          System.out.println("나머지가 1 : 홀수");
34      }
35      else
36      {
37          System.out.println("나머지가 1이 아님, 0 : 짝수");
38      }
39  }
40 }

```

7 while문 사용하기

프로그래밍을 할 때 반복적으로 어떤 일을 처리하고 싶을 때 반복문을 사용합니다. 자바는 반복문으로 while, do~while, for문을 제공합니다. 먼저 while문을 알아봅시다.

7.1 while문이란?

while문이란 어떤 조건이 성립하는 동안 반복 처리를 실행하는 제어문입니다. while문 소괄호 안 연산식 결과가 true이면 중괄호 안의 내용이 실행됩니다. 그리고 다시 소괄호 안 연산식 결과를 체크합니다. 이렇게 계속 반복이 됩니다. 이 동작은 규칙이니 외워야 합니다.

이때 반복 횟수를 세는 변수를 만들어 반복 횟수를 지정할 수 있습니다.

```
while (연산식)
{
    // 연산식의 결과가 참이면 이 부분 실행
    문장;
}
```

자, 조금 구체적인 형태로 코드를 만들어보겠습니다.

```
int num = 0;
while (num < 5)
{
    System.out.println("홍길동");
    num = num + 1;
}
```

먼저 반복 횟수를 체크할 정수형의 num 변수를 만들고 정수 0을 값으로 대입합니다.

그리고 while문 소괄호 안 'num 변수의 값이 5보다 작다'라는 연산식 결과가 true이면 중괄호 안의 내용이 실행됩니다. 이때 "홍길동"을 출력하고 그다음 줄에서 반복 횟수를 체크하는 변수의 값을 1 증가시킵니다.

중괄호 안의 실행이 끝나면 다시 소괄호의 연산식을 체크하러 갑니다. 이렇게 반복되다 num 변수의 값이 증가해서 5가 되면 소괄호 안의 연산식 조건이 false가 되므로 더 이상 중괄호 안의 내용을 실행하지 않고 while문은 끝나게 됩니다.

Tip 수업을 할 때 이 부분을 머리로만 계산하고 생각하지 말라고 합니다. 연습장에 num 변수의 값이 증가하는 모습과 소괄호 안의 연산식 결과를 써가면서 위 설명을 보면 쉽게 이해할 수 있습니다.

7.2 코드 작성 1단계

앞에서 한 방식대로 Exam05.java 파일을 만들고 기본형을 만듭니다. 그리고 다음 코드를 보고 편집기창에 입력하면 됩니다. 구구단의 2단을 출력하는 코드입니다.

```
01 class Exam05
02 {
```

Exam05.java

```

03 public static void main(String[] args)
04 {
05     // 반복문 : while, for
06     System.out.println(2 * 1);
07     System.out.println(2 * 2);
08     System.out.println(2 * 3);
09     System.out.println(2 * 4);
10     System.out.println(2 * 5);
11     System.out.println(2 * 6);
12     System.out.println(2 * 7);
13     System.out.println(2 * 8);
14     System.out.println(2 * 9);
15
16 }
17 }

```

06라인에서 14라인까지 System.out.println을 이용하여 단순히 연산 결과를 출력합니다.

이제 코드를 컴파일하고 실행해봅시다.

To Do 01 우측 콘솔창을 마우스로 클릭하고 다음과 같이 입력해 자바 소스 파일을 바이트 코드로 컴파일합니다.

```
javac Exam05.java
```

02 1~2초 커서가 깜빡이다가 프롬프트가 다음 줄로 바뀝니다. 아무런 메시지 없이 다음 프롬프트 화면이 뜬다면 정상적으로 컴파일된 겁니다. ls 명령어를 입력합니다. 컴파일 결과로 만들어진 Exam05.class를 확인합니다.

03 java 명령어를 이용해서 실행시켜봅니다. 실행 결과가 출력됩니다.

```
Console Shell
javac Exam05.java
ls
Exam01.class Exam02.java Exam04.class Exam05.java
Exam01.java Exam03.class Exam04.java Main.class
Exam02.class Exam03.java Exam05.class Main.java
java Exam05
2
4
6
8
10
12
14
16
18
```

콘솔창에 구구단 2단의 결과가 잘 출력되었습니다.

7.3 코드 작성 2단계

그런데 코드를 자세히 보면 06라인에서 14라인까지 출력문 안의 연산식에서 뒤 쪽의 숫자가 하나씩 증가하고 있습니다. 이렇게 같은 동작에서 조금씩 변하는 부분의 규칙을 찾을 수 있다면 반복문을 사용할 수 있습니다. 이제 규칙을 발견했으니 반복문을 만들어보겠습니다.

기존 Exam05.java 파일에 다음 코드를 추가합니다. 16라인에서 24라인까지가 추가된 내용입니다.

```
15
16 System.out.println("=====");
17
18 // 위의 반복에서 뒤의 자리 정수만 변하게 처리
19 int num = 1;
20 while (num < 10)
21 {
22     System.out.println(2 * num);
23     num = num + 1;
24 }
25 }
26 }
```

16 : 앞 부분의 실행 결과와 뒷부분의 실행 결과를 구분하는 구분선을 출력합니다.

- 19 : 반복 횟수를 기록하는 정수형 변수를 만들고 초깃값을 정숫값으로 대입시킵니다.
- 20 : while문 소괄호 안 연산식 결과가 true인 경우 이후 중괄호 안의 내용이 실행됩니다.
- 22 : 변수의 값을 이용해 구구단을 출력합니다.
- 23 : 반복 횟수를 증가시킵니다.
- 24 : 반복문의 중괄호가 끝나면 다시 20라인 소괄호 안 연산식 체크가 실행됩니다.

이렇게 반복문을 적용할 수 있습니다. 지금이야 기존의 코드나 반복문이나 길이가 비슷해서 체감이 잘 안 되지만 반복 횟수가 만 번 정도라면 반복문이 훨씬 효율적이라고 생각이 들 겁니다. 기존 코드는 직접 입력을 하지 않고 복사&붙여넣기를 한다 하더라도 만 번을 반복 작업해야 하지만 반복 문은 소괄호 속 연산식 조건만 바꾸면 되니까요.

Tip 한두 번 반복을 굳이 반복문을 적용할 필요는 없습니다. 저도 처음부터 반복문을 적용해서 코드를 작성할 때도 있지만, 코드를 작성하다 자꾸 반복이 되면 그때 규칙을 찾아서 반복문을 적용하기도 합니다.

이제 코드를 컴파일하고 실행해봅시다.

To Do 01 우측 콘솔창을 마우스로 클릭하고 다음과 같이 입력해 자바 소스 파일을 바이트 코드로 컴파일합니다.

```
javac Exam05.java
```

02 이제 java 명령어를 이용해서 실행시켜봅니다. 실행 결과가 출력됩니다.

```

Console  Shell
> javac Exam05.java
> java Exam05
2
4
6
8
10
12
14
16
18
=====
2
4
6
8
10
12
14
16
18
>
  
```

구분선 앞 부분은 반복적인 코드의 입력에 의한 출력이고, 구분선 뒷부분은 while문에 의한 출력입니다. 반복문 적용이 잘된 것을 확인할 수 있습니다.

7.4 while문 다른 사용 방법

다음과 같은 상황에서 while문의 불편한 점을 한번 보겠습니다.

```
int num = 0;
while (num < 5)
{
    System.out.println("홍길동");
    num = num + 1;
}
```

while문 중괄호 안의 코드가 길어져서 여러 줄이 되면, 그래서 화면상에서 여러 페이지가 되면 반복 횟수를 처리하는 변수가 한 화면에 보이지 않게 됩니다. 이렇게 한 동작을 처리하는 내용이 흩어져 있으면 프로그램의 로직을 파악하기가 힘들어집니다.

그래서 while문은 횟수가 정해져 있는 것을 처리하기보다는 조건이 정해져 있을 경우를 처리할 때 다음과 같이 사용합니다.

```
while (true)
{
    문장;
    if (연산식)
    {
        break;
    }
}
```

while의 소괄호 안에서는 연산식 결과를 체크하는 데 이미 연산 절차 없이 결과가 true로 주어져 있으므로 무조건 중괄호를 실행합니다. 중괄호 안에서 문장을 실행하고 if 조건문에서 소괄호 안의 연산식 결과가 true이면 break를 이용해 while의 반복을 멈추게 합니다. 구체적인 예제는 본 수업에서 다루도록 하겠습니다(6장 '제어문').

8 for문 사용하기

자바에서는 반복문으로 while, do ~ while, for문을 제공한다고 했습니다. 그리고 앞에서 while문을 알아보았습니다. 여기서는 for문을 알아봅니다.

8.1 for문

for문¹은 반복 횟수가 정해져 있을 때 사용하기 적합한 반복문입니다. 형식은 다음과 같습니다.

```
for (초기식; 조건 체크 연산식; 증감식)
{
    // 연산식의 결과가 참이면 이 부분 실행
    문장;
}
```

자, 조금 구체적인 형태로 코드를 만들어보겠습니다.

```
for (int i = 0; i < 5; i = i + 1)
{
    System.out.println("홍길동");
}
```

먼저 ❶ 초기식 부분에 반복 횟수를 체크할 정수형인 i 변수를 만들고 정수 0을 초깃값으로 대입합니다.

❷ 연산식 부분에서 조건의 결과를 확인합니다. 연산 결과가 true이면 ❸ 중괄호 안의 문장이 실행됩니다. 그리고 ❹ 증감식에서 반복 횟수를 체크하는 변수의 값을 증가시킵니다. 이 코드는 콘솔창에 “홍길동”을 다섯 번 출력하게 됩니다.

이처럼 반복 횟수가 정해진 경우 반복 처리 내용이 한 줄에 표시되므로 로직을 파악하기가 편하다는 장점이 있습니다.

¹ for 반복문이라고도 합니다.

Tip 변수명은 보통 의미를 알 수 있게 단어 형태로 만듭니다. 하지만 for문 안에서 반복을 처리하는 변수의 이름으로는 다들 간단하게 i 또는 j를 사용합니다.

8.2 코드 작성 1단계

앞에서 한 방식대로 Exam06.java 파일을 만들고 기본형을 만듭니다. 그리고 다음 코드를 보고 편집기창에 입력하면 됩니다. 이번에도 구구단의 2단을 출력하는 코드입니다.

```
01 class Exam06
02 {
03     public static void main(String[] args)
04     {
05         // 반복문 : while, for
06         System.out.print(2 * 1 + " ");
07         System.out.print(2 * 2 + " ");
08         System.out.print(2 * 3 + " ");
09         System.out.print(2 * 4 + " ");
10         System.out.print(2 * 5 + " ");
11         System.out.print(2 * 6 + " ");
12         System.out.print(2 * 7 + " ");
13         System.out.print(2 * 8 + " ");
14         System.out.print(2 * 9 + " ");
15         System.out.println();
16
17     }
18 }
```

06라인에서 14라인까지 System.out.print()를 이용하여 단순히 연산 결과를 출력합니다. 앞의 예제에서는 너무 여러 줄에 걸쳐 출력되었기에 이번에는 println()이 아니고 print()문을 사용하여 줄을 바꾸지 않고 있습니다. 다만 그렇게만 출력하면 숫자가 전부 다 붙어서 출력되므로 숫자와 숫자 사이에 스페이스 한 칸을 띄어주어 출력을 하고 있습니다.

07라인의 실행 결과를 순서대로 표현하면 07라인의 코드는 내부적으로 다음과 같이 변하게 됩니다.


```

System.out.print(2 * 2 + " ");
System.out.print(4 + " ");
System.out.print("4" + " ");
System.out.print("4 ");

```

❶ 소괄호 안의 연산을 왼쪽부터 연산자 우선 순위에 따라서 계산합니다. 연산자의 우선 순위는 여러분이 수학 시간에 배운 것과 거의 똑같습니다.

❷ 4하고 하나의 스페이스를 처리하는 글자 "를 수학적으로 더할 수 없기 때문에 자바는 이 둘의 산술 연산을 하지 않습니다. 그리고 print()문이 출력을 하기 위해 사용하는 것이므로 사람을 대신해서 ❷ 정수 4를 글자로 바꾸어서 ❸ 두 글자를 붙여서 출력을 해줍니다.

이제 코드를 컴파일하고 실행해봅시다.

To Do 01 우측 콘솔창을 마우스로 클릭하고 다음과 같이 입력해 자바 소스 파일을 바이트 코드로 컴파일합니다.

```
javac Exam06.java
```

02 1~2초 커서가 깜빡이다가 프롬프트가 다음 줄로 바뀝니다. 아무런 메시지 없이 다음 프롬프트 화면이 뜬다면 정상적으로 컴파일된 겁니다. ls 명령어를 입력합니다. 컴파일 결과로 만들어진 Exam06.class를 확인합니다.

03 java 명령어를 이용해서 실행시켜봅니다. 실행 결과가 출력됩니다.

```

Console  Shell
> javac Exam06.java
> ls
Exam01.class  Exam02.java  Exam04.class  Exam05.java  Main.class
Exam01.java   Exam03.class Exam04.java   Exam06.class  Main.java
Exam02.class  Exam03.java  Exam05.class  Exam06.java
> java Exam06
2 4 6 8 10 12 14 16 18
>

```

콘솔창에 구구단 2단의 결과가 잘 출력된 것을 볼 수 있습니다.

8.3 코드 작성 2단계

06라인에서 14라인까지 출력문 안의 연산식에서 뒤 쪽의 숫자가 하나씩 증가하고 있는 규칙을 찾을 수 있고, 반복 횟수를 알고 있기 때문에 이번에는 for문을 적용해보겠습니다.

기존의 Exam06.java 파일에 다음 코드를 추가합니다. 16라인에서 24라인까지가 추가된 내용입니다.

```
Exam06.java
16
17     System.out.println("=====");
18
19     // 위의 반복에서 뒤의 자리 정수만 변하게 처리
20     for (int i = 1; i < 10; i = i + 1)
21     {
22         System.out.print(2 * i + " ");
23     }
24     System.out.println();
25 }
26 }
```

- 17 : 앞 부분의 실행 결과와 뒷부분의 실행 결과를 구분하는 구분선을 출력합니다.
- 20 : 반복 횟수가 정해진 경우 반복 처리를 위한 내용이 여기 한 줄에 표시됩니다.
- 20 : 반복 횟수를 기록하는 정수형 변수 i를 만들고 정수 0을 초깃값으로 대입합니다.
- 20 : 연산식 결과가 true인 경우 이후 중괄호 안의 내용이 실행됩니다.
- 22 : 변수의 값을 이용해 구구단을 출력합니다.
- 23 : 반복문의 중괄호가 끝나면 다시 20라인에서 뒷부분인 증감식이 실행됩니다.

이렇게 반복문을 적용할 수 있습니다. 반복 횟수를 증가시키고 싶다면 역시 중간 연산식 조건만 바꾸면 됩니다.

이제 코드를 컴파일하고 실행해봅시다.

To Do 01 우측 콘솔창을 마우스로 클릭하고 다음과 같이 입력해 자바 소스 파일을 바이트 코드로 컴파일합니다.

```
javac Exam06.java
```

02 컴파일이 끝나면 java 명령어를 이용해서 실행시켜봅니다. 실행 결과가 출력됩니다.

```
Console  Shell
> javac Exam06.java
> java Exam06
2 4 6 8 10 12 14 16 18
=====
2 4 6 8 10 12 14 16 18
>
```

구분선 앞 부분은 반복적인 코드의 입력에 의한 출력이고, 구분선 뒷부분은 for문에 의한 출력입니다. 반복문 적용이 잘된 것을 확인할 수 있습니다.

9 반복문의 중첩

반복문은 앞에서 본 것처럼 하나만 단독으로 사용할 수도 있지만, 반복문 안에 반복문이 있을 수도 있습니다. 이럴 경우 반복문의 중첩이라는 표현을 사용합니다.

9.1 for문의 중첩

다음 예제를 보면 반복문 안의 내용이 기존 반복문을 만들었을 때처럼 반복문을 적용할 수 있는 내용입니다.

```
01 class Exam07
02 {
03     public static void main(String[] args)
04     {
05         // 반복문 : Exam06 참고
06         // 조건의 연산식을 충족하는 동안 중괄호 안의 내용을 실행
07         for (int i = 1; i < 10; i = i + 1)
08         {
09             System.out.print(2 * i + " ");
10         }
11         System.out.println();
12
13         System.out.println("=====");
14
15         // 반복문 : 구구단 2단에서 9단 출력
```

```

16     for (int i = 2; i < 10; i = i + 1)
17     {
18         // 반복문 안에서 반복문을 적용할 대상이 또 있음
19         System.out.print(2 * i + " ");
20         System.out.print(3 * i + " ");
21         System.out.print(4 * i + " ");
22         System.out.print(5 * i + " ");
23         System.out.print(6 * i + " ");
24         System.out.print(7 * i + " ");
25         System.out.print(8 * i + " ");
26         System.out.print(9 * i + " ");
27         System.out.println();
28     }
29 }
30 }

```

9.2 for 중첩 코드 작성

Exam06에서 반복문으로 만든 것과 똑같은 방법으로 19라인에서 26라인까지를 반복문으로 바꿀 수 있습니다. 들여쓰기에 신경써서 변경 처리를 하면 다음과 같이 코드를 추가할 수 있습니다.

```

29
30     System.out.println("=====");
31
32     // 반복문의 중첩
33     for (int i=2; i<10; i=i+1)
34     {
35         for (int j = 2; j < 10; j = j + 1)
36         {
37             System.out.print(j * i + " ");
38         }
39         System.out.println();
40     }
41 }
42 }

```

Exam07.java

33 : 반복 횟수를 기록하는 정수형 변수 i를 만들고 정수 2를 초깃값으로 대입합니다.

35 : 반복 횟수를 기록하는 정수형 변수 `j`를 만들고 정수 2를 초깃값으로 대입합니다.

반복 의사를 분명히 알고 있는 곳에서 사용하는 변수이므로 특별히 긴 이름을 사용하지 않고 관례적으로 `i`, `j`를 사용합니다.

그리고 초깃값은 무조건 0부터 시작하는 것은 아니고 이 예제에서 보는 바와 같이 필요에 따라 바뀔 수 있습니다. 여기서는 2단부터 출력하기 위해 초깃값에 2를 대입했습니다.

Tip 33라인 `for`문 소괄호 안을 보면 붙여쓰기가 되어 있는데 35라인 `for`문 소괄호 안은 띄어쓰기가 되어 있습니다. 둘 다 가능합니다. 이 부분은 어떻게 쓰라는 정답이 없습니다. 프로그래머 개개인의 취향입니다. 컴파일러는 붙여쓰기를 하던 띄어쓰기를 하던 세미콜론 `;`로 한 문장씩 구분하여 컴파일을 해줍니다.

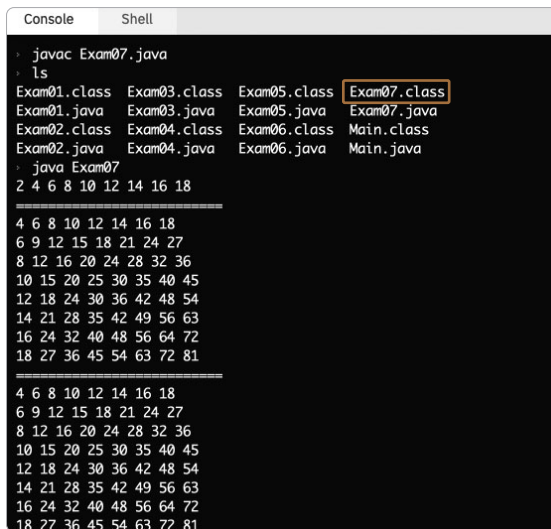
이제 코드를 컴파일하고 실행해봅시다.

To Do 01 우측 콘솔창을 마우스로 클릭하고 다음과 같이 입력해 자바 소스 파일을 바이트 코드로 컴파일합니다.

```
javac Exam07.java
```

02 1~2초 커서가 깜빡이다가 프롬프트가 다음 줄로 바뀝니다. 아무런 메시지 없이 다음 프롬프트 화면이 뜬다면 정상적으로 컴파일된 겁니다. `ls` 명령어를 입력합니다. 컴파일 결과로 만들어진 `Exam07.class`를 확인합니다.

03 `java` 명령어를 이용해서 실행시켜봅니다. 실행 결과가 출력됩니다.



```
Console  Shell
> javac Exam07.java
> ls
Exam01.class  Exam03.class  Exam05.class  Exam07.class
Exam01.java   Exam03.java   Exam05.java   Exam07.java
Exam02.class  Exam04.class  Exam06.class  Main.class
Exam02.java   Exam04.java   Exam06.java   Main.java
> java Exam07
2 4 6 8 10 12 14 16 18
4 6 8 10 12 14 16 18
6 9 12 15 18 21 24 27
8 12 16 20 24 28 32 36
10 15 20 25 30 35 40 45
12 18 24 30 36 42 48 54
14 21 28 35 42 49 56 63
16 24 32 40 48 56 64 72
18 27 36 45 54 63 72 81

4 6 8 10 12 14 16 18
6 9 12 15 18 21 24 27
8 12 16 20 24 28 32 36
10 15 20 25 30 35 40 45
12 18 24 30 36 42 48 54
14 21 28 35 42 49 56 63
16 24 32 40 48 56 64 72
18 27 36 45 54 63 72 81
```

이렇게 중첩된 반복문을 사용하면 구구단을 쉽게 출력할 수 있습니다.

Tip 이론적으로는 중첩의 단계는 더 깊어질 수 있으나 보통은 깊은 중첩은 사용하지 않습니다. 사람이 코드를 보면서 머리 속으로 생각할 수 있어야 프로그래밍의 로직을 파악하거나 디버깅을 할 수 있습니다. 따라서 직관적으로 이해할 수 있는 코드를 작성해야 합니다.

퀴즈 기존 코드를 수정하여 다음처럼 출력을 해봅시다.

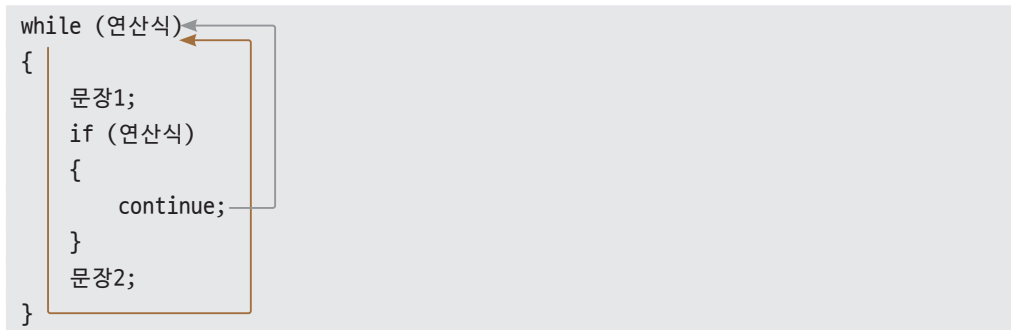
2단	:	4	6	8	10	12	14	16	18
3단	:	6	9	12	15	18	21	24	27
4단	:	8	12	16	20	24	28	32	36
5단	:	10	15	20	25	30	35	40	45
6단	:	12	18	24	30	36	42	48	54
7단	:	14	21	28	35	42	49	56	63
8단	:	16	24	32	40	48	56	64	72
9단	:	18	27	36	45	54	63	72	81

10 반복문 고급 사용

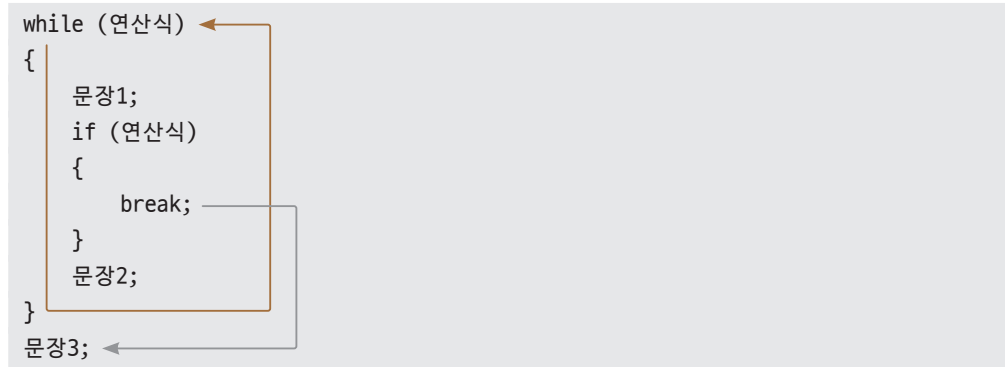
반복문은 무조건 반복만 하는 것이 아니라 조건에 따라 건너뛰거나 혹은 어떤 조건이 되면 중단시킬 수도 있습니다. 조건에 따라 반복문의 실행을 건너뛸 때는 `continue`를 사용하고, 조건에 따라 반복문을 중지시킬 때는 `break`를 사용합니다.

10.1 while문에서 break, continue 사용하기

while문에서 `continue`가 사용되면 그 이후의 문장은 실행하지 않고 조건을 검사하는 연산식으로 바로 이동하게 됩니다.



while문에서 break가 사용되면 while문의 실행을 그 즉시 중지하고 반복문 이후의 문장을 실행하기 위해 중괄호를 빠져 나갑니다.



10.2 코드 작성

앞에서 한 방식대로 Exam08.java 파일을 만들고 기본형을 만듭니다. 그리고 다음 코드를 보고 편집기창에 입력하면 됩니다.

```
01 class Exam08 {
02     public static void main(String[] args) {
03         int num = 0;
04         while (true)
05         {
06             num = num + 1;
07
08             // 짝수이면 이후 실행을 건너뛰기
09             if (num % 2 == 0)
10             {
11                 continue;
12             }
13
14             // 특정한 조건이 되면 멈추기
15             if (num > 10)
16             {
17                 break;
18             }
19         }
```

```

20         System.out.println(num);
21     }
22
23     // break로 멈추면 여기로 실행 이동
24     System.out.println("while문 종료");
25 }
26 }

```

03 : 반복 횟수를 체크하는 정수형 변수 num을 만들고 초깃값으로 정수 0을 대입합니다.

06 : num 변수의 값을 1 증가시킵니다.

09 : 2로 나누어서 나머지가 0이면 짝수입니다. 짝수인지 체크합니다.

11 : 반복문의 이후 실행을 건너뛰고 while문의 연산식 결과를 체크하기 위해 04라인으로 이동합니다.

15 : 반복문을 중지하기 위해 num 변수에 저장된 값이 10보다 큰 지를 검사합니다.

17 : while문을 중지하고 22라인으로 이동합니다.

이제 코드를 컴파일하고 실행해봅시다.

To Do 01 우측 콘솔창을 마우스로 클릭하고 다음과 같이 입력해 자바 소스 파일을 바이트 코드로 컴파일합니다.

```
javac Exam08.java
```

02 1~2초 커서가 깜빡이다가 프롬프트가 다음 줄로 바뀝니다. 아무런 메시지 없이 다음 프롬프트 화면이 뜬다면 정상적으로 컴파일된 겁니다. ls 명령어를 입력합니다. 컴파일 결과로 만들어진 Exam08.class를 확인합니다.

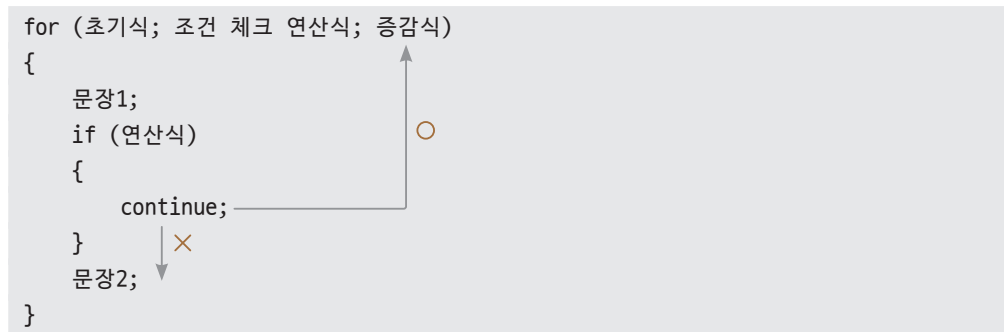
03 java 명령어를 이용해서 실행시켜봅니다. 실행 결과가 출력됩니다.


```
Console  Shell
> javac Exam08.java
> ls
Exam01.class Exam03.class Exam05.class Exam07.class Main.class
Exam01.java Exam03.java Exam05.java Exam07.java Main.java
Exam02.class Exam04.class Exam06.class Exam08.class
Exam02.java Exam04.java Exam06.java Exam08.java
> java Exam08
1
3
5
7
9
while 반복문 종료
>
```

실행 결과로 10보다 작은 정수 중에 홀수만 출력되었습니다.

10.3 for문에서 break, continue 사용하기

for문에서 continue가 사용되면 그 이후의 문장은 실행하지 않고 증감식으로 바로 이동하게 됩니다.



for문에서 break가 사용되면 for문의 실행을 그 즉시 중지하고 반복문 이후의 문장을 실행하기 위해 중괄호를 빠져 나갑니다.

```

for (초기식; 조건 체크 연산식; 증감식)
{
    문장1;
    if (연산식)
    {
        break;
    }
    문장2;
}
문장3;

```

10.4 코드 작성

앞에서 한 방식대로 Exam09.java 파일을 만들고 기본형을 만듭니다. 그리고 다음 코드를 보고 편집기창에 입력하면 됩니다.

```

01 class Exam09 {
02     public static void main(String[] args) {
03         int num = 0;
04         while (true)
05         {
06             num = num + 1;
07             // 짝수이면 이후 실행을 건너뛰기
08             if (num % 2 == 0)
09             {
10                 continue;
11             }
12
13             // 특정한 조건이 되면 멈추기
14             if (num > 10)
15             {
16                 break;
17             }
18
19             System.out.println(num);
20         }
21
22         // break로 멈추면 여기로 실행 이동

```

Exam09.java

```

23     System.out.println("반복문 종료");
24 }
25 }

```

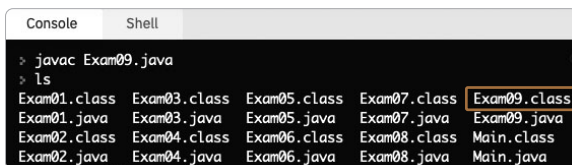
- 03 : 증감을 체크할 용도의 변수 num을 만들고 0으로 초기화합니다.
- 04 : 소괄호 안의 값이 true이므로 while문은 무조건 중괄호 안의 내용을 실행합니다.
- 06 : num 변수의 값을 1 증가시킵니다.
- 08 : 2로 나누어서 나머지가 0이면 짝수입니다. 짝수인지 체크합니다.
- 10 : 반복문의 이후 실행을 건너뛰우고 while문의 연산식 체크를 위해 04라인으로 이동합니다.
- 14 : 반복문을 중지하기 위해 num 변수에 저장된 값이 10보다 큰지를 검사합니다.
- 16 : for문을 중지하고 23라인으로 이동합니다.

이제 코드를 컴파일하고 실행해봅시다.

- To Do 01** 우측 콘솔창을 마우스로 클릭하고 다음과 같이 입력해 자바 소스 파일을 바이트 코드로 컴파일합니다.

```
javac Exam09.java
```

- 02** 1~2초 커서가 깜빡 거리다가 프롬프트가 다음 줄로 바뀝니다. 아무런 메시지 없이 다음 프롬프트 화면이 뜬다면 정상적으로 컴파일된 겁니다. ls 명령어를 입력합니다. 컴파일 결과로 만들어진 Exam09.class를 확인합니다.

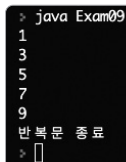


```

Console  Shell
> javac Exam09.java
> ls
Exam01.class  Exam03.class  Exam05.class  Exam07.class  Exam09.class
Exam01.java   Exam03.java   Exam05.java   Exam07.java   Exam09.java
Exam02.class  Exam04.class  Exam06.class  Exam08.class  Main.class
Exam02.java   Exam04.java   Exam06.java   Exam08.java   Main.java

```

- 03** java 명령어를 이용해서 실행시켜봅니다. 실행 결과가 출력됩니다.



```

> java Exam09
1
3
5
7
9
반복문 종료
>

```

실행 결과로 10보다 작은 정수 중에 홀수만 출력된 것을 볼 수 있습니다.

11 사용자 입력받기

프로그램을 실행하다 보면 사용자 입력을 받아 프로그램의 데이터로 사용하는 경우가 있습니다. 이번 예제에서는 사용자 입력을 어떻게 받는지 알아보겠습니다.

11.1 Scanner 사용하기

사용자 입력 처리는 많은 프로그램과 프로그래머가 사용하기 때문에 자바에 이미 기능이 만들어져 있습니다. 사용자가 기능을 직접 만들 필요는 없고 사용하기만 하면 됩니다. 하지만 모든 프로그램에서 사용자 입력을 받는 것이 아닙니다. 그러므로 사용자 입력을 받아 처리할 필요가 있을 때만 미리 만들어져 있는 모듈을 포함시켜 프로그램을 만들면 됩니다.

자바에서는 이런 기능을 패키지 또는 클래스라는 것에 기능별로 모아 만들어두었습니다. 프로그래머는 필요한 기능이 있다면 해당 기능이 들어 있는 패키지나 클래스를 우리 프로그램에 포함시켜 사용하면 됩니다.

Tip 평소에 사용하지도 않는 골프가방이 차 트렁크에 들어 있다면 차의 무게만 늘어서 차 연비가 나빠지게 됩니다. 우리가 만드는 프로그램에서도 사용하지도 않는 기능을 마구잡이로 포함시키면 프로그램 용량만 커지게 됩니다.

11.2 코드 작성

앞에서 한 방식대로 Exam10.java 파일을 만들고 기본형을 만듭니다. 그리고 다음 코드를 보고 편집기창에 입력하면 됩니다.

```
01 import java.util.Scanner;
02
03 class Exam10
04 {
05     public static void main(String[] args)
06     {
07         Scanner sc = new Scanner(System.in);
```

Exam10.java

```

08     System.out.print("숫자를 입력하고 엔터를 치세요:");
09     int num = sc.nextInt();
10     System.out.println("당신이 입력한 숫자는 : " + num);
11 }
12 }

```

01 : 입력 기능을 제공하는 패키지입니다. 무조건 써줍니다.

07 : 입력 기능을 사용하려면 무조건 구현해야 합니다. `sc`는 변수명이므로 다른 이름으로 써줄 수 있습니다.

09 : `sc.nextInt()`는 사용자로부터 정수를 입력받습니다. 정수를 입력받았으므로 정수형 타입 변수 `num`에 대입합니다.

10 : 변수에 저장된 값을 출력합니다.

사용자로부터 입력을 받으려면 일단은 01, 07라인의 내용을 그대로 사용해주고, 09라인의 `sc.nextInt()`를 이용해 정수를 입력받아 정수형 타입 변수에 값을 대입시킵니다(이론적으로는 많은 얘기를 할 수 있지만, 이론을 모르거나 혹은 이론을 많이 안다고 해도 입력받는 기능을 사용하는 것은 똑같습니다. 자세한 내용은 5.2절 '콘솔 입력'에서 다룹니다).

Tip 07라인에서 만들어지는 변수인 `sc`를 객체라고 합니다. 객체는 일반 변수처럼 값을 대입받아 저장할 수도 있지만 여러 기능도 가질 수 있습니다. 그런 여러 가지 기능 중에서 `nextInt()`는 사용자로부터 정수를 입력받습니다.

이제 코드를 컴파일하고 실행해봅시다.

To Do 01 우측 콘솔창을 마우스로 클릭하고 다음과 같이 입력해 자바 소스 파일을 바이트 코드로 컴파일합니다.

```
javac Exam10.java
```

02 1~2초 커서가 깜빡이다가 프롬프트가 다음 줄로 바뀝니다. 아무런 메시지 없이 다음 프롬프트 화면이 뜬다면 정상적으로 컴파일된 겁니다. `ls` 명령어를 입력합니다. 컴파일 결과로 만들어진 `Exam10.class`를 확인합니다.

03 `java` 명령어를 이용해서 실행시켜봅시다. 입력을 받는 곳에서 사용자 입력을 받을 때까지 프로그램의 실행이 잠시 멈춥니다.

```

Console  Shell
> javac Exam10.java
> ls
Exam01.class  Exam03.java  Exam06.class  Exam08.java  Main.class
Exam01.java  Exam04.class  Exam06.java  Exam09.class  Main.java
Exam02.class  Exam04.java  Exam07.class  Exam09.java
Exam02.java  Exam05.class  Exam07.java  Exam10.class
Exam03.class  Exam05.java  Exam08.class  Exam10.java
> java Exam10
숫자를 입력하고 엔터를 치세요 :

```

04 그림처럼 10을 입력하면 입력한 숫자를 다시 보여줍니다.

```

> java Exam10
숫자를 입력하고 엔터를 치세요 :10
당신이 입력한 숫자는 :10
>

```

자바에서 사용자 입력을 받는 기능을 이처럼 간단하게 만들 수 있습니다.

12 사용자 입력받아 사칙연산 결과 출력하기

이번에는 사용자로부터 두 정수를 입력받아 사칙 연산 결과를 출력하는 프로그램을 만들어보겠습니다. 지금까지 공부해온 것을 조금만 응용하면 충분히 만들 수 있는 내용입니다.

12.1 코드 작성 1단계

앞에서 한 방식대로 Exam11.java 파일을 만들고 기본형을 만듭니다. 그리고 일단 두 정수를 입력 받는 코드를 다음과 같이 만듭니다.

```

01 import java.util.Scanner;
02
03 class Exam11
04 {
05     public static void main(String[] args)
06     {
07         Scanner sc = new Scanner(System.in);
08         System.out.println("숫자를 입력하고 엔터를 치세요.");
09
10         System.out.print("첫 번째 숫자:");
11         int num1 = sc.nextInt();

```

```

12
13     System.out.print("두 번째 숫자:");
14     int num2 = sc.nextInt();
15
16 }
17 }

```

```

> javac Exam11.java
> java Exam11
숫자를 입력하고 엔터를 치세요.
첫 번째 숫자 :2
두 번째 숫자 :3

```

일단 입력 기능을 사용하기 위해 01라인과 07라인의 코드를 적용합니다.

- 01 : 입력 기능을 사용하려면 그대로 적용합니다.
- 07 : 입력 기능을 사용하려면 그대로 적용합니다. sc는 변수명이므로 변경해도 됩니다.
- 08 : 사용자에게 입력하라는 안내 문구를 출력합니다.
- 10 : 안내 문구를 출력하고 줄바꿈을 하지 않고 프롬프트를 옆으로 놓아둡니다.
- 11 : 정수형 변수 num1에 첫 번째로 입력한 정수를 대입합니다.
- 13 : 안내 문구를 출력하고 줄바꿈을 하지 않고 프롬프트를 옆으로 놓아둡니다.
- 14 : 정수형 변수 num2에 두 번째로 입력한 정수를 대입합니다.

Exam10에서 보았던 내용에서 조금만 응용하면 이렇게 두 정수를 입력받을 수 있습니다.

12.2 코드 작성 2단계

이제 두 변수에 정수 데이터가 저장되었으므로 두 변수를 사용해서 사칙연산을 하고 출력만 하면 됩니다. 다음 코드를 Exam11.java에 이어서 작성합니다.

```

15
16     int result1 = num1 + num2;
17     System.out.println(num1 + " + " + num2 + " = " + result1);
18
19     int result2 = num1 - num2;
20     System.out.println(num1 + " - " + num2 + " = " + result2);
21
22     int result3 = num1 * num2;

```

Exam11.java

```

23     System.out.println(num1 + " * " + num2 + " = " + result3);
24
25     int result4 = num1 / num2;
26     System.out.println(num1 + " / " + num2 + " = " + result4);
27
28     int result5 = num1 % num2;
29     System.out.println(num1 + " % " + num2 + " = " + result5);
30 }
31 }

```

25 : 정수형 변수를 만들고 나누기 연산에서 몫을 구해서 저장합니다.

27 : 정수형 변수를 만들고 나누기 연산에서 나머지를 구해서 저장합니다.

12.3 실행

이제 코드를 컴파일하고 실행해봅시다.

- To Do** 01 우측 콘솔창을 마우스로 클릭하고 다음과 같이 입력해 자바 소스 파일을 바이트 코드로 컴파일합니다.

```
javac Exam11.java
```

- 02 1~2초 커서가 깜빡이다가 프롬프트가 다음 줄로 바뀝니다. 아무런 메시지 없이 다음 프롬프트 화면이 뜬다면 정상적으로 컴파일된 겁니다. ls 명령어를 입력합니다. 컴파일 결과로 만들어진 Exam11.class를 확인합니다.
- 03 java 명령어를 이용해서 실행시켜봅니다. 입력을 받는 곳에서 사용자 입력을 받을 때까지 프로그램의 실행이 잠시 멈춥니다.


```
Console Shell
> javac Exam11.java
> ls
Exam01.class Exam03.java Exam06.class Exam08.java Exam11.class
Exam01.java Exam04.class Exam06.java Exam09.class Exam11.java
Exam02.class Exam04.java Exam07.class Exam09.java Main.class
Exam02.java Exam05.class Exam07.java Exam10.class Main.java
Exam03.class Exam05.java Exam08.class Exam10.java
> java Exam11
숫자를 입력하고 엔터를 쳐세요.
첫 번째 숫자 :5
두 번째 숫자 :2
5 + 2 = 7
5 - 2 = 3
5 * 2 = 10
5 / 2 = 2
5 % 2 = 1
>
```

안내 문구에 따라 두 번 정수를 입력하면 입력받은 두 정수를 이용해서 사칙연산을 하고 그 결과가 출력됩니다. 그동안 공부한 것을 조금만 응용해도 이처럼 프로그램을 쉽게 작성할 수 있습니다.

13 종합 : 계산기 만들기

이번에는 메뉴를 구성해서 출력된 메뉴에서 사용자가 사칙연산 중 하나의 계산 기능을 선택하게 하고, 다시 사용자가 입력한 값에 따라 해당 계산만 수행하여 결과를 출력하고 다시 메뉴를 출력하는 단순한 계산기 기능을 만들어보겠습니다.

이번 예제는 앞서 다룬 예제보다 쉽니다. 그래서 총 3단계로 나눠 차근차근 완성해나갈 겁니다. 지금까지 배운 내용 만으로도 계산기 같은 프로그램을 만들 수 있다는 자신감을 심어주고자 준비했습니다. 이제부터 함께 만들어봅시다.

STEP 1 13.1 메뉴 출력 구현하기

앞에서 한 방식으로 Exam12.java 파일을 만들고 기본형을 만듭니다. 그리고 일단 메뉴를 출력하는 코드를 작성합니다.

```
01 import java.util.Scanner;
02
03 class Exam12
04 {
```

Exam12.java

```

05     public static void main(String[] args)
06     {
07         Scanner sc = new Scanner(System.in);
08
09         System.out.println("메뉴를 선택하세요.");
10         System.out.println("1. 더하기");
11         System.out.println("2. 빼기");
12         System.out.println("3. 곱하기");
13         System.out.println("4. 나누기");
14         System.out.println("0. 끝내기");
15
16     }
17 }

```

01 : 입력 기능을 사용하기 위해 그대로 적용합니다.

07 : 입력 기능을 사용하기 위해 그대로 적용합니다. sc는 변수명이므로 변경해도 됩니다.

일단 입력 기능을 사용하기 위해 01라인과 07라인의 코드를 적용합니다. 그리고 09라인에서 14라인까지 사용자에게 보여줄 메뉴 항목을 작성해 출력합니다.

여기까지 만들고 컴파일한 후 실행시켜 메뉴가 잘 출력되는지 확인합니다.

▼ 실행 결과

```

Console  Shell
> javac Exam12.java
> ls
Exam01.class  Exam04.class  Exam07.class  Exam10.class  Main.class
Exam01.java   Exam04.java   Exam07.java   Exam10.java   Main.java
Exam02.class  Exam05.class  Exam08.class  Exam11.class
Exam02.java   Exam05.java   Exam08.java   Exam11.java
Exam03.class  Exam06.class  Exam09.class  Exam12.class
Exam03.java   Exam06.java   Exam09.java   Exam12.java
> java Exam12
메뉴를 선택하세요.
1. 더하기
2. 빼기
3. 곱하기
4. 나누기
0. 끝내기
>

```

STEP 2 13.2 사용자 입력 처리하기

이제 메뉴가 사용자 입력을 처리하고 계속해서 출력되도록 반복문을 이용하여 만들어줍니다. 횟수가 정해지지 않고 끝나는 조건만 있는 이런 경우에는 while문이 for문보다 잘 어울립니다.

```
01 import java.util.Scanner;
02
03 class Exam12
04 {
05     public static void main(String[] args)
06     {
07         Scanner sc = new Scanner(System.in);
08
09         while (true)
10         {
11             System.out.println("메뉴를 선택하세요.");
12             System.out.println("1.더하기");
13             System.out.println("2.빼기");
14             System.out.println("3.곱하기");
15             System.out.println("4.나누기");
16             System.out.println("0.끝내기");
17
18             int num = sc.nextInt();
19             if (num == 0)
20             {
21                 break;
22             }
23             else
24             {
25                 if (num > 4)
26                 {
27                     System.out.println("메뉴를 잘못 선택했습니다.");
28                     continue;
29                 }
30
31                 // 더하기, 빼기, 곱하기, 나누기 실행
32
33             }
34         }
35         System.out.println("계산기를 종료합니다.");
```

```

36     }
37 }

```

- 09 : while문을 이용하여 반복문을 만들고 그 안에 메뉴 출력 부분을 넣습니다.
- 18 : 사용자로부터 정수를 입력받습니다.
- 19 : 입력받은 값이 0이면 반복문을 종료합니다.
- 23 : 입력받은 값이 0이 아니고 메뉴의 다른 값이면 else 부분의 중괄호에서 처리를 합니다.
- 28 : 잘못된 메뉴 번호가 입력되면 메시지를 출력하고 다시 메뉴 선택을 할 수 있도록 합니다
- 35 : 반복문이 종료되면 메시지를 출력하고 프로그램을 종료합니다.

여기까지 만들고 컴파일한 후 실행시켜 반복문에서 메뉴가 잘 출력되는지, 입력을 받고 입력받은 값을 잘 처리하는지 확인합니다. 일단 입력받은 값이 0이 아닐 때 메뉴가 반복적으로 잘 출력되고, 입력받은 값이 0일 때 프로그램이 종료하면 잘 처리되고 있다고 볼 수 있습니다.

▼ 사칙연산 메뉴 선택

```

Console  Shell
> javac Exam12.java
> java Exam12
메뉴를 선택하세요 .
1.더하기
2.빼기
3.곱하기
4.나누기
0.끝내기
1
메뉴를 선택하세요 .
1.더하기
2.빼기
3.곱하기
4.나누기
0.끝내기
2
메뉴를 선택하세요 .
1.더하기
2.빼기
3.곱하기
4.나누기
0.끝내기

```

▼ 없는 메뉴 선택

```

메뉴를 선택하세요 .
1.더하기
2.빼기
3.곱하기
4.나누기
0.끝내기
6
메뉴를 잘못 선택했습니다 .
메뉴를 선택하세요 .
1.더하기
2.빼기
3.곱하기
4.나누기
0.끝내기

```

▼ 종료 메뉴 선택

```

메뉴를 선택하세요 .
1.더하기
2.빼기
3.곱하기
4.나누기
0.끝내기
0
계산기를 종료합니다 .
>

```

STEP 3 13.3 기능 구현하기

이제 해당 메뉴의 기능을 만들어주면 됩니다. if 조건문을 이용하여 선택한 메뉴에 맞는 기능을 수행할 수 있도록 코드를 추가합니다. 기존 코드의 31번 라인의 주석을 지우고 해당 부분에 다음 코드를 추가해 완성하면 됩니다.

```

30
31     System.out.print("첫 번째 숫자:");
32     int num1 = sc.nextInt();
33
34     System.out.print("두 번째 숫자:");
35     int num2 = sc.nextInt();
36
37     if (num == 1)
38     {
39         int result = num1 + num2;
40         System.out.println(num1 + " + " + num2 + " = " + result);
41     }
42     else if (num == 2)
43     {
44         int result = num1 - num2;
45         System.out.println(num1 + " - " + num2 + " = " + result);
46     }
47     else if (num == 3)
48     {
49         int result = num1 * num2;
50         System.out.println(num1 + " * " + num2 + " = " + result);
51     }
52     else if (num == 4)
53     {
54         int result1 = num1 / num2;
55         System.out.println(num1 + " / " + num2 + " = " + result1);
56
57         int result2 = num1 % num2;
58         System.out.println(num1 + " % " + num2 + " = " + result2);
59     }
60 }
61 }
62 System.out.println("계산기를 종료합니다.");
63 }
64 }

```

여기까지 작성을 했으면 코드를 컴파일하고 실행해봅시다.

▼ 더하기 선택

```
Console Shell
> javac Exam12.java
> java Exam12
메뉴를 선택하세요 .
1.더하기
2.빼기
3.곱하기
4.나누기
0.끝내기
1
첫 번째 숫자 :3
두 번째 숫자 :4
3 + 4 = 7
메뉴를 선택하세요 .
1.더하기
2.빼기
3.곱하기
4.나누기
0.끝내기
```

▼ 나누기 선택

```
메뉴를 선택하세요 .
1.더하기
2.빼기
3.곱하기
4.나누기
0.끝내기
4
첫 번째 숫자 :5
두 번째 숫자 :2
5 / 2 = 2
5 % 2 = 1
메뉴를 선택하세요 .
1.더하기
2.빼기
3.곱하기
4.나누기
0.끝내기
```

▼ 끝내기 선택

```
메뉴를 선택하세요 .
1.더하기
2.빼기
3.곱하기
4.나누기
0.끝내기
0
계산기를 종료합니다 .
```

우리가 의도한 대로 잘 실행됩니다. 하지만 이 프로그램은 프로그램에서 원하는 대로 메뉴에 대한 숫자를 정확히 입력하지 않으면 에러를 낼 수 있습니다. 또한 입력값을 사용자의 실수로 숫자가 아닌 공백이나 문자를 입력해도 에러가 발생합니다.

사용자의 실수로 인한 입력에도 에러가 발생하지 않도록 처리하는 것을 예외 처리라고 합니다. 이 부분은 본 수업에 가서 배워보겠습니다(16장 ‘예외 처리’ 참조).

하지만 지금까지 배워온 것만으로도 지금과 같은 훌륭한 프로그램을 만들 수 있습니다. 자바에 대한 흥미가 많이 생겼으면 하는 바람입니다.

학습 마무리

저는 자바 프로그래밍을 다음처럼 크게 세 부분으로 나눕니다.

- 프로그래밍 기초
- 클래스 기초
- 클래스 심화

우리가 여기 선수 수업에서 배운 부분은 프로그래밍 기초 중에서도 기초입니다. 그러나 이 정도만 해도 어느 정도의 프로그램은 작성할 수 있습니다. 여기서 추가적으로 더 배워야 할 것은 더 어려운 프로그램을 만들 지식이 아니고 세밀하고 섬세한 프로그램을 만드는 지식입니다. 앞 부분에서 잠깐

언급했듯이 에러를 내지 않기 위한 지식입니다. 하지만 그것도 조금만 더 배우면 됩니다.

이번 장에서는 복잡한 설치와 어려운 이론 없이 자바를 사용해서 프로그램을 작성해보았습니다. 프로그램을 만드는 것이 재미있었고 흥미로웠기를 바랍니다.

여기까지 따라 오시느라 수고하셨습니다. 본 수업에서 다시 만나기를 바랍니다.

※ 알려드려요

이 책은 《이재환의 자바 프로그래밍 입문》의 부록입니다.

뒷표지의 ISBN과 가격은 본책과 부록을 합한 정보입니다.

딱 필요한 만큼, 자바 핵심 문법과 개념 이해 중심으로 배우는 새로운 자바 입문서

12년간 강의를 해보니 무엇을 이해하지 못하는지, 무엇을 어려워하는지, 현업 나가서 당장 쓸모 있는 기법과 그렇지 못한 기법이 무엇인지 알게 되었습니다. 프로그래머처럼 생각하고, 프로그래머로 안착할 분께 딱 필요한 만큼 알려드립니다. 초보자도 쉽고 명확하게 이해할 수 있도록 학습 목표를 일목요연하게 제시하고 핵심 내용을 정리해 보여줍니다. 이론도 실습으로 직접 체험하여 체득하게 했습니다. 단계별로 간단한 프로젝트를 구현합니다. 프로그래밍에 입문해 프로그래머로 취업하고 싶은 분들이라면 자바에 도전해 보세요.

0 아무것도 몰라도 OK

프로그래밍의 '표'도 모르는 분도 코딩이 가능하도록 설명합니다.

12 가지 선수 수업 예제

설치 없이 코딩하는 선수 수업을 만나보세요. 12가지 예제를 풀다 보면 자바에 더 친숙해질 겁니다.

4 단계로 익히는 자바

선수 수업, 자바 기초 프로그래밍, 자바 객체지향 프로그래밍, 자바 클래스 응용 프로그래밍을 다룹니다.

"스터디를 하면서 초보자에게 프로그래밍을 가르쳐 준 적이 있는데, 만약 이 책이 있었다면 이 책으로 진행했을 겁니다. '처음부터 코딩이 가능하고, 실제 동작하는 예제들이 있어서 지루하지 않은 이 책이 있었다면 스터디에 큰 도움이 되었을 텐데'하는 생각이 드네요."

이호훈 TVING 프로그래머

"어려운 용어보다는 현실과 가까운 용어로 설명합니다. 마치 멘토를 실제로 만나서 듣는 듯한 설명이 와닿습니다. 헛갈리는 개념은 무조건 다 집어주고, 배운 내용으로 만들 수 있는 프로젝트가 소소한 재미를 선사합니다."

강은혜 University of the Potomac 학생

환경 구축

0 단계

입문

1 단계

객체지향

2 단계

응용
프로그래밍

3 단계

IT전문서 / 프로그래밍 언어

979-11-971498-7-0 9 5 0 0 0



정가 25,000원 ISBN 979-11-971498-7-0

★★★ 반드시 내 것으로 ★★★

#MUSTHAVE

27년 개발, 12년 강의 노하우를 담은 자바 입문서가 나타났다

이재환의 자바 프로그래밍 입문

Must Have 시리즈는 내 것으로 만드는 시간을 드립니다.
명확한 학습 목표와 핵심 정리를 제공하고, 간단명료한 설명과 다양한 그림으로 학습 효과를 극대화합니다. 설명과 예제를 제공해 응용력을 키워줍니다. 할 수 있습니다. 포기 없습니다. 지금 당장 밑줄 긋고 메모하고 타이핑하세요!
Must Have가 여러분의 성장을 돕겠습니다.

골든래빗은 가치가 성장하는 도서를 함께 만드실 저자님을 찾고 있습니다.

내가 할 수 있을까 망설이는 대신, 용기 내어 골든래빗의 문을 두드려보세요.

apply@goldenrabbit.co.kr

이 책은 대한민국 저작권법의 보호를 받습니다.

일부를 인용 또는 재사용하려면 반드시 저자와 골든래빗(주)의 동의를 구해야 합니다.

우리는
가치가 성장하는
시간을
만듭니다.

추천의 말

이 책은 원고 단계에서 베타 리딩을 진행했습니다. 보내주신 의견을 바탕으로 더 좋은 원고로 만들어 출간합니다. 참여해주신 모든 분께 감사드립니다.

자바 프로그래머

프로그래밍 자체가 처음인 분이나 환경 설정만 하다가 힘이 빠지신 분에게 도움이 될 책입니다. 스터디를 하면서 초보자에게 프로그래밍을 가르쳐 준 적이 있는데, 만약 이 책이 있었다면 이 책으로 진행했을 겁니다. ‘처음부터 코딩이 가능하고, 실제 동작하는 예제들이 있어서 지루하지 않은 이 책이 있었다면 스터디에 큰 도움이 되었을 텐데’하는 생각이 드네요.

이호훈 TVING 프로그래머

화려한 표현보다는 간결한 정리로 중요한 내용만 쏙쏙 전달해주는 안내자 같은 책입니다. 프로그래밍 입문자뿐만 아니라 다른 프로그래밍 언어를 다뤄본 적 있는 분에게도 추천합니다. 프로그래밍 기초 개념을 자바로 풀어놓았기 때문에 도움이 많이 됩니다. 다른 책보다 넓은 범위를 다루기 때문에 기존에 자바를 다루고 있지만 다시금 정리가 필요한 분들에게도 도움이 될 겁니다.

송종근 《이것이 iOS다》 저자

프로그래밍 입문자

기초부터 탄탄히 기초체력부터 채워간 좋은 기본서입니다. 꼼꼼하게 여러 가능성을 코드와 설명으로 풀어줍니다. 어려운 용어보다는 현실과 가까운 용어로 설명합니다. 마치 멘토를 실제로 만나서 듣는 듯한 설명이 와닿습니다. 헷갈리는 개념은 무조건 다 집어주고, 배운 내용으로 만들 수 있는 프로젝트가 소소한 재미를 선사합니다. 초보자를 겨냥한 프로젝트 책을 출간해 주시면 꼭 읽어볼 거 같습니다.

강은혜 University of the Potomac 학생

자바라는 언어를 꺼리는 사람 또는 자바를 잘 모르는 사람에게 추천합니다. 이 책은 재밌고 깊게 자바를 알려줍니다. 그래서 기초를 탄탄하게 잡을 수 있을 거라 확신합니다.

제정민 대구소프트웨어고등학교 학생

타 언어 프로그래머

초보자 눈높이로 설명했지만 완전 말랑말랑한 책은 아닙니다. 실제 자바 수업을 현장에서 듣는 듯한 느낌을 받았습니다. 단순히 코드를 따라 치는 수준이 아니라 컴퓨터 작동 원리, 알고리즘 같은 컴퓨터 과학도 함께 배울 수 있습니다.

송진영 프로그래머

자바라는 언어는 자칫 어려워 보일 수도 있지만, 대한민국의 자바 사랑에는 다 이유가 있습니다. 이 책을 통해 코딩에 입문하게 된다면 코딩을 하는 즐거움뿐 아니라, 자바라는 언어의 기초도 탄탄하게 익힐 수 있을 겁니다.

최규민 서울대학교 데이터 사이언티스트

자바 인기가 떨어지는 요즘 가뭄에 단비 같은 책입니다. 자바로 프로그래밍에 입문하는 분들께 추천합니다. 자바를 통해서 어플리케이션을 어떻게 개발하는지 알 수 있습니다.

최희욱 프로그래머

저자와 3문 3답

Q 취준생이 자바를 선택해야 하는 이유가 있을까요?

웹과 모바일 개발을 생각한다면 자바로 결론지을 수 있습니다.

특히 우리나라에서는 공공기관이나, 금융, 통신, 유통 등 대부분의 업무 영역에서 자바로 시스템을 구축하여 사용합니다. 그렇기에 구인 수요도 다른 언어에 비해 훨씬 많습니다. 우리나라에서 자바 언어는 프로그래밍의 기초 체력과 같습니다.

Q 자바 언어의 특징을 알려주세요.

자바는 변수, 상수, 함수(메서드) 개념과 각종 연산자, 표현식 개념이 있으며 객체지향 개념을 가지고 있습니다. 버전 8에 함수형 개념이 도입되면서 현대적 언어의 모든 요소를 가지고 있게 되었습니다.

Q 자바로 개발할 수 있는 분야를 알려주시겠어요?

흔히 볼 수 있는 웹 사이트 개발뿐만 아니라, 기업에서 사용하는 ERP나 웹 앱 등이 자바를 베이스로 하는 프레임워크를 이용하여 만들어집니다. 그리고 안드로이드 앱 개발도 자바로 하게 됩니다. 빅데이터 관련한 플랫폼인 하둡 자체도 자바로 개발되어 있습니다. 이처럼 자바는 특정 분야에 국한되어 사용되지 않고 아주 다양한 분야에서 널리 사용되고 있는 프로그래밍 언어입니다.

숫자로 보는 책의 특징

0

아무것도 몰라도 OK

프로그래밍을 전혀 모르는 분을 대상으로 합니다. 어려운 환경 설치 없이 웹에서 코딩을 체험할 수 있는 <선수 수업>으로 시작합니다.

3

가지를 챙겨드립니다

첫 코딩 맛이 중요하니까? 27년 개발, 12년 강의 베테랑인 저자가 코딩 재미, 프로그래밍 개념 장착, 탄탄한 기본기 모두 챙겨드립니다.

3

가지 프로젝트 제공

배운 내용만으로 만들 수 있는 간단하지만 유용한 프로젝트를 제공합니다.

★☆☆☆ 계산기 만들기

★☆☆☆ 정렬 알고리즘 만들기

★★★★ 주소록 만들기

4

단계 코딩 챌린지

0단계 <선수 수업>으로 코딩을 손에 익힌 다음, 1단계에서 자바 기초, 2단계에서 자바 객체지향, 3단계에서 클래스 응용 프로그래밍을 배웁니다.

12

<선수 수업> 예제 제공

자동차를 속속들이 몰라도 차를 몰 수 있듯이, 자바 문법 모두를 몰라도 자바 프로그램을 짤 수 있습니다. 최소한의 문법을 알려드리고 손으로 타이핑하며 재미를 느낄 수 있게 12가지 예제를 담은 <선수 수업>을 제공합니다.

180

본문 예제

180개가 넘는 예제를 활용해 설명합니다. 책에 등장하는 예제를 하나씩 따라 할 때마다 프로그래밍 실력이 차곡차곡 쌓이도록 충실히, 때로는 그림을 활용해서 설명했습니다.

대상 독자에게 드리는 편지



프로그래밍을 배운 적이 없는 초보자에게

자바를 배워 프로그래밍을 하는 것은 게임을 하는 것과 비슷합니다. 보스몹을 잡기 전 공략을 숙지하고 공략에 익숙해져야 보스몹을 잡을 수 있는 것과 같습니다. 다만 프로그래밍에서는 보스몹이 좀 많습니다. 외워야 할 공략이 조금 많은 것이죠.

자신이 1렙부터 직접 키워 만렙이 된 캐릭터라면 사용하는 모든 스킬을 정확하게 알 것이고 스킬들의 연계를 쉽고 자연스럽게 할 수 있습니다. 그러나 남이 키워준 만렙 캐릭터라면 스킬들을 내가 정확히 모르기 때문에 효율적으로 사용하지 못합니다. 기술의 효과도 모를 수 있습니다.

이렇기 때문에 프로그래밍에서도 모든 예제를 직접 다 입력해보고 실행해보면서 공부를 해야 합니다. 남의 코드를 눈으로만 보거나 머리로만 생각하면 프로그래밍 실력이 늘지 않습니다.

그리고 많은 선배나 책에서 말하는 코딩 요령이 있습니다. “코딩을 할 때는 이렇게 줄을 맞춰라, 변수명은 이렇게 지어라, 메서드명은 이렇게 지어라” 등은 대체로 “이렇게 하면 누구나 보스를 잡을 수 있어”라고 하는 방식입니다. 이것이 프로그래밍에서는 문법이고 프로그래밍을 할 때의 규칙입니다. 따라만 해도 초보자한테 좋은, 선배들이 주는 경험치입니다.

저는 이 책을 통해 꼭 필요한 기술들에 대해 빠르고 쉽게 익숙해질 수 있도록 단계별로 올바른 순서를 제시할 뿐입니다. 경험치는 스스로 열심히 해야 얻으실 수 있습니다.



프로그래밍을 배웠지만 이해가 안 되고 어렵다는 **프로그래밍 초보자**에게

프로그래밍을 공부한다는 것은 프로그래밍 언어를 배우고 이해하는 것이 아닙니다. 이해가 안 돼서 못하겠다는 것은 변명입니다. 이해가 아니고 규칙을 외우고 적용하는 익숙함의 문제이기 때문입니다.

자전거를 탈 때 어떻게 중심을 잡는지, 어떻게 페달을 밟는지 등 원리를 배우고 이해했다고 자전거를 잘 타게 되는 것은 아닙니다. 연습을 통해 익숙해져야 자전거를 잘 타게 되는 겁니다 (도레미파솔라시를 배워서 안다고 연주를 잘 할 수는 없습니다. 또한 앞에서 말한 것처럼 공략을 외웠다고 보스몹을 그냥 잡을 수는 없습니다. 공략을 익혀야 하는 겁니다).

프로그래밍도 마찬가지로 이해가 아닌 익숙함이 프로그래밍을 잘하게 해줍니다. 강의를 해보니 프로그래밍이 안 되는 학생들의 공통점은 모든 것을 이해를 하려고 합니다. 규칙에 대한 이해는 규칙을 외우는 데 도움을 줄 수 있지만 모든 규칙에 이해가 필요한 것은 아닙니다. 프로그래밍은 규칙을 외우고 연습으로 익숙해져야 하는 것이고 이해가 안 된 규칙이라도 외워서 익숙해지면 프로그래밍은 할 수 있습니다.

프로그래밍에 익숙해지는 가장 좋은 방법은 여러 규칙에 대한 예제 코드를 하나 하나 직접 입력해보면서 실행해보는 겁니다. 눈으로 읽고 머리로 이해하는 것이 아닌 손을 이용한 타이핑이 여러분을 프로그래밍에 익숙해지는 가장 빠른 길로 안내해줄 겁니다.

이 책을 보는 방법

1 프로젝트 제시

* 프로젝트 장에서만 제시합니다.

2 학습 개요 안내

학습 목표와 순서, 핵심 내용을 일목요연하게 제시합니다.

26.2 주소록 만들기

이제부터 시작합니다. 1. 프로젝트 생성 2. PhoneInfo 클래스 만들기 3. PhoneInfo 클래스의 필드 4. PhoneInfo 클래스의 메서드 5. PhoneInfo 클래스의 테스트	이제부터 시작합니다. 1. 프로젝트 생성 2. PhoneInfo 클래스 만들기 3. PhoneInfo 클래스의 필드 4. PhoneInfo 클래스의 메서드 5. PhoneInfo 클래스의 테스트
--	--

MUST HAVE

지금까지 배운 모든 내용을 적용하여 전화번호부를 작성하고 조회하는 전화번호 기능을 단계별로 만들어보겠습니다.

학습 순서

1 프로젝트 생성	1 프로젝트 생성
2 PhoneInfo 클래스 만들기	2 PhoneInfo 클래스 만들기
3 PhoneInfo 클래스의 필드	3 PhoneInfo 클래스의 필드
4 PhoneInfo 클래스의 메서드	4 PhoneInfo 클래스의 메서드
5 PhoneInfo 클래스의 테스트	5 PhoneInfo 클래스의 테스트
6 PhoneInfo 클래스의 테스트	6 PhoneInfo 클래스의 테스트
7 PhoneInfo 클래스의 테스트	7 PhoneInfo 클래스의 테스트
8 PhoneInfo 클래스의 테스트	8 PhoneInfo 클래스의 테스트

26.1 프로젝트 구성하기

이번 프로젝트에서는 전화번호부를 작성하고 조회하는 기능을 가진 프로그램을 만들어보겠습니다. 대어리는 다음과 같이 간단한 정보를 저장하고, 최소한의 기능만 만들어보겠습니다.

데이터	기능
이름	전화번호 입력
전화번호	전화번호 조회
이메일	전화번호 삭제
	전화번호 목록

대어리는 PhoneInfo 클래스를 생성하여 객체의 멤버 변수를 통해 저장하고 조회하겠습니다. 그리고 이 PhoneInfo 클래스를 사용하는 MyPhoneBook 클래스를 만들어 앞에서 말한 기능들을 구현해두겠습니다.

26.2 PhoneInfo 클래스 만들기

01 Chapter26로 프로젝트를 만듭니다. 그리고 전화번호부를 저장하는 클래스를 먼저 만들겠습니다. 02 step01 패키지 생성을 만들고 03 PhoneInfo 클래스를 만들어 추가합니다.

```

package com.example;
import java.util.*;

public class PhoneInfo {
    String name;
    String phoneNum;
    String email;
}
    
```

02 PhoneInfo 클래스는 정보를 저장할 클래스로 사용합니다. 대어리 사용 내용은 이름, 전화번호, 이메일입니다.

```

package step01;
import java.util.*;

public class PhoneInfo {
    String name;
    String phoneNum;
    String email;

    public PhoneInfo(String name, String phoneNum) {
        this.name = name;
        this.phoneNum = phoneNum;
    }

    public PhoneInfo(String name, String phoneNum, String email) {
        this.name = name;
        this.phoneNum = phoneNum;
        this.email = email;
    }

    public void showPhoneInfo() {
        System.out.println("이름 : " + name);
        System.out.println("전화번호 : " + phoneNum);
    }
}
    
```

04 번역일한 후 실행시켜 입력한 내용이 잘 출력되는지 확인합니다.

```

C:\Program Files\Java\jdk-1.8.0_101\bin> javac PhoneInfo.java
C:\Program Files\Java\jdk-1.8.0_101\bin> java PhoneInfo
이름 : 홍길동
전화번호 : 010-1234-5678
이메일 : hong@gmail.com
    
```

두 객체에서 저장된 정보가 잘 출력되었습니다.

26.3 메뉴 구성하기

01 step02 패키지 생성을 만듭니다. 그리고 step01 패키지에서 두 클래스를 복사해서 step02 패키지로 옮겨줍니다.

```

package step02;
import java.util.*;

public class PhoneInfo {
    String name;
    String phoneNum;
    String email;
}
    
```

복사에서 붙여넣으면 다음 그림처럼 자동으로 패키지명이 변경됩니다.

```

package step02;
import java.util.*;

public class PhoneInfo {
    String name;
    String phoneNum;
    String email;
}
    
```

02 MyPhoneBook에 다음과 같이 메인 메뉴 구성을 코드로 작성합니다. PhoneInfo 클래스를 테스트하면 기존 코드는 제거합니다.

```

package step02;
import java.util.*;

public class MyPhoneBook {
    public static void main(String[] args) {
        PhoneInfo info = new PhoneInfo("홍길동", "010-1234-5678", "hong@gmail.com");
        info.showPhoneInfo();
    }
}
    
```

ToDo 3

독자가 실습해야 하는 내용을 확실하게 알려드립니다.

STEP 4

길고 복잡한 내용도 길들일지 않게 단계별로 안내해드립니다.

이클립스와 최대한 같도록 행 번호
넣고 실행결과는 스샷으로!

유연도유리	종류	연산자
1		$\frac{1}{2} \mid \frac{1}{3} \text{ or } 0.5 \text{ or } 0.333333$
2	단항	$+x^2 \text{ or } -x^2 \text{ or } \sin x \text{ or } \cos x$
3	선형	$a \cdot x^2 + b \cdot x + c \text{ or } \frac{a}{b} \text{ or } \frac{a}{c}$
4	선형	$a \cdot x^2 + b \cdot x + c$
5	비교	$(x < y) \text{ or } (x > y) \text{ or } (x \leq y) \text{ or } (x \geq y)$
6	관계	$x \cdot y \text{ or } x / y \text{ or } x \% y$
7	논리곱	$x \& y$
8	논리합	$x \mid y$
9	조건	$\text{booleanExpression ? exp1 : exp2}$
10	입력	$x++ \text{ or } ++x \text{ or } x-- \text{ or } --x$

후의 코드의 많은 부분에서 괄호가 사용되었습니다. 연산자의 우선순위가 헷갈릴 땐 괄호를 사용하면 연산 순위의 의미가 확실해집니다. 소괄호는 표에서도 확인되듯 우선순위가 가장 높기 때문에 사용될 땐 먼저 연산을 해줍니다.

다음은 계산 순위를 이해하기 위한 몇 가지 순회 예제입니다.

```

1 public class Ex09_Order
2 {
3     public static void main(String[] args)
4     {
5         System.out.println(); // 0번 프린트()를 이용한 줄바꿈
6         System.out.println("a"); // 1번 줄을 0번 줄과 연결하는 줄바꿈
7         System.out.println("-----");
8
9         int num = 5;
10
11         System.out.println("num = " + num); // 1번 자료형변환을 이용한 연산
12         System.out.println("num = " + num);
13         System.out.println("num = " + num); // 0
14     }
15 }

```

```
19 }
20 }
```

▼ 실행 결과

Console 11
 Environment: C:\00_BuildLabel Java Application\ C:\00_BuildLabel\Java.m
 8.3 : 2016

몇까지 더하면 2000보다 커지는지 알아보는 코드입니다

▶ **별 의미는 없지만** 앞줄의 중점을 두기 위해서 무한 반복을 **while**로 변경했습니다. 그리고
▶ **반복문에 break를** myExit를 추가합니다. 중첩된 반복문을 단번에 종료합니다.

음향 대역폭은 63까지 다들 알고 계시는 2000보다 저저속 이펙트 발현범위를 훨씬 더 넓습니다. 그럼 음향 대역폭이 작을수록 음향이 실감될수록 단음으로 느껴질수록 악기들의 움직임이 break-off 될수 있습니다. 그럼 음향 폭이 넓은 음향대역폭의 장점을 보겠습니다.

학습 마무리

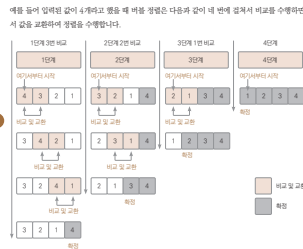
여기까지 자바 프로그래밍에서 사용하는 제어문을 알아보았습니다

핵심 요약

1. 제어문이란 프로그램의 진행 흐름을 필요에 따라 변경하고 실행을 제어하는 문이다.
2. 제어문에는 조건문과 반복문이 있다.
3. if문은 조건식의 결과가 'true'인 경우와 'false'인 경우의 두 가지 흐름을 만들어낼 수 있다.
4. if문은 조건식 결과를 단언의 형태로 만든다.
5. switch문을 사용하면 일정한 선택지들 가운데 하나를 할 수 있다.
6. 반복문은 어떤 조건이 성립하는 동안 반복 지위를 실행하는 제어문이다.
7. while문은 반복의 횟수보다는 조건 자체에 정해져 있을 때, for문은 반복의 횟수가 정해져 있을 때 적합하다.

7 실행 순서

그림을 이용해서 실행
순서를 알아봅니다.



비율 정렬은 배열을 순차 탐색하여 $i, i+1$ 번째 요소를 비교하여 큰 것을 뒤로 이동시키는 겁니다. 단계를 한 번 처리할 때마다 가장 큰 수가 배열의 마지막에 위치하게 됩니다. 그래서 다음 탐색부터는 마지막 요소는 비교 및 교환을 안 해도 됩니다.

정렬할 요소가 4개라면 위 그림처럼 4번의 단계를 걸쳐서 비교하여 정렬하게 되며, 각 단계별로 비교를 반복적으로 수행하면 됩니다. 즉, 중첩된 반복문과 같은 방식입니다.

STEP 1 19.1.2 프로젝트 생성

ToDo 01 ① Chapter19로 프로젝트를 만들되 ② BubbleSort 클래스를 만들어 추가합니다.

8 선수 수업

일단 해보면서 익히는 선수
수업 별책을 제공합니다.

학습 마무리 6

핵심 개념을
한 방에 정리해드립니다.

● **선수 수업**

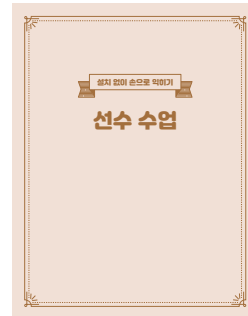
이 책의 구성

추후 현업에서 자바로 개발하는 데 필요한 기술을 단계별로, 그리고 익히기 쉬운 순서로 예제를 통해 설명합니다. 예제를 직접 손으로 입력해서 실행해보고, 미니 프로젝트도 만들어보며 빠르고 쉽게 자바 프로그래밍을 익힐 수 있도록 구성했습니다.

초보자를 위한 쉬운 책이라는 이유로 어려운 개념을 생략하거나 모호한 용어를 사용해 억지로 외우게 하지 않습니다. 자바를 자바답게 이해하기 위해 필요하다면 어려운 개념도 다룹니다. 하지만 초보자라도 잘 이해할 수 있도록 그림과 예제를 사용해 최대한 쉽게 설명하고 있습니다.

선수 수업

자바 선수 수업은 무작정 자바 코드를 보고 실행해보면서 프로그램이 어떻게 돌아가는지 눈과 손으로 확인하고 흥미를 유발하는 데 목적을 둡니다. 다른 언어를 익힌 현업 프로그래머라면 선수 수업을 건너뛰고 0장 '로컬 환경 설치'로 이동해도 좋습니다. 프로그래밍 입문자라면 하나하나 타이핑해가며 실습하기 바랍니다.



00장 자바 개발 환경 구축

윈도우에 자바 개발 환경을 구축해봅시다. 프로그래밍 입문자를 배려해 따라 하면 개발 환경이 구축되도록 안내합니다. 예제 코드를 내려받는 방법도 알아보겠습니다.

1단계 자바 기초 프로그래밍

모든 프로그램 언어에서 변수, 상수, 자료형, 표현식, 메서드 표현은 거의 유사합니다. 여기서는 자바를 통해 이런 프로그래밍의 기초를 배우게 됩니다. 여기서 배운 기초는 모든 프로그램 언어에서 거의 그대로 사용할 수 있습니다.

01장 Hello Java World

프로그래밍 언어가 무엇인지, 자바와 JVM은 어떤 관계인지 알아봅니다. 첫 자바 프로젝트

도 만들어보고 이클립스 사용법도 알아봅시다.

02장 자료형

프로그래밍을 이해하려면 하드웨어 동작 원리를 알아야 할 필요가 있습니다. 하드웨어를 제어하는 코드의 묶음이 프로그래밍이기 때문입니다. 하드웨어의 동작 이해와 함께 자료형을 배워보겠습니다.

03장 변수, 상수, 자료형의 형변환

선수 수업에서 배운 변수와 상수의 개념을 확장해봅니다. 그리고 자료형의 형변환이 무엇인지 알아봅니다.

04장 연산자

흔히 연산이라고 하면 수학 시간에 배운 사칙연산을 떠올리겠지만, 프로그래밍에서 사용하는 연산은 그외에도 다양한 종류가 있습니다. 우리가 흔히 생각할 수 있는 산술연산 외에도 대입 연산, 비교 연산, 증감 연산, 논리 연산 등이 있습니다. 자바에서 다루는 다양한 연산을 알아보겠습니다.

05장 콘솔 출력과 입력

우리는 이미 `System.out.println()`과 `System.out.print()`로 출력을 많이 해보았습니다. 그래서 출력을 간단히 정리하고, 입력을 더 살펴보겠습니다. 입력을 배우게 되면 지금까지 배운 것만으로도 재미있는 프로그램을 만들 수 있습니다.

06장 제어문

자바에서 다루는 다양한 제어문(`if`문, `switch`문, 반복문)을 알아봅니다.

07장 메서드와 변수의 사용 가능 범위

메서드를 만들고 사용하는 방법과 변수의 사용 가능한 범위를 알아보겠습니다.

08장 **Project** 계산기 만들기(선수 수업 업그레이드) ★☆☆☆

선수 수업 맨 마지막 단계에서 계산기를 만들었습니다. 출력된 메뉴에서 사용자가 사칙연산 중 하나를 선택하고, 다시 사용자가 입력한 값으로 계산을 수행하여 결과를 출력하고 다시

이 책의 구성

메뉴를 출력하는 계산기였습니다. 지금부터 구현할 계산기는 선수 수업에서 만든 계산기와 기능이 같습니다. 하지만 지금까지 배운 모든 내용을 적용해서 더 섬세한 프로그램으로 만들어보겠습니다.

2단계 자바 객체지향 프로그래밍

자바에서 다루는 객체지향 이론을 알아봅니다. 객체지향 4대 요소인 추상화, 캡슐화, 상속, 다형성을 자바에서는 어떻게 사용하는지 익히면서 클래스 사용 방법도 익히게 됩니다.

09장 클래스의 기초

클래스의 기본 개념을 알아보고, 클래스를 통해 객체를 생성하고 사용하는 방법을 알아봅니다.

10장 자바의 메모리 모델

자바에서 사용하는 메모리 모델의 구조를 이해하면 자바 프로그래밍에 큰 도움이 됩니다. 그러나 모든 것을 자세히 알 필요는 없습니다. 우리가 공부한 것과 연관해서 필요한 개념만 조금 이해하면 됩니다. 자바에서 사용하는 메모리 모델의 구조를 알아봅니다.

11장 스택의 이해

예약어 `static`은 변수 및 메서드, 그리고 지정한 영역에 붙일 수 있습니다. 자바 프로그래밍에서 스택틱 예약어를 붙이면 어떻게 동작하는지 직접 보면서 확인하고 이해합니다. 스택틱의 다양한 사용 방법도 알아봅니다.

12장 클래스의 상속

자바 클래스의 상속을 알아보고, 상속과 관련된 오버라이딩, 추상 클래스, 인터페이스, 다형성을 알아봅니다.

13장 패키지과 클래스 패스

클래스의 경로를 지정하는 데 사용하는 클래스 패스와 패키지를 알아봅니다.

14장 String 클래스

자바 String 클래스에서 다루는 다양한 메서드를 알아봅니다.

15장 배열

자바의 배열을 알아보고, 배열을 다룰 때 사용하는 Array 클래스를 알아봅니다.

16장 예외 처리

자바에서 다루는 다양한 예외 처리를 알아봅니다.

17장 자바의 기본 클래스

Object 클래스와 Object 클래스를 상속받아 여러 기본 기능을 제공해주는 다양한 기본 클래스를 알아봅니다.

18장 열거형, 가변 인수, 어노테이션

자바 프로그래밍을 할 때 큰 개념은 아니지만 프로그래밍을 편하게 해주는 소소한 요소들이 있습니다. 열거형, 가변 인수, 어노테이션 등이 그런 요소들입니다. 이번 장에서 가볍게 알아보겠습니다.

19장 **Project** 정렬 알고리즘 만들기 ★★☆☆

버블 정렬과 삽입 정렬 알고리즘을 알아보고 구현하겠습니다.

3단계 자바 클래스 응용 프로그래밍

자바 클래스에서 가장 빈번하게 사용한다고 할 수 있는 컬렉션 프레임워크를 통해 자바에서 다루는 자료구조와 사용법을 익히고, 람다식을 통해 자바에서 다루는 함수형 프로그래밍 기법을 배웁니다.

20장 제네릭

자바에서 다루는 제네릭을 알아봅니다.

이 책의 구성

21장 컬렉션 프레임워크

예전에는 자바 기본 문법을 배우고 알고리즘과 자료구조를 따로 더 배웠지만 지금의 자바에는 개발자들이 많이 사용하는 자료구조가 컬렉션 프레임워크에 구현되어 있습니다. 자바에서 제공하는 컬렉션 프레임워크를 알아봅니다.

22장 내부 클래스, 람다식

자바에서 사용하는 내부 클래스와 람다식을 알아봅니다.

23장 스트림

자바에서 다루는 스트림을 알아봅니다.

24장 입출력 스트림

자바에서 다루는 입출력 스트림을 알아봅니다.

25장 스레드

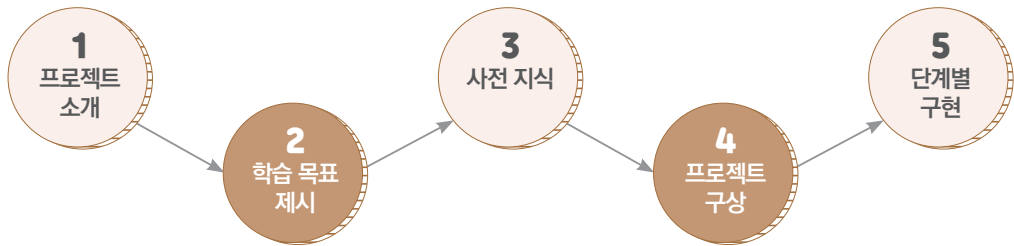
자바에서 다루는 스레드를 알아봅니다.

26장 **Project** 주소록 만들기 ★★★☆

지금까지 배운 모든 내용을 적용해서 전화번호를 저장하고 조회하는 전화번호부 기능을 단계별로 만들어보겠습니다.

프로젝트 소개

프로젝트를 체계적으로 확실하게 이끌어드립니다(프로젝트 특성에 따라 일부 단계를 생략하기도 합니다).



```
Console
MyCalculator [Java Application] C:\Dev\jdk-11.0.8\bin\javaw.exe
메뉴를 선택하세요.
1.더하기
2.빼기
3.곱하기
4.나누기
0.끝내기
1
```

계산기 만들기 (선수 수업 업그레이드) ★☆☆☆

선수 수업 때 만든 사칙연산 계산기를 업그레이드하세요(8장).

```
Console
<terminated> BubbleSort (1) [Java Application] C:\Dev\jdk-11.0.8\bin\javaw.exe
10개의 정수를 무작위로 입력하세요.
4 3 2 6 7 5 8 1 9 10
1 2 3 4 5 6 7 8 9 10
```

정렬 알고리즘 만들기 ★★☆☆

입력된 숫자들을 정렬하는 알고리즘을 만드세요 (19장).

```
Console
MyPhoneBook (6) [Java Application] C:\Dev\jdk-11.0.8\bin\javaw.exe (2021. 3. 7. 오후
[메뉴 선택]
1.전화번호 입력
2.전화번호 조회
3.전화번호 삭제
4.종료
선택 : 2
조회할 이름 : 전우치
Name : 전우치
PhoneNumber : 010-1234-6666
Email :

[메뉴 선택]
1.전화번호 입력
2.전화번호 조회
3.전화번호 삭제
4.종료
선택 :
```

주소록 만들기 ★★☆☆

프로그램을 종료해도 이전에 입력한 정보를 읽어와 사용할 수 있는 전화번호부 프로그램을 만들어봅시다(26장).

목차

추천의 말	004
저자와 3문 3답	006
숫자로 보는 책의 특징	007
대상 독자에게 드리는 편지	008
이 책을 보는 방법	010
이 책의 구성	012
프로젝트 소개	017
00 자바 개발 환경 구축	027
0.1 JDK 설치 및 설정	028
0.2 이클립스 설치	035
0.3 이클립스 메뉴 및 설정	041
0.4 예제 코드 다운로드 및 점검	047
학습 마무리	051

1

단계

자바 기초 프로그래밍

052

01 Hello Java World	055
1.1 프로그래밍 언어	056
1.2 자바	059
1.3 JVM	061
1.4 첫 자바 프로젝트 만들기	062
학습 마무리	068

02 자료형	069
2.1 진수 계산법	070
2.2 컴퓨터에서 데이터 처리 방식	072
2.3 자바 기본 자료형	074
학습 마무리	084
03 변수, 상수, 자료형의 형변환	085
3.1 변수	086
3.2 상수	088
3.3 자료형의 형변환	090
학습 마무리	096
04 연산자	097
4.1 산술 연산자	098
4.2 대입 연산자와 복합 대입 연산자	099
4.3 부호 연산자와 증감 연산자	101
4.4 비교 연산자(관계 연산자)	103
4.5 논리 연산자	105
4.6 조건 연산자	109
4.7 단항 • 이항 • 삼항 연산자	110
4.8 연산자 우선순위	110
학습 마무리	114
05 콘솔 출력과 입력	115
5.1 콘솔 출력	116
5.2 콘솔 입력	118
학습 마무리	121

06 제어문	123
6.1 조건문	124
6.2 반복문	130
학습 마무리	137
07 메서드와 변수의 사용 가능 범위	139
7.1 메서드 정의하기	141
7.2 메서드 종료하기	145
7.3 변수의 사용 가능 범위	146
학습 마무리	149
Project 08 계산기 만들기(선수 수업 업그레이드)	151
STEP 1 8.1 메뉴 만들기	153
STEP 2 8.2 메뉴 출력 및 사용자 입력	154
STEP 3 8.3 연산 기능 만들기	156
STEP 4 8.4 선택 메뉴 실행하기	158
STEP 5 8.5 유효성 검사	159
학습 마무리	161

2

단계

자바 객체지향 프로그래밍

162

09 클래스의 기초	165
9.1 객체	166
9.2 클래스	167

9.3	객체와 클래스	168
9.4	오버로딩	173
9.5	생성자	175
9.6	접근 제한자	178
	학습 마무리	182
10	자바의 메모리 모델	183
10.1	자바의 메모리 모델	184
10.2	디버깅하며 배우는 스택 영역 원리	186
10.3	디버깅하며 배우는 힙 영역 원리	195
10.4	디버깅하며 배우는 힙 영역 객체 참조	206
	학습 마무리	212
11	스태틱의 이해	213
11.1	스태틱	214
11.2	전역 변수로 사용	215
11.3	main()보다 먼저 실행	218
11.4	유틸 메서드로 사용	221
	학습 마무리	223
12	클래스의 상속	225
12.1	상속	226
12.2	오버라이딩	230
12.3	상속이 제한되는 final	233
12.4	추상 클래스	233
12.5	인터페이스	236
12.6	다형성	245

12.7 instanceof 연산자	250
학습 마무리	253
13 패키지과 클래스 패스	255
13.1 클래스 패스	256
13.2 패키지	262
13.3 패키지로 문제 해결	265
13.4 임포트	268
13.5 자바에서 기본 제공하는 패키지와 클래스	269
학습 마무리	270
14 String 클래스	271
14.1 String을 선언하는 두 가지 방법	272
14.2 문자열형 변수의 참조 비교	274
14.3 문자열형 변수의 내용 비교	276
14.4 String 클래스의 메서드	277
14.6 문자열 결합	283
14.7 문자열 분할	285
학습 마무리	287
15 배열	289
15.1 1차원 배열	290
15.2 for ~ each문	303
15.3 다차원 배열	305
15.4 배열 관련 유틸리티 메서드	309
학습 마무리	314

16 예외 처리	315
16.1 예외와 에러	316
16.2 예외 종류	317
16.3 예외 처리하기	320
16.4 예외 처리 미루기(던지기)	327
학습 마무리	333
17 자바의 기본 클래스	335
17.1 java.lang 클래스	336
17.2 Object 클래스	337
17.3 래퍼 클래스	346
17.4 Math 클래스	354
17.5 Random 클래스	356
17.6 Arrays 클래스	357
학습 마무리	362
18 열거형, 가변 인수, 어노테이션	363
18.1 열거형	364
18.2 가변 인수	368
18.3 어노테이션	370
학습 마무리	374
Project 19 정렬 알고리즘 만들기	375
19.1 버블 정렬	376
19.2 퀵스 : 삽입 정렬	382
학습 마무리	388

20 제네릭	393
20.1 제네릭의 필요성	394
20.2 제네릭 기반의 클래스 정의하기	401
20.3 제네릭 기반의 코드로 개선한 결과	402
20.4 매개변수가 여러 개일 때 제네릭 클래스의 정의	405
20.5 제네릭 클래스의 매개변수 타입 제한하기	407
20.6 제네릭 메서드의 정의	408
학습 마무리	410
21 컬렉션 프레임워크	411
21.1 자료구조	412
21.2 컬렉션 프레임워크의 구조	415
21.3 List<E> 인터페이스를 구현하는 컬렉션 클래스들	416
21.4 Set<E> 인터페이스를 구현하는 컬렉션 클래스들	425
21.5 Queue<E> 인터페이스를 구현하는 컬렉션 클래스들	439
21.6 Map<K, V> 인터페이스를 구현하는 컬렉션 클래스들	443
21.7 컬렉션 기반 알고리즘	447
학습 마무리	452
22 내부 클래스, 람다식	453
22.1 내부 클래스	454
22.2 멤버 내부 클래스	455
22.3 지역 내부 클래스	457
22.4 익명 내부 클래스	458

22.5 람다식	461
22.6 함수형 인터페이스	467
학습 마무리	468
23 스트림	469
23.1 스트림	470
23.2 중간 연산, 최종 연산	471
23.3 파이프라인 구성	472
23.4 컬렉션 객체 vs 스트림	473
23.5 여러 가지 연산들	474
학습 마무리	479
24 입출력 스트림	481
24.1 자바의 입출력 스트림	482
24.2 입출력 스트림의 구분	483
24.3 파일 대상 입출력 스트림 생성	485
24.4 보조 스트림	493
24.5 문자 스트림	495
학습 마무리	506
25 스레드	507
25.1 스레드의 이해	508
25.2 스레드 생성과 실행	510
25.3 스레드 동기화	516
25.4 스레드 풀	520
25.5 Callable & Future	525
25.6 ReentrantLock 클래스 : 명시적 동기화	527
25.7 컬렉션 객체 동기화	529

	학습 마무리	537
Project	26 주소록 만들기	539
	26.1 프로젝트 구상하기	541
STEP 1	26.2 PhoneInfo 클래스 만들기	542
STEP 2	26.3 메인 메뉴 구성하기	544
STEP 3	26.4 연락처 입력	547
STEP 4	26.5 연락처 조회	549
STEP 5	26.6 연락처 삭제	551
STEP 6	26.7 프로그램 종료 시 연락처 저장	553
STEP 7	26.8 프로그램 시작 시 연락처 로드	556
	학습 마무리	558
	부록 : ASCII 코드표	559
	에필로그	560
	용어 찾기	561
	코드 찾기	565

Chapter

00

자바 개발 환경 구축

#MUSTHAVE

☐ 학습 목표	윈도우에 자바 개발 환경을 구축해봅시다. 프로그래밍 입문자를 배려해 따라 하면 개발 환경이 구축되도록 안내합니다. 예제 코드를 내려받는 방법도 알아보겠습니다.
☐ 학습 순서	1 JDK 설치 및 설정 2 이클립스 설치 및 설정 3 예제 코드 다운로드
☐ JAVA 개발 환경 안내	자바를 개발하려면 자바 개발 키트 Java Development Kit, JDK를 설치해야 합니다. JDK로는 OracleJDK와 OpenJDK가 있는데, OpenJDK 11을 설치하겠습니다. ¹ 유료 라이선스인 OracleJDK와 달리 OpenJDK 11은 무료 라이선스이기 때문입니다(맥OS나, 리눅스 등에 설치되는 기본 버전이기도 합니다). 선수 수업에서 사용했던 콘솔창에도 OpenJDK 11이 설치되어 있었습니다. 통합 개발 환경으로는 이클립스를 사용하겠습니다.



알려드려요

이 책은 윈도우에 설치하는 방법만 다룹니다. 다른 운영체제에서는 일부 내용이 상이할 수 있습니다.

0.1 JDK 설치 및 설정

JDK 설치를 'JDK 설치→윈도우 환경 변수 등록→자바 실행 확인' 단계로 진행하겠습니다.

0.1.1 JDK 설치

OpenJDK 11을 내려받아 설치하겠습니다.

¹ 더 최신 버전을 선택해 설치해도 됩니다. 11 버전을 선택한 이유는 2021년 현재 실무에서 가장 많이 사용하기 때문입니다.

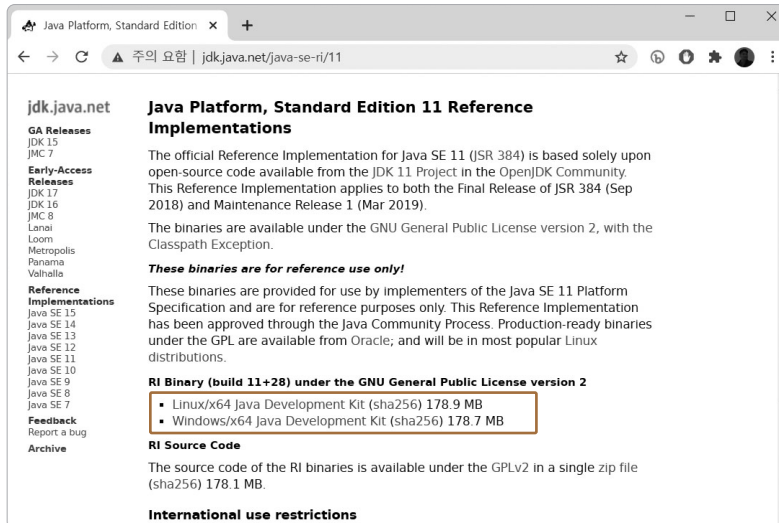
To Do 01 OpenJDK 다운로드 사이트로 접속합니다(OpenJDK는 오라클 홈페이지에서도 제공합니다).

- jdk.java.net

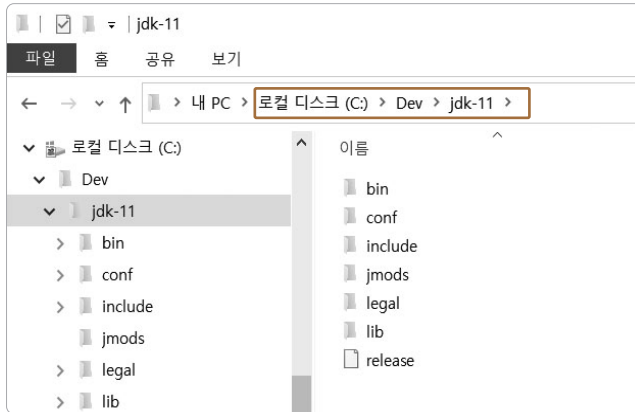
02 내려받을 버전으로 11을 선택합니다.



03 본인의 OS에 맞는 버전을 선택해서 다운받습니다.



- 04** 다운로드한 파일의 압축을 풀고 원하는 폴더로 이동시킵니다. 우리가 다운로드한 자바는 별도의 설치 과정을 진행하지는 않습니다. 저는 개발 도구를 하나의 폴더에 모아놓은 것을 선호하기 때문에 C 드라이브에 Dev 폴더를 만들고 그 하위 폴더로 압축 해제한 자바를 옮겨 놓겠습니다.



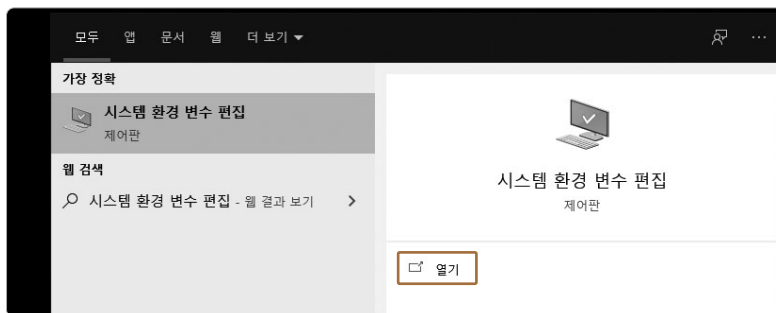
0.1.2 JDK 환경 변수 설정

윈도우 명령 프롬프트를 사용해서 컴파일하거나, 다른 프로그램에서 자바를 참조할 수 있으려면 환경 설정을 해주어야 합니다. 우리가 통합 개발 환경으로 사용할 이클립스도 자바 기반 프로그램이기 때문에 환경 설정이 제대로 되어 있지 않으면 실행되지 않습니다.

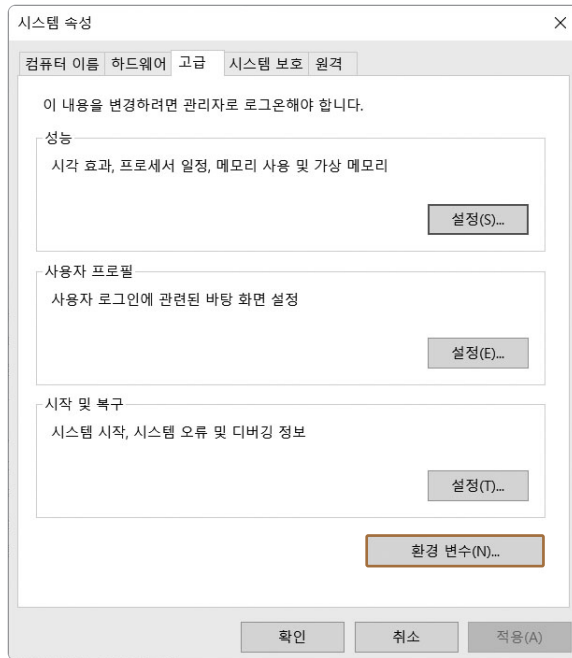
안내하는 순서대로 진행하면 어렵지 않게 환경 변수를 설정할 수 있습니다.

To Do 01 + **S**를 누르거나, 윈도우 검색창을 클릭합니다.

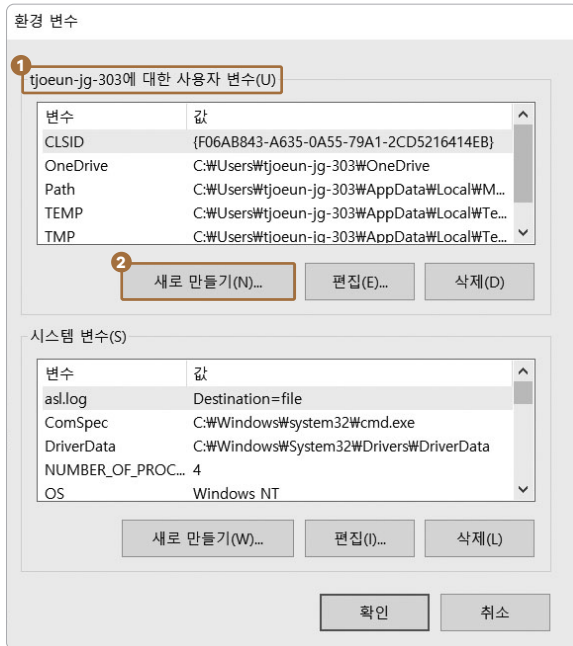
02 ① '시스템 환경 변수 편집'을 입력하고 ② 시스템 환경 변수 편집 [열기] 버튼을 클릭합니다.



03 시스템 속성 창이 열리면 [고급] 아래쪽에서 [환경 변수(N)...]를 선택합니다.



04 환경 변수 창이 열리면 환경 변수를 등록할 수 있습니다. ❶ [사용자 변수(U)]에서 ❷ [새로 만들기(N)...]를 선택합니다.



‘사용자 변수’와 ‘시스템 변수’ 선택

윈도우는 혼자 사용뿐 아니라 여럿이 사용하는 기능도 제공합니다. 그래서 윈도우가 부팅되고 나면 대기 화면에서 사용자를 선택해 로그인하게 됩니다. ‘사용자 변수’에 환경 변수를 등록하면 로그인한 아이디에만 해당 환경 변수가 적용되고, ‘시스템 변수’에 환경 변수를 등록하면 모든 사용자에게 환경 변수가 적용됩니다.

혼자만 사용할 때는 ‘사용자 변수’와 ‘시스템 변수’ 중 무얼 써도 무방하나, 학교나 학원에서 처럼 같은 PC를 여러 명이 사용할 때는 윈도우에 로그인하는 사용자 아이디를 여러 개 만들어 사용자를 변경해가면서 로그인하게 됩니다. 그래서 사용자가 여러 명이라면 환경 변수를 ‘사용자 변수’에 등록하는 것이 좋습니다.

- 05** 환경 변수를 입력하는 창이 열리면 ❶ 변수명으로 'JAVA_HOME'을 입력하고 ❷ 변수 값으로 JDK가 있는 경로를 입력(저와 같은 위치에 설치했다면 'C:\Dev\jdk-11' 입력)합니다.
- ❸ [확인]을 선택합니다.

새 사용자 변수

변수 이름(N): ❶ JAVA_HOME

변수 값(V): ❷ C:\Dev\jdk-11

❸ 확인 취소

디렉터리 찾아보기(D)... 파일 찾아보기(F)...

- 06** [사용자 변수]에서 기존의 Path를 더블 클릭하여 환경 변수를 입력할 창을 엽니다.

환경 변수

tjoen-jg-303에 대한 사용자 변수(U)

변수	값
CLSID	{F06AB843-A635-0A55-79A1-2CD5216414EB}
JAVA_HOME	C:\Dev\jdk-11
OneDrive	C:\Users\tjoen-jg-303\OneDrive
Path	C:\Users\tjoen-jg-303\AppData\Local\WM...
TEMP	C:\Users\tjoen-jg-303\AppData\Local\WTe...

새로 만들기(N)... 편집(E)... 삭제(D)

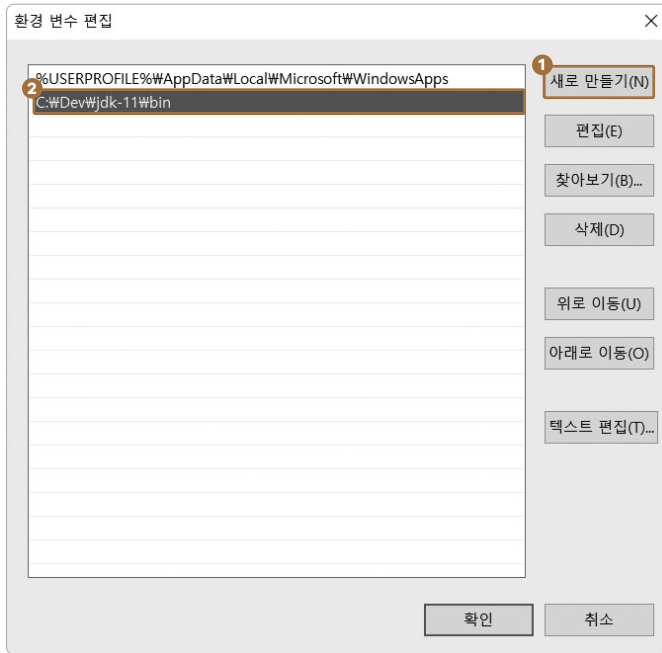
시스템 변수(S)

변수	값
asl.log	Destination=file
ComSpec	C:\Windows\system32\cmd.exe
DriverData	C:\Windows\System32\Drivers\DriverData
NUMBER_OF_PRO...	4
OS	Windows NT

새로 만들기(W)... 편집(I)... 삭제(L)

확인 취소

- 07 ① [새로 만들기(N)]를 선택하고, ② JDK가 설치된 폴더에서 bin 폴더의 경로를 입력합니다(저와 같은 위치에 설치했다면 'C:\Dev\jdk-11\bin' 입력).



Tip 앞에서 입력한 JAVA_HOME 환경 변수값을 이용하여 다음과 같이 적어 줄 수도 있습니다.
%JAVA_HOME%\bin

- 08 [확인]을 클릭해 창을 닫습니다.

0.1.3 자바 실행 확인

OpenJDK 환경 설정이 제대로 되었는지 확인해보겠습니다.

- To Do** 01 ① 윈도우 검색창에서 'cmd'를 입력하여 ② [명령 프롬프트]를 실행합니다.



02 창이 열리면 다음과 같이 입력하고 실행을 해서 버전을 확인해봅니다.

```
javac -version Enter
java -version Enter
```

우리가 설치한 버전이 표시된다면 환경 설정이 잘 된 겁니다.

0.2 이클립스 설치

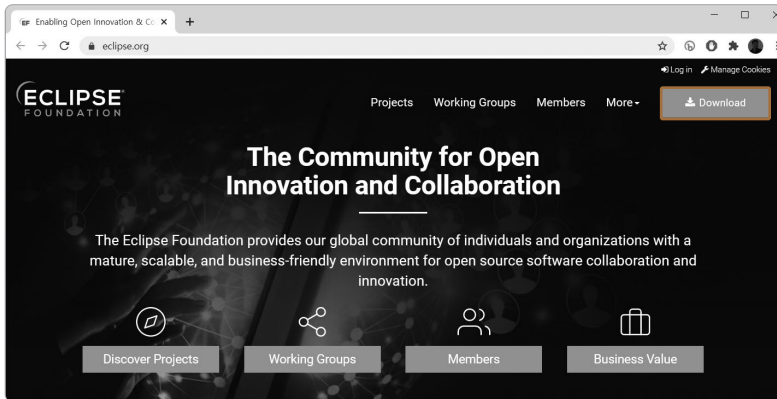
통합 개발 환경(integrated development environment, IDE)을 제공하는 이클립스를 설치해 자바 프로그래밍에 활용하겠습니다. 선수 수업 때처럼 일반 에디터를 이용해서 프로그램을 만들 수도 있지만 많이 불편하고 생산성도 떨어집니다. 참고로 이클립스가 자바 전용 개발 툴은 아니지만 전 세계 자바 개발자들이 애용하고 있습니다.

To Do 01 OS를 재부팅합니다. 그래야 새로 추가한 환경 설정이 반영됩니다.

02 이클립스 사이트에 접속합니다.

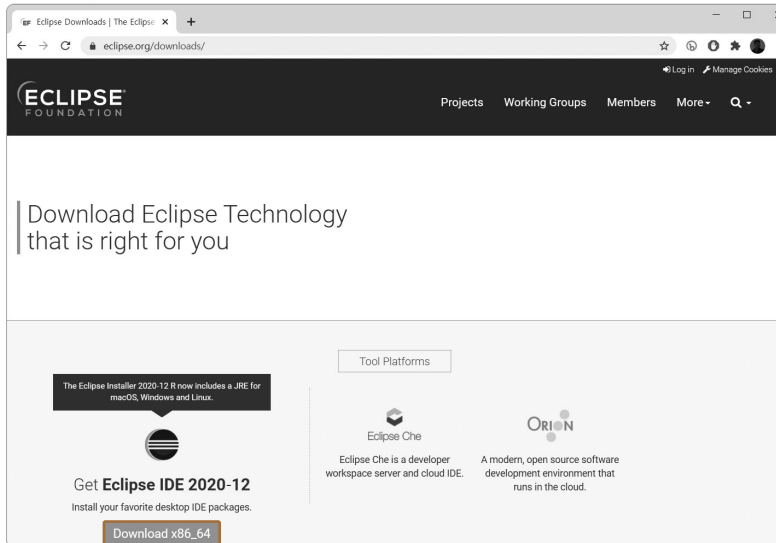
- www.eclipse.org

03 우측 상단 [Download]를 선택합니다.

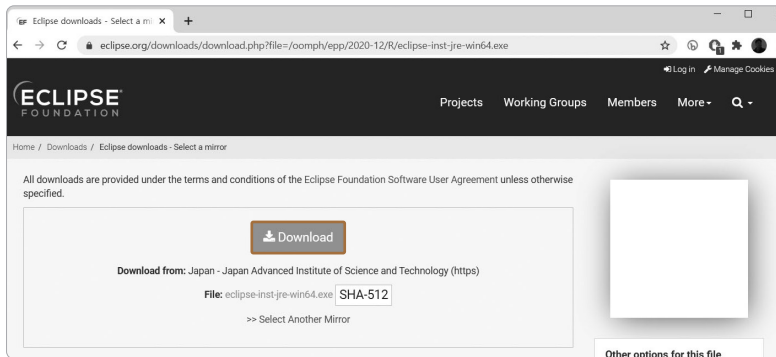


Notice 홈페이지 디자인은 변경될 수 있습니다. 화면 디자인이 변하더라도 [Download] 메뉴는 있을 것이니 주의 깊게 보고 메뉴를 선택하면 됩니다.

04 본인 OS에 맞는 이클립스를 다운로드합니다. 다운로드 버튼을 누르면 진짜로 다운로드를 할 수 있는 화면으로 이동합니다.



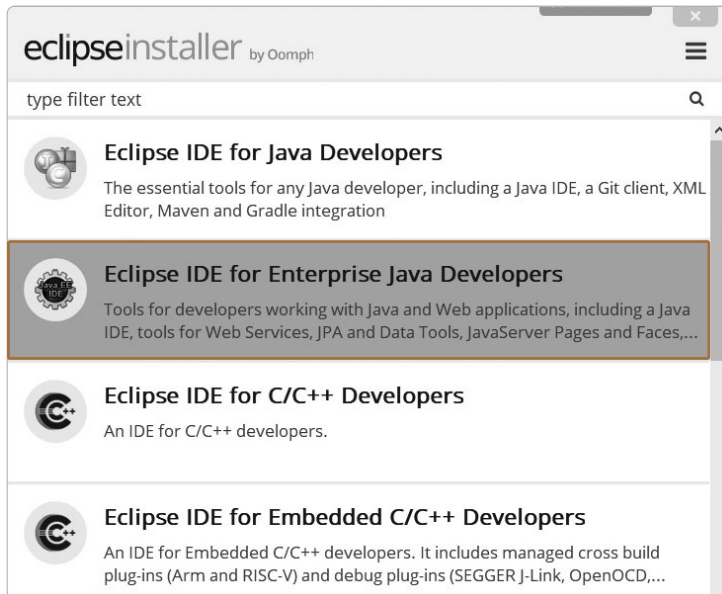
05 [Download] 버튼을 눌러 다운로드합니다.



- 06** 다운로드한 파일을 더블 클릭하여 실행하면 다음과 같은 창이 열릴 수 있습니다. 혹시 이런 창이 뜬다면 [실행(R)]을 선택하여 프로그램을 계속 실행시키세요(안 뜰 수도 있습니다).

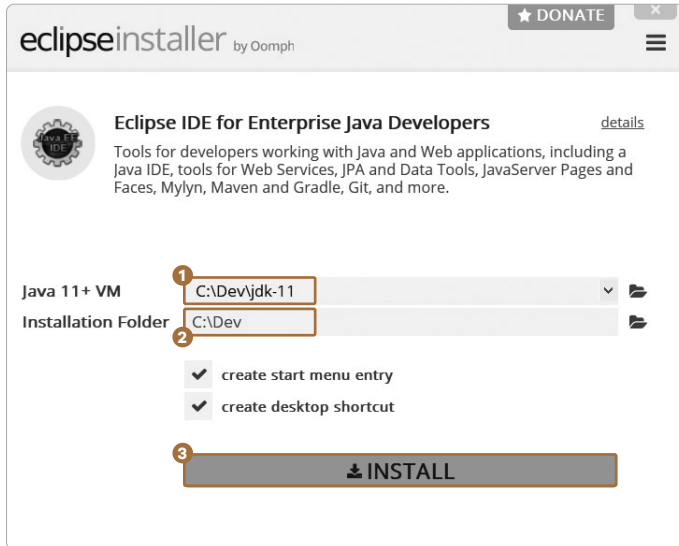


- 07** 설치 프로그램이 실행되면 설치할 버전을 선택하는 창이 열립니다. [Eclipse IDE for Enterprise Java Developers]를 선택하여 설치합니다.

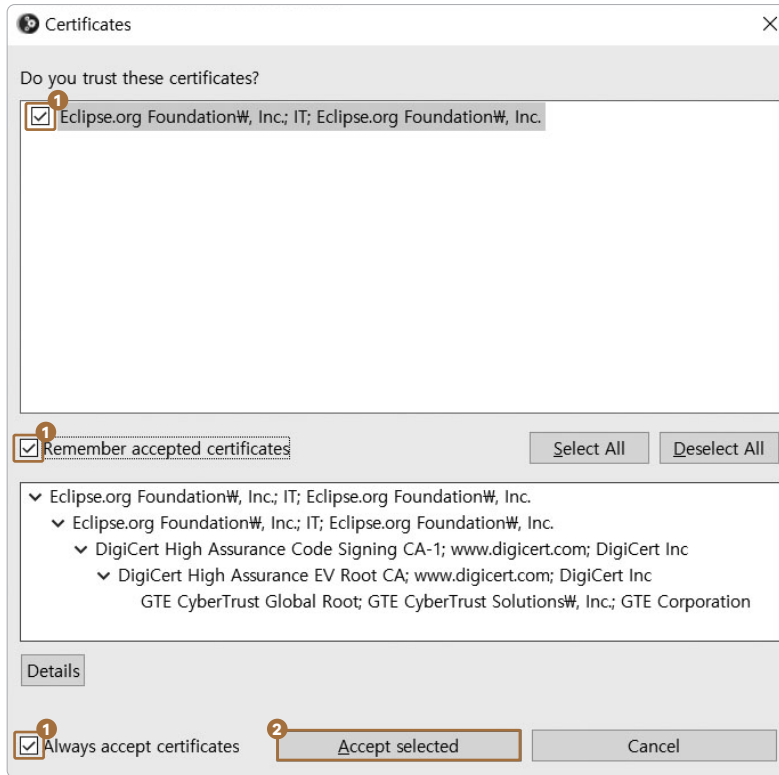


Note 순수하게 자바만 개발할 때는 [Eclipse IDE for Java Developers]를 선택해도 됩니다. 하지만 JSP / Servlet 등 웹 개발을 하려면 [Eclipse IDE for Enterprise Java Developers]를 선택하여 설치해야 합니다.

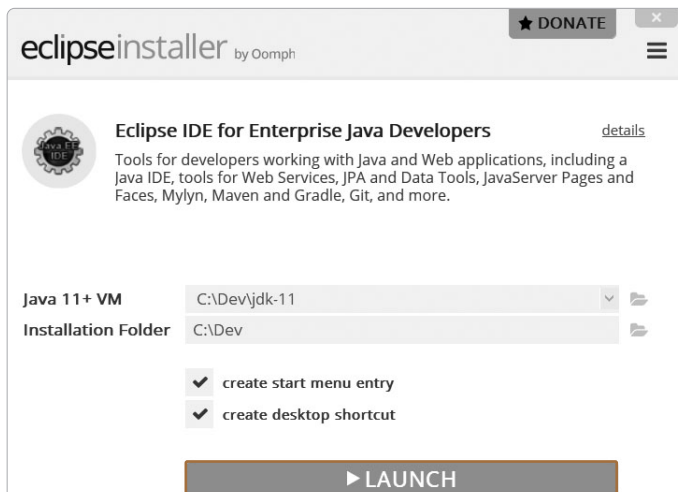
- 08 다음 화면에서 ❶ 'Java 11+VM'에는 앞에서 환경 변수로 등록한 JAVA_HOME이 미리 입력되어 있을 겁니다(그렇지 않다면 OS를 재부팅하거나 환경 설정이 제대로 되었나 확인 하세요). ❷ 'Installation Folder'에 설치할 폴더만 입력해줍니다(저는 자바 개발에 관련 된 모든 프로그램을 C:\Dev 폴더에 설치할 겁니다. 그래서 C:\Dev를 입력했습니다). ❸ [INSTALL] 버튼을 클릭합니다.



- 09 설치 중간에 다음 창이 뜨면 ❶ 다음과 같이 다 체크를 해주고 ❷ [Accept selected] 버튼을 클릭해 설치를 계속 진행합니다.

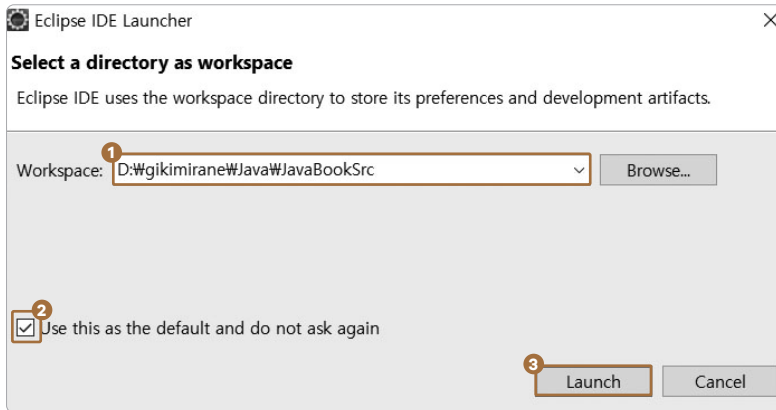


10 설치가 완료되면 [LAUNCH] 버튼을 클릭하여 이클립스를 실행시킵니다.

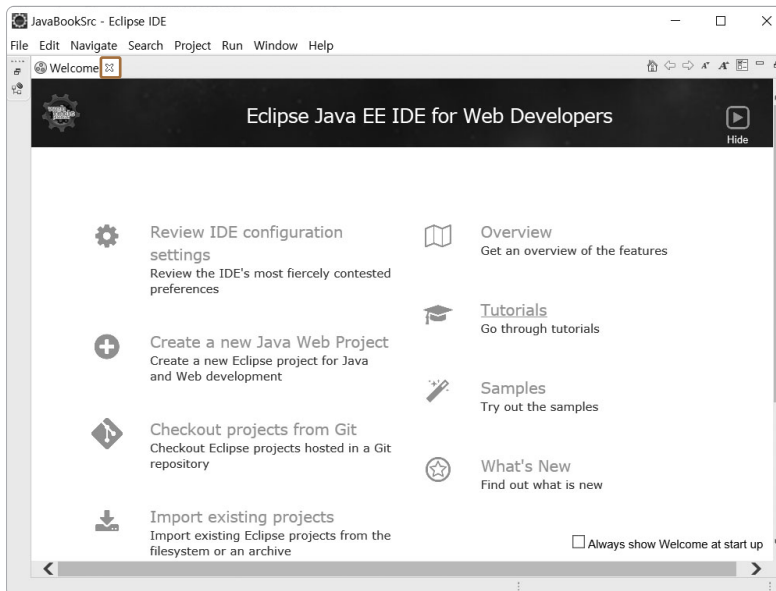


- 11 처음으로 이클립스를 실행하면 워크스페이스를 지정하는 창이 뜹니다. ❶ 원하는 폴더를 선택하세요(기본 경로는 윈도우에 로그인한 사용자의 문서 폴더입니다).

이클립스가 실행될 때 더 이상 워크스페이스의 경로를 물어보지 않도록 ❷ 체크 박스에 체크해주고 ❸ [Launch] 버튼을 클릭합니다.



- 12 드디어 이클립스 IDE가 실행됐습니다. 현재 보이는 Welcome 화면은 ✖를 클릭해서 닫아줍니다.



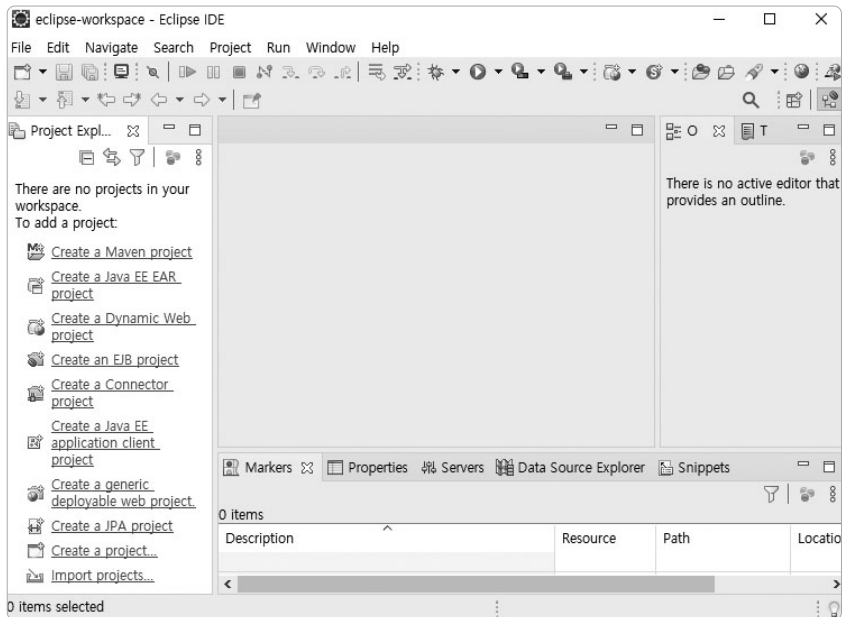
0.3 이클립스 메뉴 및 설정

이클립스를 설치했으니 자바용으로 메뉴 구성을 바꾸고, 화면 구성을 알아보고, 추가 환경 설정을 진행하겠습니다.

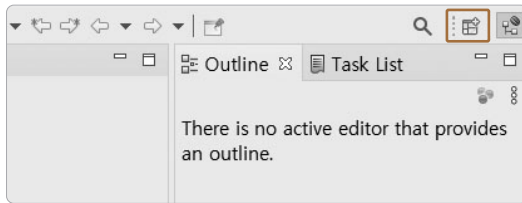
- 1 메뉴 구성 바꾸기
- 2 화면 구성 알아보기
- 3 추가 환경 설정하기

0.3.1 자바용으로 메뉴 구성 바꾸기

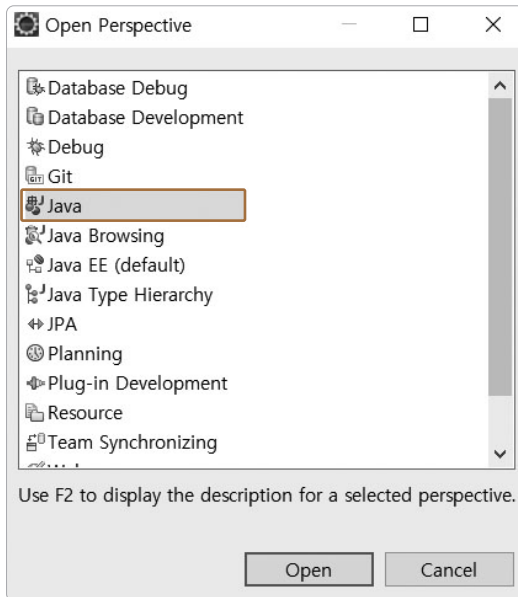
이클립스는 자바 개발뿐 아니라 자바 웹 개발, 다른 언어 개발에도 사용되기 때문에 개발 상황에 맞는 여러 메뉴 구성을 제공합니다. 이클립스의 메뉴 구성을 자바 개발용으로 변경하겠습니다.



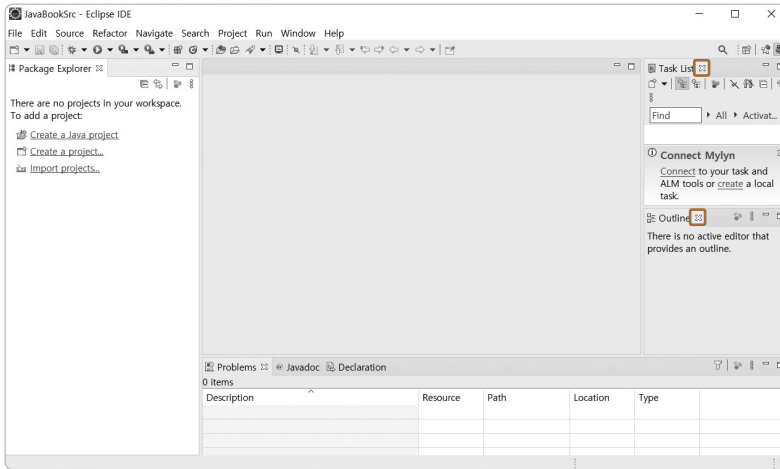
To Do 01 우측 상단의 퍼스펙티브 선택 아이콘을 클릭합니다.



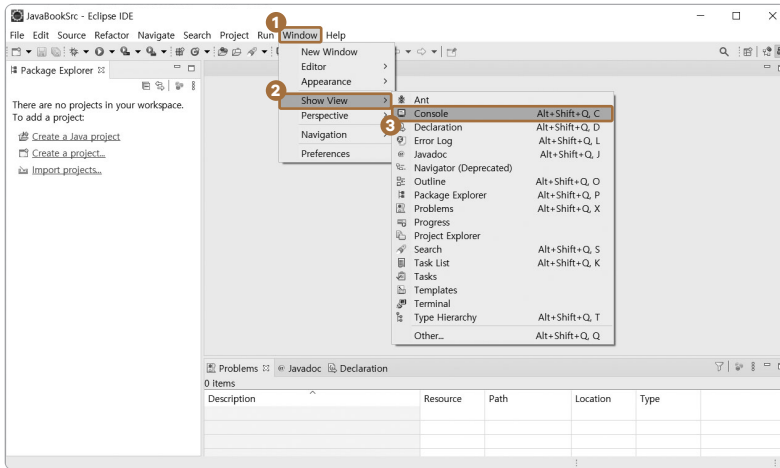
02 퍼스펙티브 창이 뜨면 다음처럼 [Java]를 선택합니다.



- 03** 우측에 있는 창들은 개발 시 유용하게 사용되기도 하지만 초보자들에게에는 불필요하므로 일단 창의  부분을 클릭해 다 닫도록 하겠습니다.



- 04** 콘솔창은 프로그램을 작성하고 실행하면 자연스럽게 나오는 창이긴 하지만 미리 꺼내보겠습니다. 메뉴에서 **1** [Window] → **2** [Show View] → **3** [Console]을 클릭해주세요.

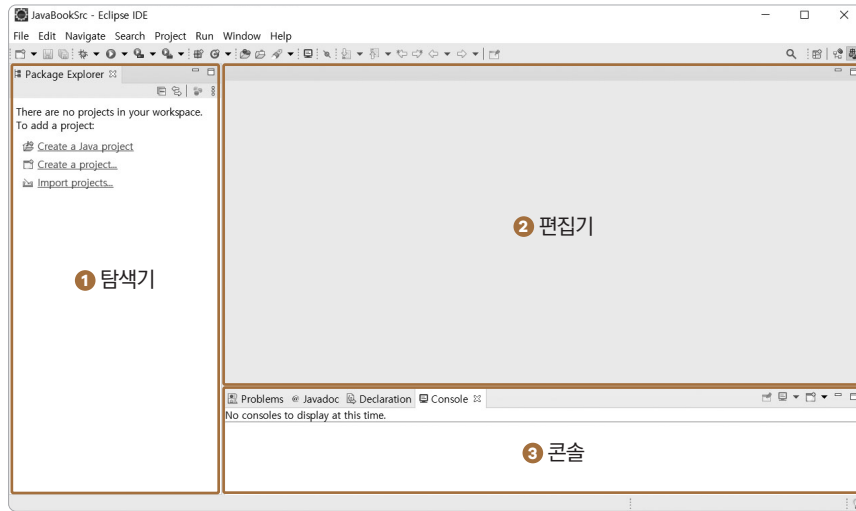


Tip 실수로 어떤 창을 닫게 되면 이 메뉴에서 닫힌 창을 선택해 다시 열면 됩니다.

0.3.2 이클립스 화면 구성 익히기

이클립스의 기본적인 화면 구성을 살펴봅시다.

▼ 이클립스의 기본 화면



❶ 패키지 익스플로러창입니다. 우리가 만든 자바 프로그램을 프로젝트별로 구분해서 보여줍니다. 이 책에서는 ‘탐색기창’으로 부르겠습니다.

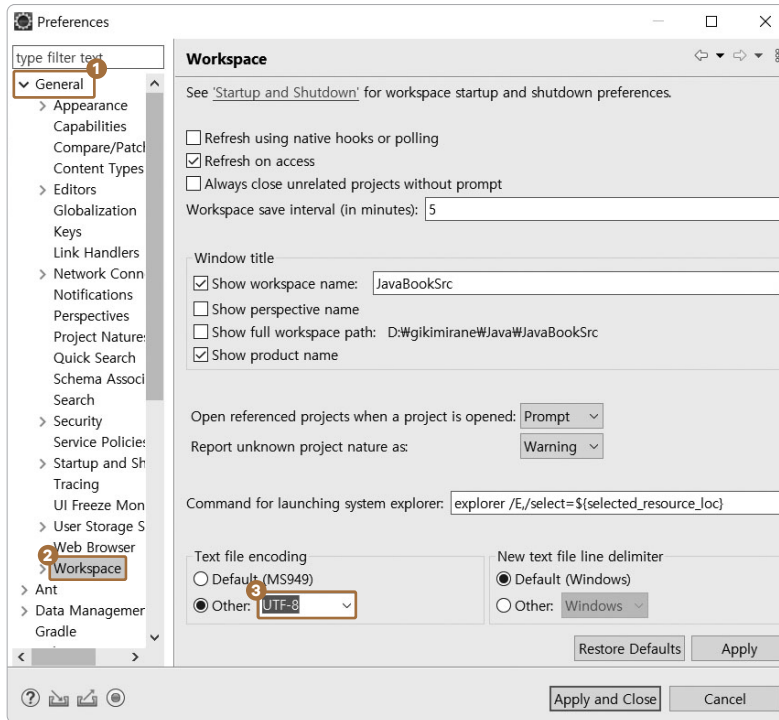
❷ 편집기창입니다. 자바 프로그램을 작성하는 곳입니다.

❸ 자바 코드를 컴파일하고 실행할 때 그 결과를 볼 수 있는 콘솔창입니다. 선수 수업 때 보았던 화면 구성과 거의 비슷합니다.

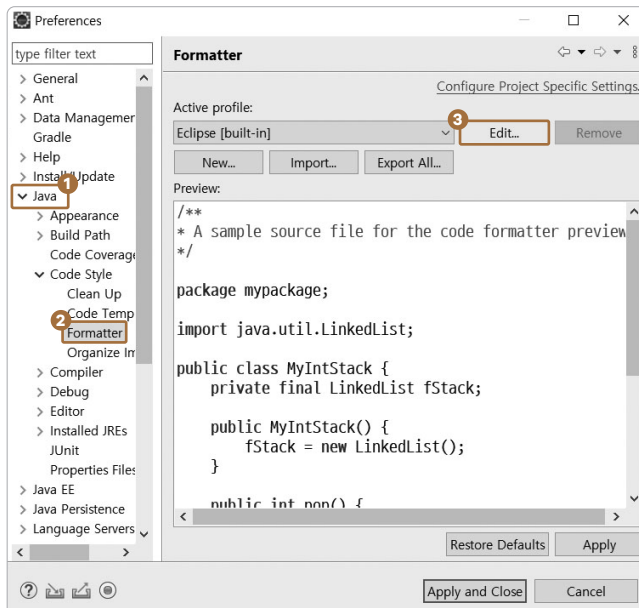
0.3.3 이클립스 추가 환경 설정

To Do 01 메뉴에서 ❶ [Window] → ❷ [Preferences]를 선택합니다.

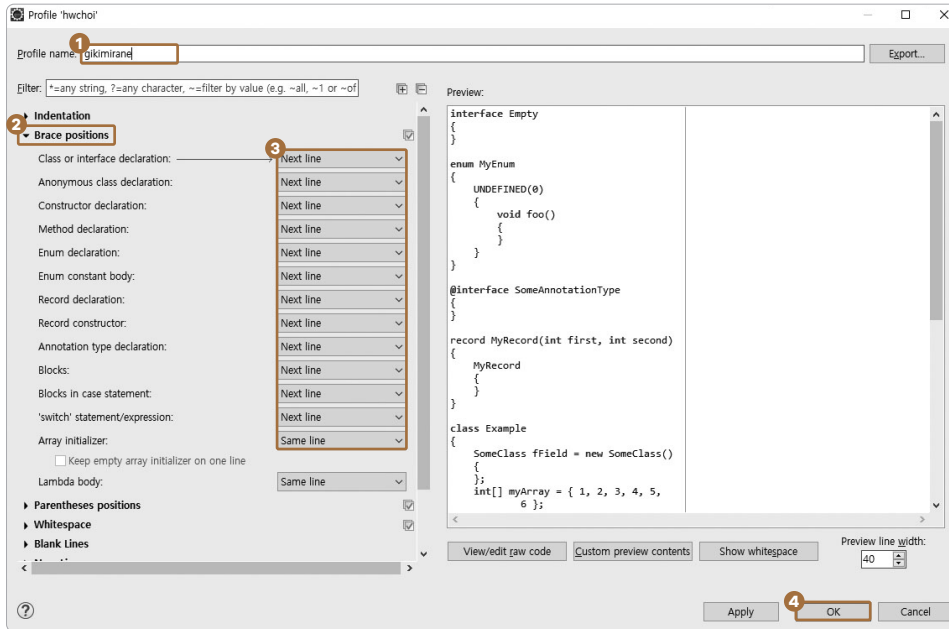
02 문서의 인코딩 타입을 변경합니다. ❶ [General] → ❷ [Workspace] → ❸ [Other]에서 [UTF-8]을 선택합니다.



03 중괄호 위치, 즉 코드 작성 시 자동으로 만들어지는 중괄호 위치를 변경합니다. ❶ [Java] → ❷ [Code Style]에서 [Formatter] → ❸ [Edit...]를 선택합니다.



- 04 ① 프로파일 이름을 본인 아이디 등으로 바꿉니다. ② [Brace positions]를 클릭하여 펼친 후 ③ 개행이 되도록 설정을 바꿔주세요. 설정을 마쳤다면 ④ [OK]를 클릭합니다.



이렇게 해놓으면 코드의 중괄호 위치가 다음과 같이 바뀌게 됩니다.

▼ 기존 코드 생성 시 중괄호 위치

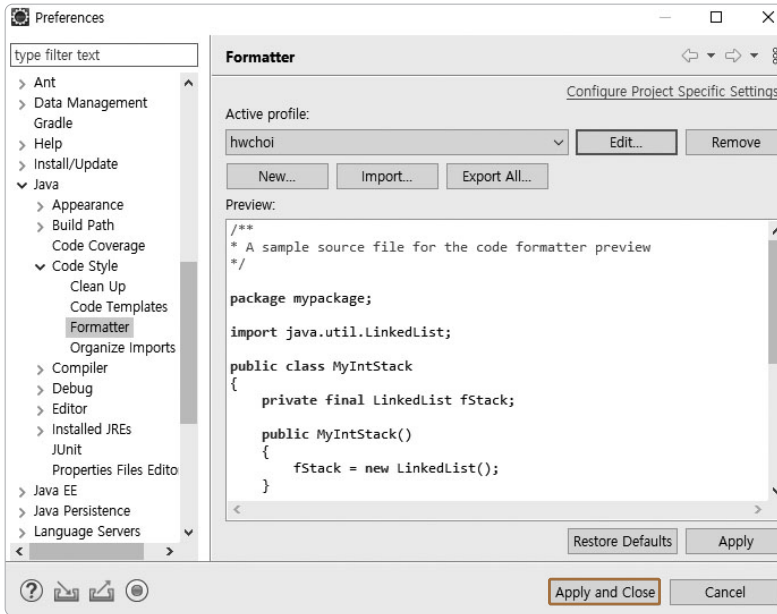
```
for (int i = 0; i < 10; i++) {  
    // 코드;  
}
```

▼ 이후 코드 생성 시 중괄호 위치

```
for (int i = 0; i < 10; i++)  
{  
    // 코드;  
}
```

이 상태로 코드를 작성하면 코드 들여쓰기에 적응하는 데 도움이 될 겁니다. 선수 수업 때는 중괄호를 위·아래로 연결한 지시선이 보였지만 이클립스에서는 나오지 않습니다. 그러므로 열고 닫는 중괄호끼리 가상의 선을 상상으로 그어서 코드에 들여쓰기가 제대로 반영되게 코드를 작성하는 노력을 해야 합니다. 이렇게 노력하다 보면 자연스럽게 들여쓰기에 익숙해지게 될 겁니다.

05 [Apply and Close]를 클릭해 전체 변경 사항을 반영하고 종료합니다.



0.4 예제 코드 다운로드 및 점검

깃허브에서 예제 코드를 다운로드하고 실행하는 방법을 알아보겠습니다.

예제 저장소 위치

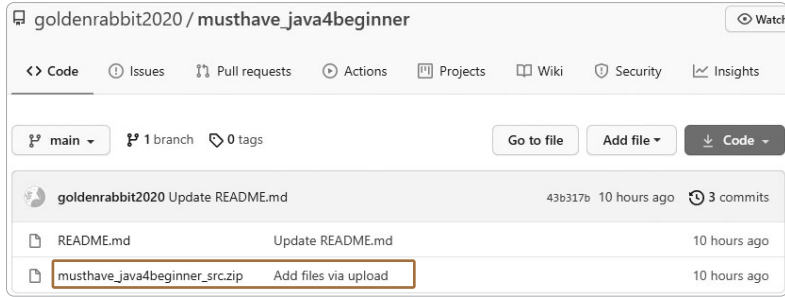
이 책에 등장하는 모든 예제 코드를 깃허브에서 내려받을 수 있습니다. 저장소 위치는 다음과 같습니다.

- 깃허브 : github.com/goldenrabbit2020/musthave_java4beginner

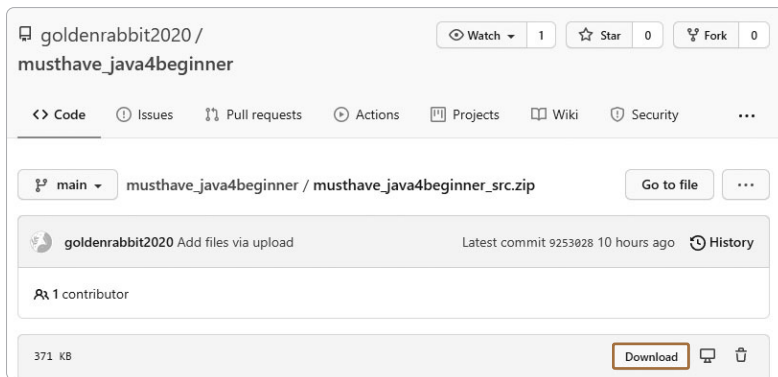
예제 코드 내려받기

01 브라우저로 이 책의 깃허브 URL에 접근하세요.

02 `musthave_java4beginner_src.zip` 파일을 클릭하세요.



03 [Download] 버튼을 클릭하세요.



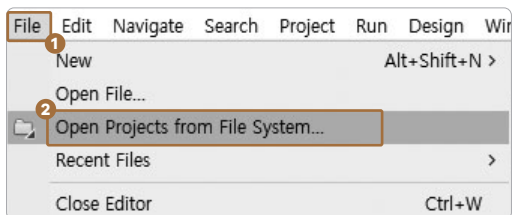
04 내려받은 ZIP 파일을 압축 해제해주세요.

05 압축 해제한 파일을 저장하고 싶은 곳에 위치시켜줍니다.

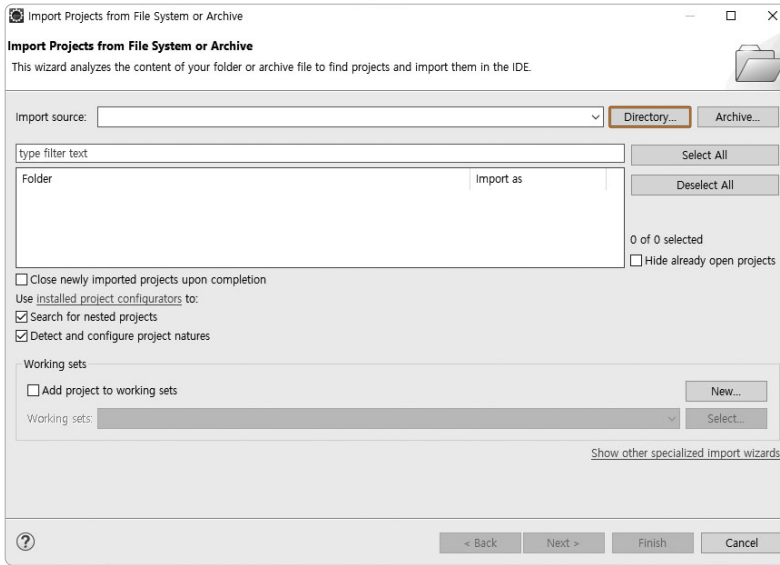
0.4.1 예제 소스 코드 열기

이클립스에서 원하는 파일을 찾는 방법을 알아보겠습니다.

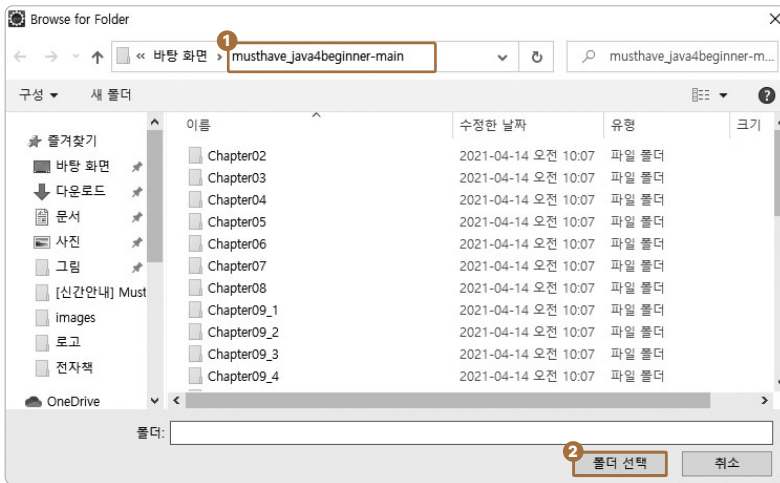
01 메뉴에서 [File] → [Open Project from File System...]을 클릭합니다.



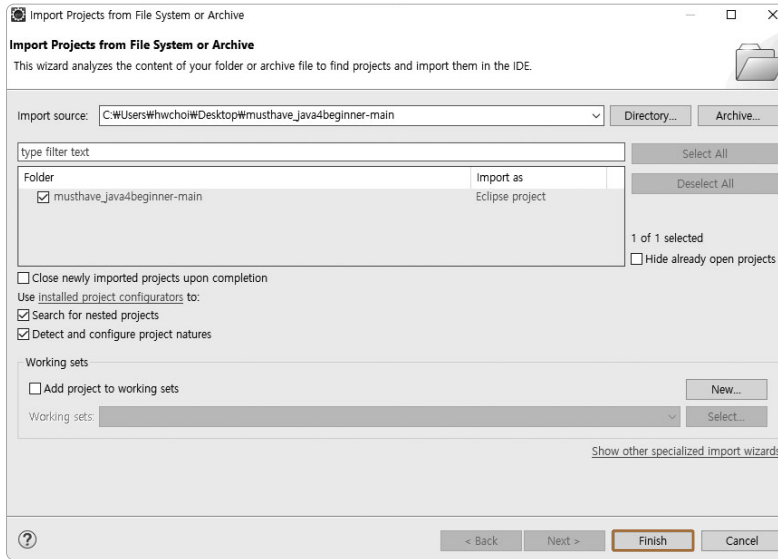
02 팝업창에서 [Directory...] 버튼을 선택합니다.



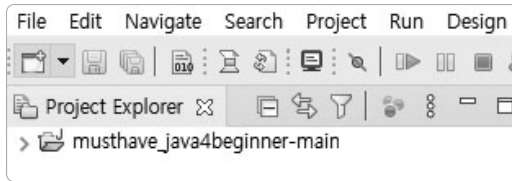
03 ① 압축을 푼 폴더로 이동 후 ② [폴더 선택]을 클릭합니다(압축을 푼 폴더명은 사용자마다 다를 수 있습니다).



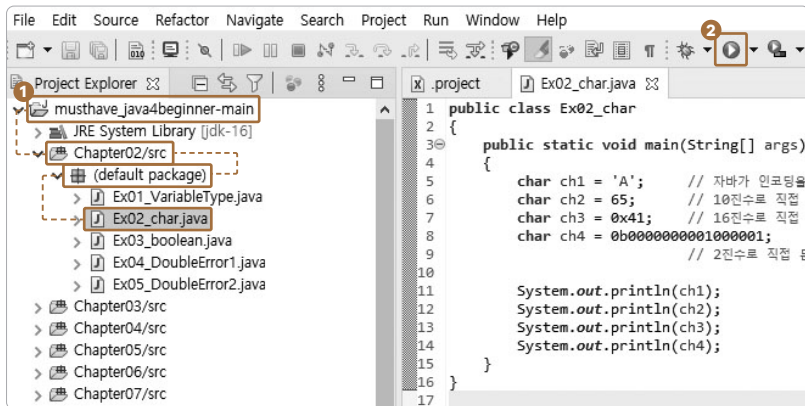
04 [Finish] 버튼을 클릭합니다.



05 [Project Explorer] 창에 다음과 같이 보일 겁니다.



06 ❶ 클릭해서 원하는 소스 코드를 열면 됩니다. ❷ ▶ 버튼을 클릭해 실행할 수 있습니다.



07 이런 방식으로 이 책의 모든 예제를 확인하고 실행할 수 있습니다.

학습 마무리

이번 장에서 로컬 PC에 자바 개발 환경을 만들어보았습니다. JDK 설치, 환경 설정, 이클립스 설치 및 이클립스 환경 설정을 이번 장에서 배웠습니다. 개발 환경을 구축했으므로 이제부터 함께 열심히 공부합시다.

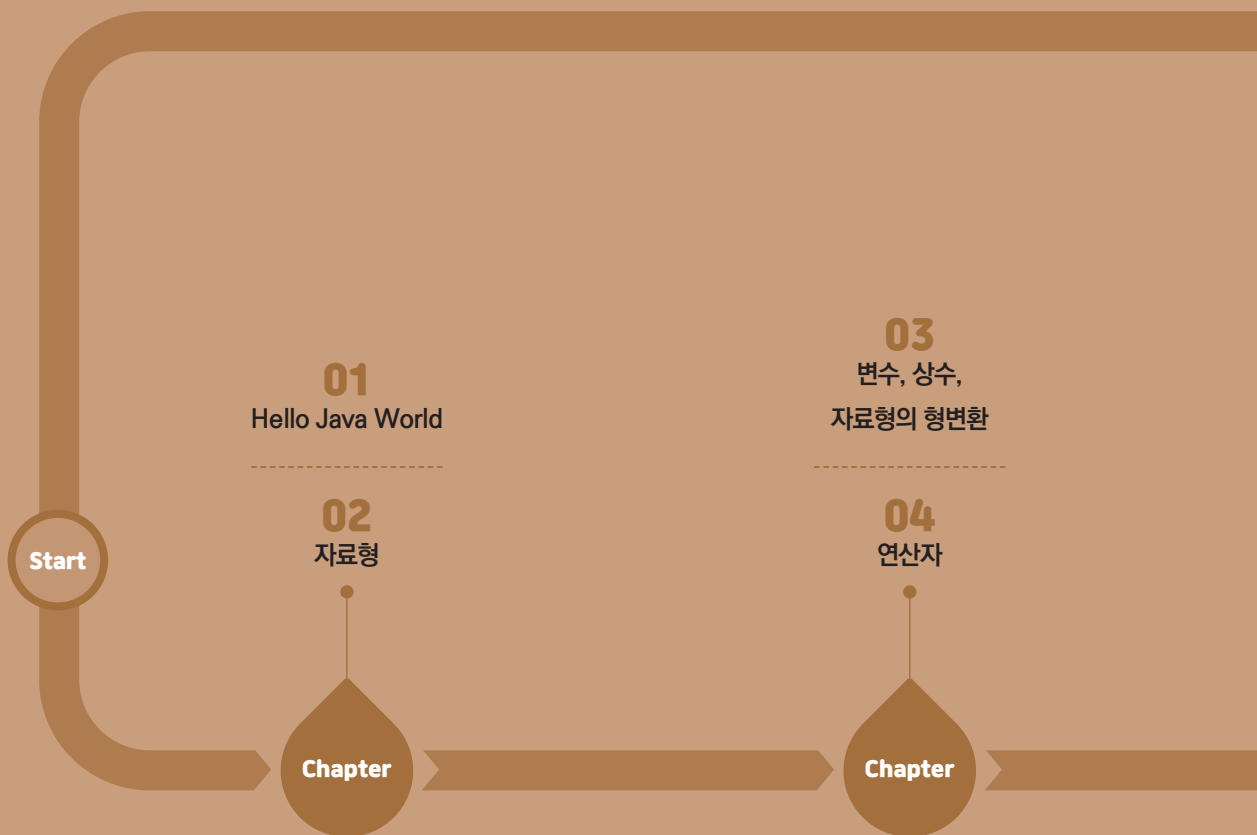
핵심 요약

- 1 JDK는 Java Development Kit의 약자입니다.
- 2 JDK로는 OracleJDK와 OpenJDK가 있습니다. 무료 라이선스인 OpenJDK 11을 설치했습니다.
- 3 이클립스는 통합 개발 환경을 제공하는 도구입니다.

학습 목표

모든 프로그램 언어에서 변수, 상수, 자료형, 표현식, 메서드 표현은 거의 유사합니다.

여기서는 자바를 통해 이런 프로그래밍의 기초를 배우게 됩니다. 여기서 배운 기초는 모든 프로그램 언어에서 거의 그대로 사용할 수 있습니다.



단계

1

자바 기초 프로그래밍

Chapter

05

콘솔 출력과 입력

06

제어문

Chapter

07

메서드와 변수의 사용 가능 범위

08

Project

계산기 만들기 (선수 수업 업그레이드)

Finish

Chapter

01

Hello Java World

#MUSTHAVE

☐ 학습 목표	프로그래밍 언어가 무엇인지, 자바와 JVM은 어떤 관계인지 알아봅니다. 첫 자바 프로젝트도 만들어보고 이클립스 사용법도 알아봅니다.
☐ 학습 순서	1 프로그래밍 언어
	2 자바
	3 JVM
	4 첫 자바 프로젝트 만들기

1.1 프로그래밍 언어

초기의 저수준 프로그래밍 언어부터 현대의 고수준 프로그래밍 언어까지 프로그래밍 언어가 무엇인지 간단하게 알아봅니다.

1.1.1 초기 프로그래밍

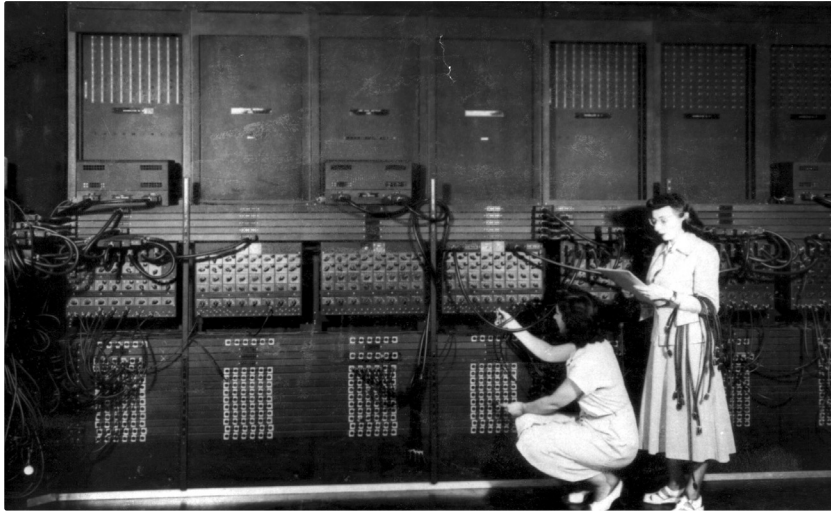
컴퓨터 하드웨어는 미리 정해진 일을 고속으로 처리합니다. 이때 ‘미리 정해진 일’이 프로그램입니다. 프로그램은 미리 정해진 일을 순서대로 컴퓨터 하드웨어에 전기 신호를 보내서 컴퓨터 하드웨어를 동작시킵니다. 컴퓨터 하드웨어에 신호를 보내는 데는 전류를 약하게 보냈다가 강하게 보냈다가 하는 방법을 사용합니다. 이런 처리 절차를 일일이 적으면 길고 복잡합니다. 전류를 약하게 보내는 것을 0, 강하게 보내는 것을 1로 표현한다고만 알면 됩니다.

... 0101000000101001 ...

위와 같이 0과 1로 표현하는 프로그래밍 언어를 기계어라고 합니다. 초기에는 사람이 직접 0과 1로 된 숫자를 보고 코드를 입력했습니다(코드^{code}를 연결했죠).

전기 신호를 한 번에 하나만 보내면 두 가지 동작밖에 구분을 못합니다. 두 가지 상태로는 스위치 역할밖에 시킬 일이 없습니다. 그래서 신호 여덟 개를 묶어서 동작을 구분하게 만들었습니다. 그러면 2의 8승, 즉 256개 동작을 구분할 수 있습니다. 하드웨어에 시킬 수 있는 일이 많아져서 기쁘네요.

▼ 초기 기계어 프로그래밍 : 에니악



출처 : www.columbia.edu

0과 1을 표현하는 한 자리를 비트^{bit}라고 합니다. 비트를 8개를 묶어 1바이트^{byte}라고 합니다.

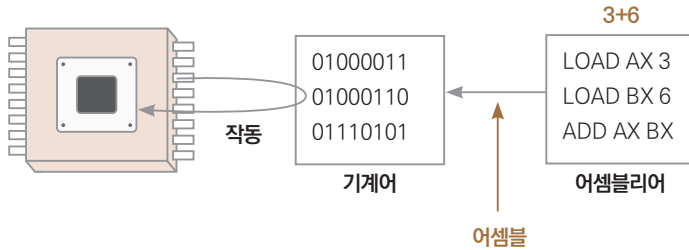


- 1비트 : 신호 하나를 0 또는 1로 처리
- 1바이트 : 8비트 묶음. 컴퓨터에서 정보를 처리하는 기본 단위로 삼음

Tip 동시에 신호를 몇 개 처리하느냐에 따라 32비트(4바이트) 컴퓨터, 64비트(8바이트) 컴퓨터라고 부릅니다.

0과 1로 된 기계어는 알아보고 작성하고 유지보수하기가 어렵기 때문에 기계어에 일대일 대응하는 단어를 이용한 프로그래밍 방식이 나왔습니다. 이것이 어셈블리어입니다.

▼ 기계어와 어셈블리어



연속된 숫자를 보는 것보다는 편하지만 이것도 그렇게 편해 보이진 않습니다.

1.1.2 현재 프로그래밍

어셈블리어보다 더 사람 친화적으로 만든 고수준 언어가 출현했습니다. 1950년대부터 포트란을 필두로 많은 언어가 지속적으로 출현했습니다. 지금도 많이 쓰는 C 언어는 1970년대에, C++는 1980년에, 이 책의 주제인 자바는 1995년에 개발되었습니다. 위 내용을 자바 코드로 보면 다음과 같습니다.

```
int x = 3 + 6;
```

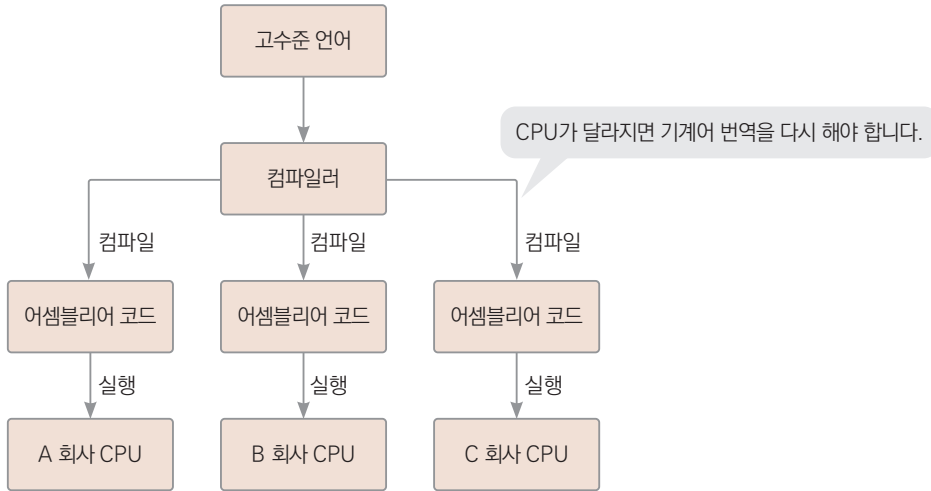
수학 시간에 배운 표현하고 거의 비슷합니다. 기계어보다 이해하기가 훨씬 쉽습니다. 이런 고수준 언어^{high level language}가 나오면서 어셈블리어를 저수준 언어^{low level language}라고 부르게 됩니다. 저수준이라고 해서 수준이 낮다는 뜻은 아니고 사람에게 어렵고 멀게 느껴진다는 뜻입니다.

- 고수준 언어 : 사람에 더 친화적인 언어
- 저수준 언어 : 기계에 더 친화적인 언어(사람보다는 '저 아래 기계쪽에 가깝다'로 이해하세요)

오늘날 위키피디아에는 700개 언어가 등록돼 있고 깃허브에서 370개 언어가 사용됩니다. 1995년 처음 공개된 이후에도 꾸준히 개발자에게 사랑을 받으며 대표적인 프로그래밍 언어로 성장할 수 있었던, 자바만의 특징은 무엇일까요?

CPU마다 CPU를 제어하는 기계어가 다르기 때문에 C/C++ 언어는 CPU가 달라지면 같은 코드라도 해당 CPU에 맞게 다시 컴파일을 해주어야 합니다.

Tip CPU는 제조사마다 만드는 노하우가 다릅니다. 같은 설계도를 가지고 똑같은 성능의 CPU를 만드는 게 아니기 때문에, 이를 컨트롤하는 기계어도 달라지게 됩니다.



또한 운영체제마다 CPU를 컨트롤하는 방식이 다르므로 운영체제가 달라지면 기계어 번역(컴파일)을 다시 해야 합니다. 코드는 하나인데 다양한 플랫폼에서 동작하게 하려면 플랫폼마다 컴파일을 해주어야 하니 여간 불편한 게 아닙니다. 이때 혜성처럼 등장한 언어가 바로 자바입니다. 자바 1.0 버전이 공개되면서 내세운 표어는 'Write Once, Run Anywhere'입니다. '한 번 코드를 작성하면 어떤 운영체제에서도 잘 작동한다'라는 장점을 부각한 표어입니다.

1.2 자바

자바 언어는 1991년에 썬마이크로시스템즈(Sun Microsystems)에서 제임스 고슬링(James Gosling)이 이끄는 팀에서 시작되었습니다. 냉장고, TV 등 가전제품에서 사용되는 컨트롤러 칩의 기능을 프로그래밍하는 언어 개발이 목표였습니다.

가전제품을 만든 제조사도 많고, 각 제조사가 만드는 제품도 다양합니다. 당시에는 C/C++ 언어가 많이 사용되었는데, 다양한 플랫폼마다 매번 다른 기계어로 컴파일해야 하는 기존 언어와 달리 플랫폼 독립적인 기능이 필요했습니다.

또한 네트워크에서 자동으로 내려받는 기능이 필요한데, 악성 코드 침투를 막으려면 포인터 개념도 없어야 했습니다(이 점에서 C/C++보다 안전하다는 말을 듣게 됩니다). 이런 필요성에 의해 자바가 개발된 겁니다.

썬마이크로시스템즈는 자바의 첫 번째 정식 버전인 자바 1.0 버전을 1995년에 공개합니다. 그런데 썬마이크로시스템즈에서 자바를 관리할 때 자바를 오픈 소스로 공개하라는 많은 요청이 있었습니다. 그래서 썬마이크로시스템즈는 JDK 소스 코드를 GPL 라이선스로 공개하고 OpenJDK라는 오픈 소스 프로젝트를 만들었습니다. 그리고 OpenJDK는 버전 7 이후로 자바 SE의 공식 레퍼런스 구현체가 됩니다. 이후 썬마이크로시스템즈가 2010년 1월 오라클에 인수된 후 썬마이크로시스템즈에서 관리하던 자바는 오라클에서 관리하게 됩니다.

▼ 자바 버전

자바 버전	발표 일자	비고
JDK 1.0	1996.01	Java Development Kit
JDK 1.1	1997.02	
J2SE 1.2	1998.12	JDK를 Java 2 Platform Standard Edition으로 변경 JAVA 2
J2SE 1.3	2000.05	JAVA 3
J2SE 1.4	2002.02	JAVA 4
J2SE 5.0	2004.09	JAVA 5
JAVA SE 6	2006.12	J2SE를 JAVA SE로 변경 JAVA 6
JAVA SE 7	2011.07	JAVA 7
JAVA SE 8	2014.03	LTS 버전으로 2025년 3월까지 기술 지원. 최초로 자바에 람다식이 도입된 버전으로 JAVA 8이라고 부름
JAVA SE 9	2017.09	non-LTS. 이후 64비트만 지원
JAVA SE 10	2018.03	non-LTS
JAVA SE 11	2018.09	LTS 버전. 2026년 9월까지 기술 지원
JAVA SE 12	2019.03	non-LTS
JAVA SE 13	2019.09	non-LTS
JAVA SE 14	2020.03	non-LTS
JAVA SE 15	2020.09	non-LTS

오라클은 Java SE 8 버전(이하 자바 8)까지는 10년 이상 기술 지원을 제공했습니다. 그런데 Java SE 9부터 짧은 기술 지원(6개월)과 긴 기술 지원^{LTS, long term support}으로 나눠서 발표하고 있습니다. 현재 JAVA SE 16까지 발표되었지만, 자바 8과 자바 11만 LTS로 지정되었습니다.

2021년 02월 현재 Java SE 8의 버전은 Java SE 8u281입니다. 버전 끝에 붙는 281은 버그를 수정하고 성능을 개선한 281번째 업데이트라는 뜻입니다. LTS 버전에는 이렇게 장기간의 기술 지원이 이루어집니다. 따라서 LTS 버전으로 공부하고 프로젝트를 진행하는 것이 현명합니다.

Tip 버그 수정과 성능 개선만 이루어지면서 주된 내용이 변하지 않는 이유로 많은 책에서 Java SE 8을 선택했습니다. 많은 회사에서도 실무적으로 Java SE 8을 사용했습니다. 오랜 기간 안정적인 운영 및 유지보수가 가능했기 때문입니다.

또한 오라클은 Java SE 9부터 라이선스 정책도 유료로 변경했습니다. 비상업 용도로는 여전히 무료로 사용할 수 있지만, 상업용으로 사용할 때는 라이선스를 지불해야 합니다. 이런 라이선스 정책으로 인해 많은 개발자와 회사에서 OpenJDK를 사용합니다.

OpenJDK 11은 현재 맥OS, 리눅스에 설치되는 기본 버전입니다. 우리가 선수 수업에서 사용했던 콘솔창에도 OpenJDK 11이 설치되어 있었습니다. 우리도 앞서 개발 환경을 구축할 때 OpenJDK 11을 다운받아서 개발 환경을 구축했습니다. OpenJDK를 더 알고 싶다면 openjdk.java.net을 참고하면 됩니다.

1.3 JVM

자바는 ‘Write Once, Run Anywhere’를 위해 자바 가상 머신^{JVM, Java Virtual Machine}을 사용합니다. 이를 위해 인기 있는 플랫폼마다 실행되는 java 실행 파일을 제공합니다. 자바는 실행 파일인 java가 실행될 때마다 소프트웨어적으로 가상 머신을 즉시 만듭니다. 그리고 그 자바 가상 머신에서 우리가 만든 바이트 코드를 실행시켜주는 겁니다.

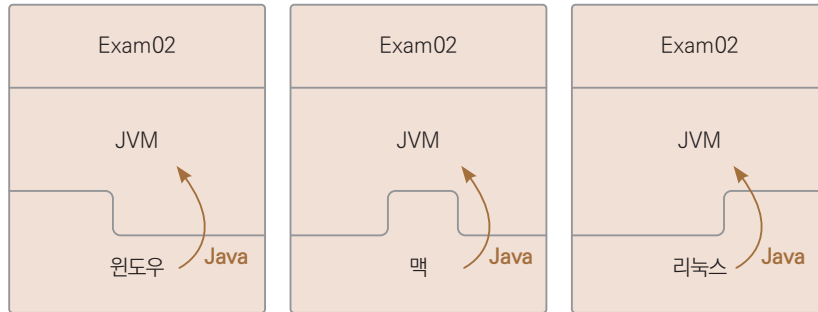
여러분이 선수 수업에서 사용했던 java를 기억해봅시다. 프롬프트에 ‘java Exam01’이라고 입력하고 **Enter**를 치면 자바 프로그램이 실행되었습니다. 리눅스 운영체제에서 java가 실행되면서 자바 가상 머신을 소프트웨어적으로 구현을 하고 그다음에 우리가 만든 바이트 코드인 Exam01이 자바 가상 머신 위에서 실행된 겁니다.

윈도우에서도 cmd를 이용해서 명령 프롬프트 창을 열고 java Exam01이라고 입력해서 우리가

만든 자바 프로그램을 실행할 수 있습니다. 이때도 역시 윈도우용 실행 파일인 java.exe가 가상의 자바 실행 환경을 소프트웨어적으로 먼저 만들어주고, 즉 자바 가상 머신을 만들고 그 안에서 우리가 만든 바이트 코드를 실행시켜주는 겁니다.

그림으로 표현하면 다음과 같습니다.

▼ 플랫폼에 독립적인 자바 언어



Tip 가상 머신을 여러분이 쉽게 이해하려면 맥을 사용하는 사람이라면 패럴렐즈(Parallels), 안드로이드 게임을 PC에서 하는 사람이라면 블루스택(BlusStacks)이나 노스(Nox)를 생각하시면 됩니다. 이 외에도 VirtualBox나 VMware 등을 생각하시면 됩니다. 이런 가상 머신을 지원하는 프로그램들은 해당 소프트웨어를 (영구적으로) 설치하고 다른 운영체제를 추가 설치해서 사용합니다.

이렇게 자바의 실행 환경인 자바 가상 머신을 알아보았습니다.

1.4 첫 자바 프로젝트 만들기

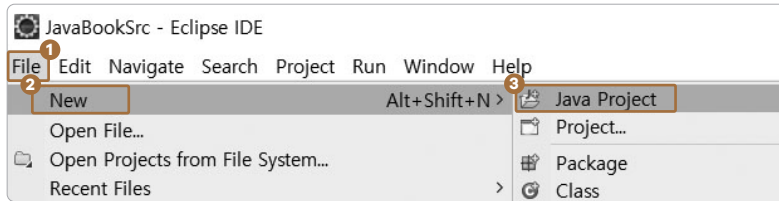
선수 수업 때는 프로그램을 만들 때 파일 하나를 만들고 코드를 작성했지만, 이클립스에서는 파일 대신 프로젝트를 단위로 사용합니다. 프로젝트 하나에는 자바 파일이 하나 또는 여러 개일 수 있습니다. 즉, 클래스 여러 개를 묶어서 하나의 큰 프로그램으로 구성할 수 있게 하는 겁니다. 이제부터 다음과 같이 두 가지를 알아봅니다.

- 1 첫 프로젝트 만들기
- 2 에러를 찾는 간단한 방법

1.4.1 첫 프로젝트 만들기

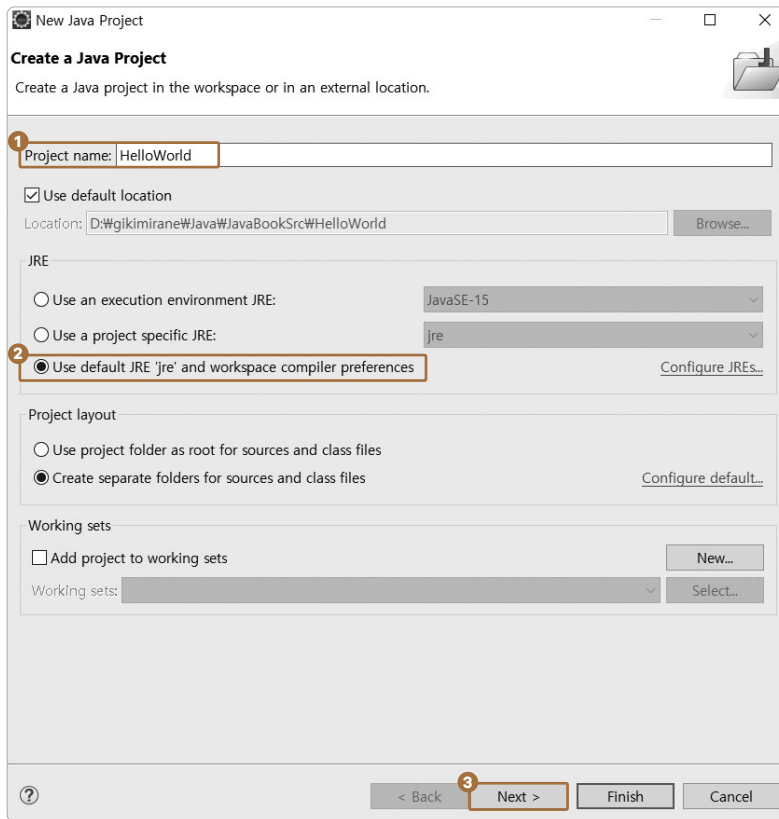
“Hello”를 출력하는 간단한 프로젝트를 만들어보겠습니다.

To Do 01 메뉴에서 ❶ [File] → ❷ [New] → ❸ [Java Project]를 선택합니다.

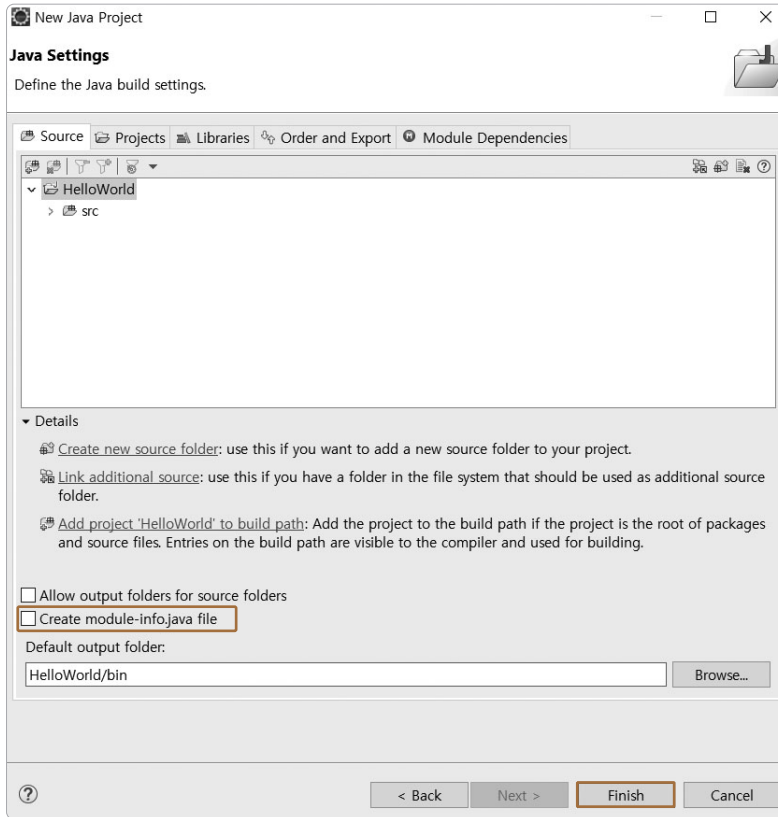


Tip 메뉴 구성이 그림과 다르다면 퍼스펙티브 설정이 제대로 안 된 겁니다. 퍼스펙티브 뷰에서 [Java]를 선택해주세요.

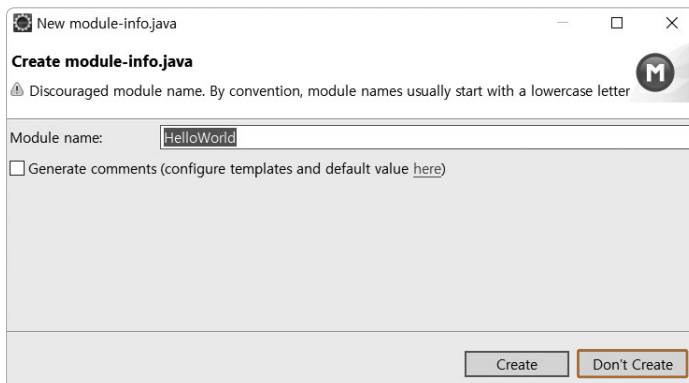
02 ❶ 프로젝트 이름으로 HelloWorld를 입력하고 ❷ JRE는 세 번째 버튼을 선택해주세요. ❸ [Next >]를 선택합니다.



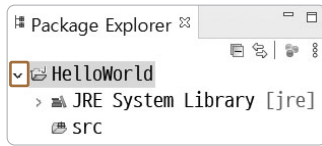
03 ① [Create module-info.java file] 체크를 해제합니다. 자바 9에서 추가된 내용을 프로젝트 생성 시 반영하는 옵션인데 이 책에서는 사용하지 않습니다. **②** [Finish]를 클릭합니다.



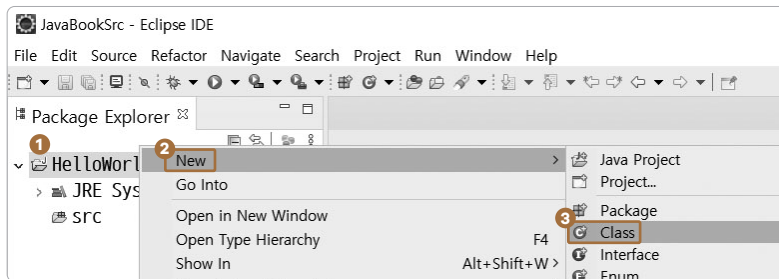
혹시라도 **02**에서 [Finish]를 눌렀다면 다음과 같은 창이 뜹니다. 이때는 [Don't Create] 버튼을 선택하면 됩니다.



04 탐색기창에서 > 를 클릭하면 ▼로 모양이 바뀌면서 하위 상세 내용이 펼쳐집니다.

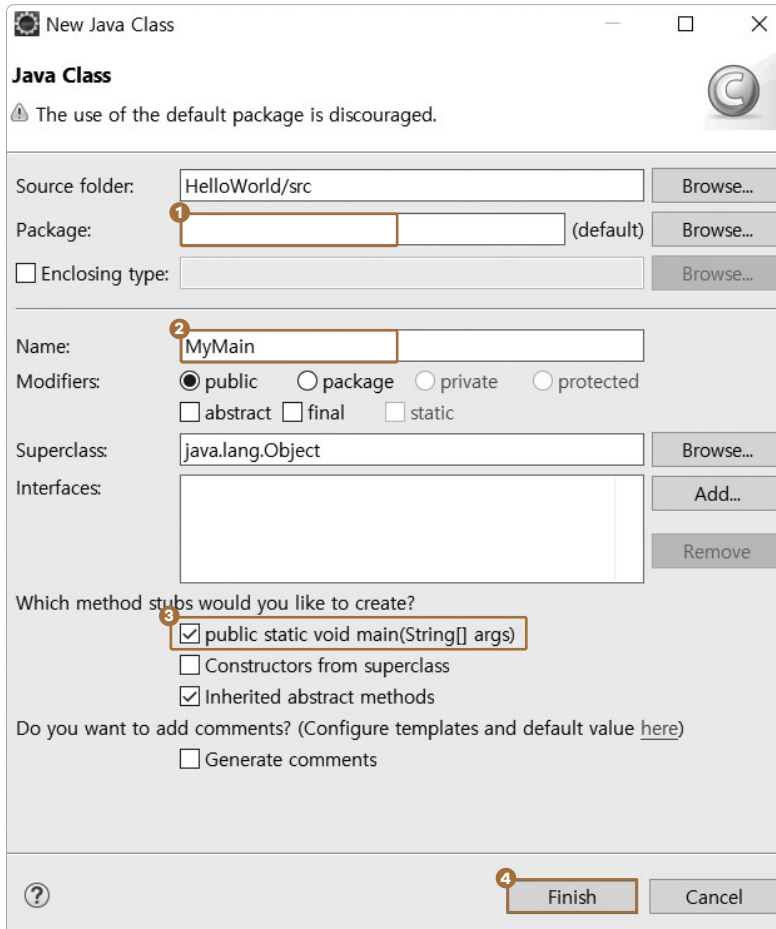


05 ❶ 프로젝트명(여기서는 HelloWorld) 위에서 마우스 우클릭 → 팝업에서 ❷ [New] → ❸ [Class]를 선택합니다.



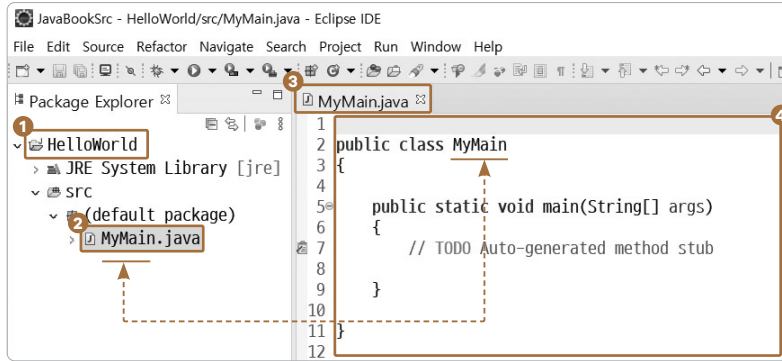
06 클래스 정보를 입력하여 클래스 파일을 생성합니다.

- ❶ Package : 패키지명은 입력하지 않습니다. 기존의 것이 있다면 지웁니다.
- ❷ Name : MyMain을 입력합니다.
- ❸ 클래스 파일이 만들어질 때 main()이 자동으로 만들어지게 체크합니다.
- ❹ [Finish]를 눌러 클래스 파일을 생성합니다.

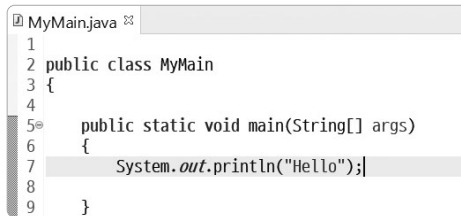


07 클래스 파일이 만들어진 모습을 살펴보겠습니다.

탐색기창에 ❶ HelloWorld 프로젝트가 있고, ❷ MyMain.java 파일이 프로젝트 안에 생성되었습니다. 편집기창에는 ❸ MyMain.java 파일이 생성되면서 ❹ 자동 생성된 코드가 보입니다. 파일명과 클래스명이 대소문자까지 같은 것을 볼 수 있습니다. 선수 수업에서 본 클래스 파일의 내용과 구성이 같습니다.

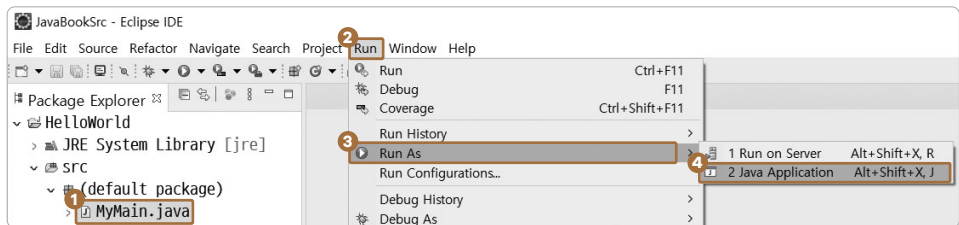


- 09 ① 주석을 지우고 “Hello”를 출력하는 코드를 입력하고 ② 메뉴에서 [File] → [Save]를 클릭해 저장합니다.

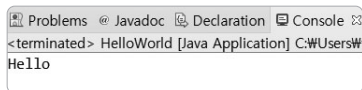


Tip 저장 단축키 : **Ctrl + S**

- 10 이클립스는 클래스 파일을 저장할 때 자동으로 컴파일을 수행하므로 에러 없이 컴파일되었다면 에러 표시가 안 보일 겁니다. 실행을 해보겠습니다. ① 클래스 파일을 선택하고 메뉴 ② [Run] → ③ [Run As] → ④ [Java Application]을 선택합니다.



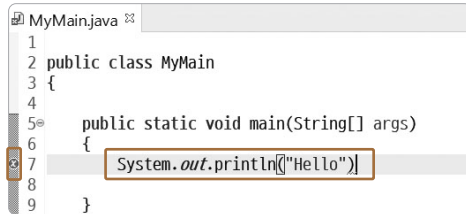
- 11 하단 콘솔창에 실행 결과가 표시됩니다.



이클립스로 만든 첫 자바 프로젝트(안의 클래스)가 잘 실행되었습니다. 축하합니다.

1.4.2 이클립스에서 에러 찾기

HelloWorld 프로젝트에 일부러 오류를 만들어봅니다. ❶ 문장의 끝에 세미콜론을 삭제합니다. ❷ 행 번호 앞에 에러 표시가 나타납니다. 이 표시를 참고하면 쉽게 에러를 발견할 수 있습니다.



```
1 public class MyMain
2 {
3
4     public static void main(String[] args)
5     {
6         System.out.println('Hello')
7     }
8 }
9
```

이클립스는 클래스 파일을 저장할 때 자동으로 컴파일을 수행합니다. 그러므로 저장할 때 에러를 발견한 라인 번호 앞에 에러 표시를 보여줍니다.

학습 마무리

지금까지 자바 프로그래밍 언어의 발전사와 JVM을 알아보았습니다. 이클립스를 사용해 프로젝트를 만들고 에러를 찾는 방법은 책 전반에 사용합니다. 꼭 익혀두세요.

핵심 요약

- ❶ 고수준 언어 : 사람에게 더 친화적인 언어
- ❷ 저수준 언어 : 기계에 더 친화적인 언어
- ❸ JVM은 자바가 실행되는 가상머신 환경입니다.
- ❹ 이클립스는 저장하면서 자동으로 컴파일도 진행합니다. 에러가 발생한 행 번호 앞에 를 표시합니다.



이재환의 자바 프로그래밍 입문

설치 없이 익히는 《선수 수업》부터 딱 필요한 만큼
핵심 문법과 미니 프로젝트까지

초판 1쇄 발행 2021년 08월 01일

지은이 이재환

펴낸이 최현우 · 기획 최현우 · 편집 최현우, 이복연

디자인 Nu:n · 조판 Nu:n

펴낸곳 골든래빗(주)

등록 2020년 7월 7일 제 2020-000183호

주소 서울 마포구 신촌로2길 19, 302호

전화 0505-398-0505 · 팩스 0505-537-0505

이메일 ask@goldenrabbit.co.kr

SNS facebook.com/goldenrabbit2020

정가 20,000원

ISBN 979-11-971498-7-0 95000

* 파본은 구입한 서점에서 바꿔드립니다.

딱 필요한 만큼, 자바 핵심 문법과 개념 이해 중심으로 배우는 새로운 자바 입문서

12년간 강의를 해보니 무엇을 이해하지 못하는지, 무엇을 어려워하는지, 현업 나가서 당장 쓸모 있는 기법과 그렇지 못한 기법이 무엇인지 알게 되었습니다. 프로그래머처럼 생각하고, 프로그래머로 안착할 분께 딱 필요한 만큼 알려드립니다. 초보자도 쉽고 명확하게 이해할 수 있도록 학습 목표를 일목요연하게 제시하고 핵심 내용을 정리해 보여줍니다. 이론도 실습으로 직접 체험하여 체득하게 했습니다. 단계별로 간단한 프로젝트를 구현합니다. 프로그래밍에 입문해 프로그래머로 취업하고 싶은 분들이라면 자바에 도전해보세요.

0 아무것도 몰라도 OK

프로그래밍의 ‘표’도 모르는 분도 코딩이 가능하도록 설명합니다.

12 가지 선수 수업 예제

설치 없이 코딩하는 선수 수업을 만나보세요. 12가지 예제를 풀다 보면 자바에 더 친숙해질 겁니다.

4 단계로 익히는 자바

선수 수업, 자바 기초 프로그래밍, 자바 객체지향 프로그래밍, 자바 클래스 응용 프로그래밍을 다룹니다.

“스터디를 하면서 초보자에게 프로그래밍을 가르쳐 준 적이 있는데, 만약 이 책이 있었다면 이 책으로 진행했을 겁니다. ‘처음부터 코딩이 가능하고, 실제 동작하는 예제들이 있어서 지루하지 않은 이 책이 있었다면 스터디에 큰 도움이 되었을 텐데’하는 생각이 드네요.”

이호훈 TVING 프로그래머

“어려운 용어보다는 현실과 가까운 용어로 설명합니다. 마치 멘토를 실제로 만나서 듣는 듯한 설명이 와닿습니다. 헛갈리는 개념은 무조건 다 집어주고, 배운 내용으로 만들 수 있는 프로젝트가 소소한 재미를 선사합니다.”

강은혜 University of the Potomac 학생

환경 구축

0 단계

입문

1 단계

객체지향

2 단계

응용
프로그래밍

3 단계

IT전문서 / 프로그래밍 언어

979-11-971498-6-3 9 3 0 0 0



정가 25,000원 ISBN 979-11-971498-6-3